



Physical Aimbot สำหรับเกม FPS ด้วย Computer Vision

นายภณลภัส สุทธิมาลา รหัสนักศึกษา 65340500046

โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
สถาบันวิทยาการหุ่นยนต์ภาคสนาม
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี
ปีการศึกษา 2567

สารบัญ

บทที่ 1 บทนำ	6
1.1 ที่มา ความสำคัญ	6
1.2 ประโยคปัญหาทางานวิจัย (Problem Statement)	6
1.3 ผลผลิตและผลลัพธ์ (Outputs and Outcomes)	7
ผลผลิต	7
ผลลัพธ์	7
1.4 ความต้องการของระบบ (Requirements)	7
1.5 ขอบเขตของงานวิจัย (Scopes)	7
1.6 ข้อกำหนดของงานวิจัย (Assumptions)	8
1.7 ขั้นตอนการดำเนินงาน	8
บทที่ 2 ทฤษฎี/งานวิจัย/การศึกษาที่เกี่ยวข้อง	10
2.1 Computer Vision สำหรับ Object Detection	10
2.1.1 หลักการทำงานพื้นฐานของ YOLO (You Only Look Once)	10
2.1.2 การเปรียบเทียบรุ่น YOLO (YOLO Version Comparison)	12
2.2 Robotics และการควบคุมการเคลื่อนที่	14
2.2.1 Omni Wheels และ Mecanum Wheels	14
2.2.2 ทฤษฎี Inverse Kinematics สำหรับล้อ Mecanum	14
2.2.3 PID Controller สำหรับการควบคุมความเร็วของมอเตอร์	15
2.3 Hardware และการสื่อสารข้อมูล	17
2.3.1 ESP32 Microcontroller	17
2.3.2 การสื่อสารข้อมูลผ่าน PySerial	17
2.3.3 Motor และ Driver	18
2.3.3.1 ข้อสังเกตเพิ่มเติมจากการใช้งานจริงของมอเตอร์ Yellow TT (1:48)	19
บทที่ 3 ระเบียบวิธีวิจัย	21
3.1 ภาพรวมของระบบ (System Overview)	21
3.2 วิธีการบันทึกวิดีโอการเล่นโปรแกรม Kovaak	22
3.2.1 วิธีการบันทึกวิดีโอการเล่นโปรแกรม Kovaak	22
3.2.1 การดึงภาพจากวิดีโอด้วย OpenCV และขั้นตอนการ Label ข้อมูลเป้าหมาย (Object Labeling)	23
3.2.3 การทำ Augmentation ผ่าน Roboflow	23
3.2.4 การสร้าง Dataset v2 (Scenarios ขยาย + Quality Control)	24
3.2.4.1 Pre-processing	25
3.2.4.2 Augmentation (7x per source)	26

3.3 การฝึกและประเมินผลโมเดล YOLO (Model Training and Evaluation)	26
3.3.1 สภาพแวดล้อมที่ใช้ในการฝึกโมเดล (Hardware Spec, Software Environment, Libraries)	26
3.3.1.1 การย้ายและปรับปรุงสภาพแวดล้อมการฝึกโมเดล	27
3.3.2 ขั้นตอนการ Train โมเดล YOLOv5 (เวอร์ชัน Small) และการตั้งค่าพารามิเตอร์ (Hyperparameter Tuning)	27
3.3.2.1 การฝึก YOLOv5s รุ่นแรก (Training V1 – Dataset v1)	27
3.3.3 การประเมินผลโมเดล (Evaluation Metrics)	29
3.3.3.1 ผลลัพธ์ YOLOv5s รุ่นแรก (Training V1)	29
3.3.3.2 ผลลัพธ์ Training V2 (Dataset v2)	31
3.3.4 ปัญหา & แนวทางแก้ไข	33
3.3.4.1 ปัญหาในการใช้งาน Weight ระหว่าง Windows และ Linux	33
3.3.4.2 ปัญหา VRAM Overflow บน RTX 3060	33
3.4 การออกแบบและพัฒนาฮาร์ดแวร์ (Hardware Design and Development)]	34
3.4.1 รายละเอียดการออกแบบ CAD	34
3.4.2 รายการอุปกรณ์ที่ใช้ (Bill of Materials - BOM)	35
3.4.3 กระบวนการประกอบชิ้นส่วนจริงตามแบบ CAD	36
3.4.4 การทดสอบระบบการเคลื่อนที่ของหุ่นยนต์เบื้องต้น (Preliminary Mobility Test)	40
3.5 การพัฒนาแอปพลิเคชันสำหรับการตรวจจับเป้าหมาย	42
3.5.1 การพัฒนาแอปพลิเคชันสำหรับการตรวจจับเป้าหมาย (Target Detection App Development)	42
3.5.2 การพัฒนาและปรับปรุงระบบการควบคุมคลิก (Clicking Control Refinement)	45
3.6 การผสมผสานระบบ (System Integration)	47
3.6.1 การทดสอบการส่งข้อมูลผ่าน PySerial เบื้องต้น	47
3.6.2 การทดสอบและปรับเทียบค่าควบคุมความเร็วขั้นต่ำของมอเตอร์ (Minimum Speed Calibration)	49
3.6.3 การพัฒนาโปรแกรมควบคุมมอเตอร์ ESP32 แบบ Mapping ความเร็วจากเปอร์เซ็นต์	50
3.6.4 การคำนวณ Inverse Kinematics และการผสมผสานระบบเบื้องต้น	54
3.6.5 การพัฒนาและปรับแต่ง PID Controller สำหรับการควบคุมทิศทางและระยะ	56
3.6.6 สรุปการผสมผสานระบบและการทำงานของระบบจริง (Final Integration and System Completion Summary)	64
บทที่ 4 การทดลองและผลการทดลอง/วิจัย	67
4.1การทดสอบซอฟต์แวร์ (Software Testing)	67
4.1.1 การทดสอบความสามารถในการตรวจจับวัตถุจากวิดีโอ (Object Detection Performance)	67
4.1.2 การทดสอบ Latency บน RTX 3060 (Inference Latency Testing)	72
4.1.3 การทดสอบผลกระทบต่ออัตราเฟรม-เรตของเกม (In-game FPS Impact Test)	74
4.1.4 สรุปผลการทดสอบเบื้องต้น (Overall Test Summary)	76
4.2 การทดสอบฮาร์ดแวร์ (Hardware Testing)	78
4.3 การทดสอบระบบรวมในสภาพแวดล้อมจริง (Full-System Integration Test in Real Scenarios)	80
4.3.1 การทดสอบเปรียบเทียบผลลัพธ์ของโมเดลในสถานการณ์จริง (Model Integration Testing with Real Scenarios)	80
4.3.2 การทดสอบ PID บนสถานการณ์จริง (PID Validation in Real Scenarios)	85
4.3.3 สรุปผลการทดสอบระบบรวม (Full-System Integration Summary)	91
บทที่ 5 บทสรุป	93

5.1 สรุปวัตถุประสงค์และขอบเขตของโครงการ	93
5.2 สรุปการออกแบบและการพัฒนา	95
5.2.1 ระบบตรวจจับเป้าหมาย (Object Detection with YOLOv5)	95
5.2.2 ระบบควบคุมการเคลื่อนที่ (Motion Control with PID)	96
5.2.3 ระบบการเชื่อมต่อและควบคุมฮาร์ดแวร์ (Integration & Control)	96
5.3 สรุปผลการทดลอง	97
5.3.1 ระบบตรวจจับเป้าหมาย (Object Detection with YOLOv5)	97
5.3.2 ระบบควบคุม PID (PID Controller Performance)	97
5.3.3 ระบบรวม (Full-System Integration)	98
5.4 ปัญหาและข้อจำกัดของระบบ	99
5.5 แนวทางการพัฒนาในอนาคต	100
5.6 สรุปภาพรวมของโครงการ	103
เอกสารอ้างอิง	105
ภาคผนวก	107
ภาคผนวก A : Source Code	107
GitHub	107
extract_frame.py	107
test.cpp	109
minspeedtest.py	112
frame_count.py	114
modeltest.py	116
modeltest_latency.py	119
modeltest_runwithgame.py	122
test_latency.py	124
main.cpp	126
annotator.py	130
capture.py	132
clicker.py	133
config.py	135
control.py	136
detect.py	137
distance.py	137
kinematics.py	138
logger.py	139
pid.py	140
plotter.py	140
serial_sender.py	141
wheel_control.py	142
main.py	144

ภาคผนวก B : รูปภาพ	147
รูป รายการของที่สั่งและรอเบิกเงิน	147
รูป ESP32 ที่ได้รับ	147
รูป ขั้นตอนการ Train และผลลัพธ์จากการ Train ครั้งแรก	148
รูป ขั้นตอนการ Train และผลลัพธ์จากการ Train โมเดล Small V2 และ Medium	163
รูป ขั้นตอนการ Train และผลลัพธ์จากการ Train โมเดล Large และ Extra	164
รูป เปรียบเทียบชิ้นส่วน 3D Print เก่าที่มีปัญหาและใหม่	166
รูป ตัวหุ่นระหว่างทำการประกอบ	171
รูป ตัวหุ่นเมื่อประกอบเสร็จ	173
รูป กราฟ log ค่าระหว่างทดสอบการ Tune PID	175

บทที่ 1 บทนำ

1.1 ที่มา ความสำคัญ

ในปัจจุบันเกมแนว First-Person Shooter (FPS) เป็นประเภทเกมที่ได้รับความนิยมสูง ผู้เล่นส่วนมากให้ความสำคัญกับการเล็งเป้าที่รวดเร็วและแม่นยำ ซึ่งนำไปสู่การพัฒนาเครื่องมือหรือโปรแกรมต่าง ๆ ในการฝึกฝนทักษะการเล็ง เช่น Aim Training Software เป็นต้น อย่างไรก็ตาม กระบวนการฝึกเล็งดังกล่าวยังต้องอาศัยการขยับเมาส์ของผู้เล่นเองเป็นหลัก ซึ่งอาจเกิดข้อจำกัดด้านความแม่นยำและความสม่ำเสมอในการเคลื่อนที่ของมือมนุษย์

โครงการนี้จึงมีแนวคิดในการพัฒนาระบบ "Physical Aimbot" โดยนำหลักการทางด้านวิศวกรรมหุ่นยนต์ (Robotics) และการประมวลผลภาพ (Computer Vision) มาผสมผสานกัน เพื่อสร้างหุ่นยนต์ที่สามารถควบคุมเมาส์จริงบนพื้นผิวจริงได้อย่างแม่นยำและอัตโนมัติ ผ่านการตรวจจับตำแหน่งเป้าหมายบนหน้าจอจากภาพหรือวิดีโอที่ประมวลผลด้วยโมเดล Machine Learning ประเภท Object Detection เช่น YOLO (You Only Look Once) โดยมีวัตถุประสงค์หลักคือ เพื่อศึกษาความเป็นไปได้ในการผสานเทคโนโลยีหุ่นยนต์และ Computer Vision รวมทั้งสร้างความรู้พื้นฐานและประสบการณ์ในการออกแบบและพัฒนาระบบที่มีทั้งองค์ประกอบทางด้าน Software และ Hardware มากกว่าจะมุ่งเน้นการนำไปใช้เพื่อประโยชน์เชิงแข่งขันหรือเชิงโกงภายในเกม

1.2 ประโยคปัญหาทางานวิจัย (Problem Statement)

ในการเล็งเป้าหมายในเกม FPS หรือโปรแกรมฝึกเล็งที่ต้องการความแม่นยำสูง ผู้เล่นมักประสบปัญหาในการควบคุมการเคลื่อนที่ของเมาส์ให้แม่นยำและรวดเร็ว เนื่องจากการปรับมุมและตำแหน่งของเมาส์ซ้ำ ๆ ส่งผลให้เสียเวลาและลดประสิทธิภาพในการเล่น โครงการนี้จึงมุ่งสำรวจและพัฒนาระบบการควบคุมการเคลื่อนที่ของเมาส์ด้วยกลไกหุ่นยนต์และการประมวลผลภาพ (Computer Vision) แบบเรียลไทม์ เพื่อสร้างระบบช่วยเล็งที่สามารถตรวจจับเป้าหมายบนหน้าจอและส่งให้หุ่นยนต์เคลื่อนที่เมาส์ให้ตรงตามเป้าหมายได้อย่างแม่นยำและอัตโนมัติ ทั้งนี้เพื่อเพิ่มความต่อเนื่องในการเล่นและเป็นการศึกษาการผสานเทคโนโลยี Robotics กับ Computer Vision ให้เกิดเป็นรูปธรรม

1.3 ผลผลิตและผลลัพธ์ (Outputs and Outcomes)

ผลผลิต

1. ต้นแบบหุ่นยนต์ (Robot Prototype) ที่สามารถควบคุมและเคลื่อนที่ของเมาส์จริงบนพื้นผิว เพื่อเล็งไปยังตำแหน่งเป้าหมายที่ระบุจากระบบตรวจจับภาพ
2. โมเดล Machine Learning (YOLO-based Object Detection Model) ที่สามารถตรวจจับและระบุตำแหน่งเป้าหมายจากภาพหรือวิดีโอแบบเรียลไทม์ ในโปรแกรมฝึกเล็ง (Aim Training Software) หรือเกม FPS

ผลลัพธ์

1. ความรู้และความเข้าใจเชิงลึก เกี่ยวกับการผสมผสานระหว่างเทคโนโลยี Robotics และ Computer Vision ในการประยุกต์ใช้สำหรับการควบคุมอุปกรณ์เชิงกายภาพ
2. แนวทางหรือกรอบการทำงาน (Framework) ในการพัฒนาระบบช่วยเล็งแบบกายภาพ (Physical Aiming Assistant) ที่สามารถนำไปต่อยอดหรือปรับปรุงในการศึกษาหรือวิจัยอื่น ๆ ในอนาคตได้

1.4 ความต้องการของระบบ (Requirements)

1. ระบบต้องสามารถจับภาพหน้าจอหรือรับข้อมูลจากโปรแกรมฝึกเล็งใน Kovaak โดยใช้ Python library สำหรับการจับภาพ เพื่อระบุพิกัดเป้าหมาย
2. ระบบหุ่นยนต์ต้องสามารถเคลื่อนที่ได้อย่างมีประสิทธิภาพในการเล็งและปรับตำแหน่งเมาส์ตามข้อมูลที่ได้รับ โดยใช้มอเตอร์แบบ Yellow TT motor ที่ติดกับล้อ Mecanum
3. ในอนาคตจะพัฒนาโมดูลสำหรับการส่งสัญญาณคลิก โดยใช้การสื่อสารผ่าน ESP32 Bluetooth ที่เชื่อมต่อกับ PC เพื่อส่งข้อมูลการคลิก
4. ใช้ซอฟต์แวร์และฮาร์ดแวร์ที่หาได้ทั่วไป เช่น Logitech G Pro X Superlight, ESP32 สำหรับการสื่อสาร Bluetooth, และ Python library สำหรับการจับภาพหน้าจอ (แทนการใช้ Capture Card)

1.5 ขอบเขตของงานวิจัย (Scopes)

1. ใช้โปรแกรมฝึกเล็ง (Kovaak) เป็นหลักในการเก็บข้อมูลภาพหรือวิดีโอ เพื่อนำมาประมวลผลด้วยโมเดล Machine Learning ในการระบุตำแหน่งเป้าหมาย

2. มุ่งเน้นการทดสอบประสิทธิภาพของระบบ โดยวัดผลจากคะแนน (Score) ที่ได้จากแต่ละ Scenario ในโปรแกรม Kovaak เป็นหลัก เช่น คะแนนรวม (Total Score), ความแม่นยำ (Accuracy), เปอร์เซนต์การยิงโดน (Percent Hit), จำนวนเป้าหมายที่ยิงโดน (Targets Hit), จำนวนเป้าหมายที่พลาด (Targets Missed), และข้อมูลเชิงสถิติอื่นๆ ที่โปรแกรมมีให้ เพื่อศึกษาความสามารถในการเล็งเป้าของระบบหุ่นยนต์อย่างชัดเจนและวัดผลได้

3. ในขั้นตอนนี้ยังไม่เน้นเก็บสถิติเรื่องความเร็วในการตอบสนอง (Latency) หรือรายละเอียดด้านเวลาเป็นหลัก

4. โครงการนี้มีจุดประสงค์เพื่อศึกษาและพัฒนาองค์ความรู้ในการประยุกต์ใช้ Robotics และ Computer Vision เท่านั้น ไม่มีเจตนาที่จะนำไปใช้งานเพื่อประโยชน์เชิงแข่งขันจริงหรือเพื่อการโกงเกม

1.6 ข้อกำหนดของงานวิจัย (Assumptions)

1. สภาพแวดล้อม (Environment) ที่ใช้ในการทดสอบ เช่น ความละเอียดหน้าจอ (Screen Resolution) และการตั้งค่ากราฟิก (Graphic Settings) ในโปรแกรม Kovaak จะถูกกำหนดให้มีความคงที่และไม่เปลี่ยนแปลงตลอดช่วงการทดลอง เพื่อความสม่ำเสมอในการเก็บข้อมูล
2. ผู้วิจัยสามารถปรับค่าความไวของเมาส์ (Mouse Sensitivity) และการตั้งค่าอื่น ๆ ที่เกี่ยวข้องได้ตามความเหมาะสม เพื่อให้ตรงกับเงื่อนไขและความต้องการในการทดลองร่วมกับโปรแกรม Kovaak

1.7 ขั้นตอนการดำเนินงาน

1. เก็บข้อมูลและสร้าง Dataset
 - 1.1. บันทึกการเล่นโปรแกรม Kovaak (Scenarios ต่างๆ เช่น Tracking และ Dynamic Clicking)
 - 1.2. ใช้ OpenCV ดึงภาพจากวิดีโอที่บันทึกไว้ นำไปทำ Label และ Augmentation
2. พัฒนาและฝึกโมเดล Machine Learning
 - 2.1. บันทึกการเล่นโปรแกรม Kovaak (Scenarios ต่างๆ เช่น Tracking และ Dynamic Clicking)
 - 2.2. ใช้ OpenCV ดึงภาพจากวิดีโอที่บันทึกไว้ นำไปทำ Label และ Augmentation
3. ศึกษาและคัดเลือกฮาร์ดแวร์
 - 3.1. สำรวจตัวเลือกและตัดสินใจเลือกใช้ล้อ Mecanum และมอเตอร์ประเภท Yellow TT motor เนื่องจากข้อจำกัดด้านงบประมาณ
 - 3.2. ออกแบบระบบจ่ายไฟโดยใช้แบตเตอรี่ 18650 สองก้อน พร้อมวงจรควบคุมแรงดัน (XL4016E1)
4. ออกแบบและเขียนแบบ CAD

4.1. ออกแบบโครงสร้างหุ่นยนต์และส่วนยึดอุปกรณ์ทั้งหมด โดยใช้โปรแกรม CAD เพื่อจัดวางตำแหน่งของมอเตอร์, ล้อ, ESP32 และส่วนประกอบอื่นๆ

5. สร้างต้นแบบหุ่นยนต์ (Prototype)

5.1. ประกอบวงจรและอุปกรณ์ต่างๆ ตามแบบ CAD

5.2. ทดลองระบบเบื้องต้นเพื่อทดสอบการเคลื่อนที่ของหุ่นยนต์ (ยังไม่รวมการเชื่อมต่อกับระบบ Software)

6. ผสานระบบ (Integration)

6.1. เขียนโปรแกรม Python เพื่อจับภาพหน้าจอจากโปรแกรม Kovaak แล้วส่งไปยังโมเดล YOLOv5 ที่ฝึกไว้

6.2. ส่งค่าตำแหน่งเป้าหมายที่ได้จากการตรวจจับไปยัง ESP32 ผ่าน PySerial เพื่อควบคุมการเคลื่อนที่ของหุ่นยนต์

6.3. ใช้ ESP32 Bluetooth ส่งข้อมูลคลิกกลับไปยัง PC (พัฒนาเพิ่มเติมในอนาคต)

7. ทดสอบและปรับแต่งระบบ

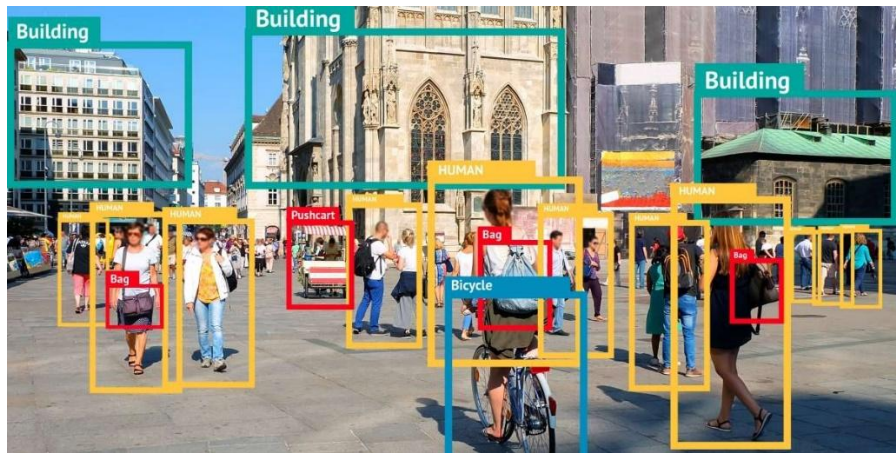
7.1. ทดสอบการทำงานจริงโดยวัดผลจากคะแนน (Scores) และความแม่นยำ (Accuracy) จาก Kovaak

7.2. วิเคราะห์ผลเบื้องต้น และวางแผนเพิ่มประสิทธิภาพ เช่น การปรับปรุง Dataset, การ Train โมเดลเพิ่มเติม, หรือปรับแต่ง Hardware

บทที่ 2 ทฤษฎี/งานวิจัย/การศึกษาที่เกี่ยวข้อง

2.1 Computer Vision สำหรับ Object Detection

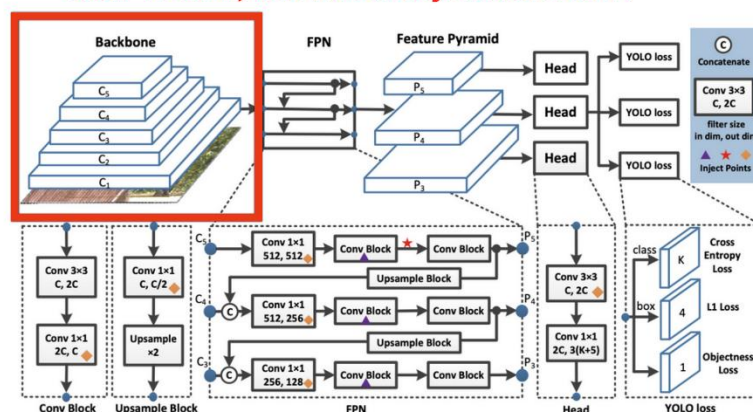
การตรวจจับวัตถุ (Object Detection) เป็นเทคนิคในวิทยาการคอมพิวเตอร์ที่ใช้ในการระบุและกำหนดตำแหน่งของวัตถุภายในภาพหรือวิดีโอ โดยทั่วไปจะใช้กรอบสี่เหลี่ยมล้อมรอบวัตถุที่ตรวจพบ



2.1.1 หลักการทำงานพื้นฐานของ YOLO (You Only Look Once)

YOLO ทำงานโดยแบ่งภาพอินพุตออกเป็นกริดขนาด $S \times S$ และสำหรับแต่ละเซลล์ในกริดนั้น โมเดลจะทำนาย B กล่องขอบเขต (bounding boxes) พร้อมกับคะแนนความเชื่อมั่น (confidence scores) ที่บ่งบอกถึงความน่าจะเป็นที่กล่องนั้นจะมีวัตถุอยู่ และความแม่นยำของกล่องที่ทำนาย นอกจากนี้ แต่ละเซลล์ยังทำนายความน่าจะเป็นแบบมีเงื่อนไขสำหรับแต่ละคลาสของวัตถุ โดยอิงตามความน่าจะเป็นที่เซลล์นั้นมีวัตถุอยู่ การทำนายทั้งหมดนี้เกิดขึ้นพร้อมกันในการประมวลผลเพียงครั้งเดียว ทำให้ YOLO มีความเร็วสูงในการตรวจจับวัตถุ

Until YOLOs, this has always been a CNN



- การแบ่งภาพเป็นกริด: ภาพอินพุตจะถูกแบ่งออกเป็นกริดขนาด $S \times S$ โดยแต่ละเซลล์ในกริดจะรับผิดชอบในการทำนายวัตถุที่มีศูนย์กลางอยู่ในเซลล์นั้น
- การทำนายกล่องขอบเขตและคะแนนความเชื่อมั่น: สำหรับแต่ละเซลล์ โมเดลจะทำนาย กล่องขอบเขต พร้อมกับคะแนนความเชื่อมั่นที่บ่งบอกถึงความน่าจะเป็นที่กล่องนั้นจะมีวัตถุอยู่ และความแม่นยำของกล่องที่ทำนาย
- การทำนายความน่าจะเป็นของคลาสวัตถุ: โมเดลจะทำนายความน่าจะเป็นแบบมีเงื่อนไขสำหรับแต่ละคลาสของวัตถุ โดยอิงตามความน่าจะเป็นที่เซลล์นั้นมีวัตถุอยู่
- การคูณคะแนนความเชื่อมั่นกับความน่าจะเป็นของคลาส: คะแนนความเชื่อมั่นจะถูกคูณกับความน่าจะเป็นของคลาสเพื่อให้ได้คะแนนสุดท้ายสำหรับแต่ละกล่องขอบเขต
- การกรองกล่องขอบเขต: ใช้เทคนิค Non-Maximum Suppression (NMS) เพื่อลบกล่องขอบเขตที่ซ้ำซ้อนและเก็บเฉพาะกล่องที่มีคะแนนสูงสุดสำหรับแต่ละวัตถุ

2.1.2 การเปรียบเทียบรุ่น YOLO (YOLO Version Comparison)

ในงานนี้ เราเน้นการใช้โมเดล YOLOv5 ซึ่งมีหลายเวอร์ชันที่แตกต่างกันในเรื่องของขนาดและประสิทธิภาพ เราสามารถแบ่งเวอร์ชันหลัก ๆ ได้ดังนี้:

1. YOLOv5s (Small)

- ข้อดี:
 - มีขนาดเล็ก ใช้ทรัพยากรน้อย
 - ประมวลผลได้รวดเร็ว เหมาะกับงานที่ต้องการการตอบสนองแบบเรียลไทม์ เช่น การตรวจจับเป้าหมายในโปรแกรม Aim Training (Kovaak)
- ข้อเสีย:
 - ความแม่นยำอาจต่ำกว่าเมื่อเทียบกับเวอร์ชันที่มีขนาดใหญ่กว่า

2. YOLOv5m (Medium)

- ข้อดี:
 - ให้ความแม่นยำสูงขึ้นจาก YOLOv5s ด้วยจำนวนพารามิเตอร์ที่เพิ่มขึ้น
 - ยังสามารถประมวลผลแบบเรียลไทม์ได้ดีในระบบที่มีฮาร์ดแวร์ประสิทธิภาพสูง
- ข้อเสีย:
 - ใช้ทรัพยากรมากขึ้น และอาจมีความช้ากว่า YOLOv5s เล็กน้อย

3. YOLOv5l (Large) และ YOLOv5x (Extra Large)

- ข้อดี:
 - ให้ความแม่นยำสูงสุด เนื่องจากมีจำนวนพารามิเตอร์มากที่สุด
- ข้อเสีย:
 - ประมวลผลช้ากว่าและต้องการทรัพยากรระบบที่สูงขึ้น ซึ่งอาจไม่เหมาะกับงานที่ต้องการความเร็วแบบเรียลไทม์ในสภาพแวดล้อมที่มีข้อจำกัดด้านทรัพยากร

สำหรับโครงการนี้ เราเริ่มต้นด้วย YOLOv5s (Small) เนื่องจากมีความเร็วสูงและเหมาะสมกับการทดลองเบื้องต้นในระบบ Aim Training (Kovaak) อย่างไรก็ตาม ด้วยประสิทธิภาพของ PC ที่มีอยู่ (สามารถรันได้ 200+ FPS) ในอนาคตจึงมีความเป็นไปได้ที่จะปรับปรุงและฝึกเพิ่มเติมโดยพิจารณาให้เหมาะสมกับข้อจำกัดของฮาร์ดแวร์ และลักษณะของงาน เช่น หากต้องการตรวจจับอย่างรวดเร็วและใช้งานร่วมกับระบบที่มีข้อจำกัดด้าน latency อาจเลือกใช้ YOLOv5s หรือ m ในขณะที่หากระบบสามารถรองรับการประมวลผลที่ช้าลงได้และต้องการความแม่นยำสูง YOLOv5l หรือ x ก็เป็นทางเลือกที่เหมาะสม

2.2 Robotics และการควบคุมการเคลื่อนที่

2.2.1 Omni Wheels และ Mecanum Wheels

ในการออกแบบระบบเคลื่อนที่สำหรับหุ่นยนต์ที่ควบคุมเมาส์ในงาน Physical Aimbot มีล้อสองแบบที่มักถูกนำมาใช้ได้แก่ Omni Wheels และ Mecanum Wheels:

- Omni Wheels:

- หลักการ: ล้อ Omni มีแผ่นสัมผัสที่สามารถหมุนได้ในแนวตั้งและมีล้อเล็กติดอยู่รอบๆ ซึ่งช่วยให้เคลื่อนที่ได้ในทิศทางหลายทิศทาง
- ข้อดี: ให้ความสามารถในการเคลื่อนที่แบบ holonomic (เคลื่อนที่ในทุกทิศทาง)
- ข้อเสีย: โดยทั่วไปราคาแพงกว่าแบบอื่นถึง 4-5 เท่า ทำให้ไม่เหมาะสำหรับงานที่มีข้อจำกัดด้านงบประมาณ

- Mecanum Wheels:

- หลักการ: ล้อ Mecanum มีลูกกลิ้งติดอยู่ที่ขอบล้อในมุมเอียง ช่วยให้สามารถเคลื่อนที่แบบ omnidirectional ได้ด้วยการควบคุมความเร็วของแต่ละล้อแยกกัน
- ข้อดี: ราคาถูกกว่า Omni Wheels อย่างมาก, ให้ความสามารถในการเคลื่อนที่แบบหลายทิศทางได้ดีเพียงพอสำหรับงานควบคุมเมาส์ในระบบ Physical Aimbot
- ข้อเสีย: การควบคุมอาจซับซ้อนกว่าเล็กน้อยเนื่องจากต้องคำนวณการเคลื่อนที่ที่สัมพันธ์กันของล้อทั้งหมด
-

สาเหตุที่เลือก Mecanum Wheels:

เนื่องจากข้อจำกัดด้านงบประมาณ โดย Omni Wheels มีราคาสูงถึง 4-5 เท่าของ Mecanum Wheels ทำให้การเลือกใช้ Mecanum Wheels เป็นทางเลือกที่ประหยัดและยังคงให้ประสิทธิภาพในการเคลื่อนที่ที่เพียงพอสำหรับการควบคุมเมาส์ในโครงการนี้

2.2.2 ทฤษฎี Inverse Kinematics สำหรับล้อ Mecanum

การควบคุมการเคลื่อนที่ในหุ่นยนต์ที่ใช้ล้อ Mecanum ต้องอาศัยหลักการของ Inverse Kinematics ซึ่งเป็นการคำนวณหาความเร็วและทิศทางของแต่ละล้อจากคำสั่งการเคลื่อนที่ที่ต้องการ (เช่น การเล็งเป้าหมาย)

- **หลักการคำนวณ:**

สำหรับหุ่นยนต์ที่ใช้ล้อ Mecanum จำนวน 4 ตัว เราสามารถคำนวณความเร็วของแต่ละล้อจากความต้องการเคลื่อนที่ในแกน X (V_x), แกน Y (V_y) และการหมุน (angular velocity, ω) ได้ดังนี้:

- ความเร็วล้อหน้าซ้าย (V_{FL}) = $V_x - V_y - (L + W) * \omega$
- ความเร็วล้อหน้าขวา (V_{FR}) = $V_x + V_y + (L + W) * \omega$
- ความเร็วล้อหลังซ้าย (V_{RL}) = $V_x + V_y - (L + W) * \omega$
- ความเร็วล้อหลังขวา (V_{RR}) = $V_x - V_y + (L + W) * \omega$

โดยที่:

- V_x : ความเร็วในแนวแกน X
- V_y : ความเร็วในแนวแกน Y
- ω : ความเร็วเชิงมุม (การหมุน)
- L: ระยะจากจุดศูนย์กลางไปยังล้อในแนวตั้ง
- W: ระยะจากจุดศูนย์กลางไปยังล้อในแนวนอน

• แนวคิดเบื้องต้นในการควบคุม:

เมื่อระบบ Computer Vision ตรวจจับเป้าหมายแล้ว ระบบจะคำนวณตำแหน่งเป้าหมายที่ต้องเล็ง จากนั้นใช้ Inverse Kinematics ในการแปลงคำสั่งเคลื่อนที่ (เช่น ค่า V_x , V_y , ω) ไปเป็นความเร็วที่แต่ละล้อควรหมุนเพื่อให้หุ่นยนต์เคลื่อนที่ไปในทิศทางที่ต้องการได้อย่างแม่นยำ การคำนวณเหล่านี้ช่วยให้ระบบสามารถปรับตำแหน่งเมาส์ให้ตรงกับเป้าหมายได้โดยอัตโนมัติ

2.2.3 PID Controller สำหรับการควบคุมความเร็วของมอเตอร์

ในการควบคุมความเร็วหรือทิศทางของมอเตอร์ให้แม่นยำในระบบหุ่นยนต์ มักนิยมใช้ **PID Controller** (Proportional-Integral-Derivative) ซึ่งเป็นการควบคุมแบบ Feedback ที่สามารถลดความคลาดเคลื่อน (error) ระหว่าง "ค่าที่ต้องการ" (Setpoint) และ "ค่าที่เกิดขึ้นจริง" (Measured Value) ได้อย่างมีประสิทธิภาพ

แนวคิดพื้นฐานของ PID

PID ประกอบด้วยสามส่วนหลัก ดังนี้:

- **P (Proportional):** เป็นส่วนที่ตอบสนองต่อค่า error ปัจจุบัน โดยมีค่าสัดส่วนเป็นตัวคูณ

$$P_{out} = K_p * e(t)$$
- **I (Integral):** เป็นส่วนที่รวมสะสมค่า error ย้อนหลัง เพื่อกำจัดค่า steady-state error

$$I_{out} = K_i * \int e(t) dt$$
- **D (Derivative):** เป็นส่วนที่ตอบสนองต่อการเปลี่ยนแปลงของ error เพื่อช่วยลดการ overshoot และทำให้ระบบมีความเสถียรมากขึ้น

$$D_{out} = K_d * de(t)/dt$$

ผลรวมของทั้งสามส่วนคือสัญญาณควบคุมที่ส่งไปยังมอเตอร์:

$$u(t) = P_out + I_out + D_out$$

หรือรวมในสมการเดียว:

$$u(t) = K_p * e(t) + K_i * \int e(t) dt + K_d * de(t)/dt$$

โดยที่:

- $u(t)$ = ค่าคำสั่งควบคุมที่ส่งออก
- $e(t)$ = ค่า error = ค่าที่ต้องการ - ค่าที่วัดได้
- K_p, K_i, K_d = ค่าพารามิเตอร์ของ PID ซึ่งต้องทำการจูนให้เหมาะสมกับระบบ

การประยุกต์ใช้กับโครงงาน Physical Aimbot

ในโครงงานนี้ PID ถูกออกแบบมาเพื่อใช้ควบคุม **ความเร็วของมอเตอร์แต่ละตัว** เพื่อเคลื่อนที่หุ่นยนต์ให้ไปยังตำแหน่งที่ตรวจจับได้จากระบบ Vision โดยมีขั้นตอนดังนี้:

1. ระบบตรวจจับภาพจะระบุ **ตำแหน่งเป้าหมาย (Target)** บนหน้าจอ
2. คำนวณตำแหน่งเป้าหมายเทียบกับจุดศูนย์กลางของหน้าจอ เพื่อหา **ค่าความคลาดเคลื่อน (error)** ทั้งในแนวระยะ (Distance) และมุม (Angle)
3. นำค่า error ที่ได้เข้าสู่ฟังก์ชัน PID เพื่อคำนวณ "ความเร็วของแต่ละล้อ" ที่จะทำให้หุ่นยนต์เคลื่อนที่เข้าหาเป้าหมายอย่างแม่นยำ

ระบบนี้สามารถใช้ distance เป็นค่าความห่างจากเป้าหมาย และ angle เป็นทิศทางเปรียบเทียบกับแนวตั้งฉากของหุ่นยนต์ ทำให้สามารถนำทั้งสองค่ามาควบคุม PID แบบแยกแกนได้ หรือแบบรวมก็ได้ ขึ้นอยู่กับกลยุทธ์การควบคุม

การตั้งค่าพารามิเตอร์ PID (PID Tuning)

การตั้งค่าค่า K_p, K_i, K_d ที่เหมาะสมเป็นขั้นตอนที่สำคัญมาก โดยวิธีที่นิยมคือ **Manual Tuning** หรือ "trial-and-error" โดย:

1. เริ่มจาก K_i และ $K_d = 0$
2. เพิ่ม K_p ทีละน้อยจนระบบเริ่มตอบสนอง
3. เพิ่ม K_i เพื่อแก้ปัญหา steady-state error

4. เพิ่ม Kd เพื่อช่วยลด overshoot และเพิ่มความเสถียร

ในกรณีของโครงงานนี้ เนื่องจากไม่มี encoder feedback เพื่อวัดความเร็วของล้อจริง จึงมีการใช้ “distance to target” หรือ “angle to target” จาก Vision เป็น input แทน โดยจะต้องสังเกตพฤติกรรมของหุ่นยนต์เพื่อทำการจูนค่า PID ให้เหมาะสม

2.3 Hardware และการสื่อสารข้อมูล

2.3.1 ESP32 Microcontroller

เราเลือกใช้ ESP32 เนื่องจากมีคุณสมบัติที่โดดเด่นหลายประการที่สอดคล้องกับความต้องการของโครงงานนี้

- **ความเร็วและประสิทธิภาพสูง:** ESP32 สามารถประมวลผลคำสั่งและสื่อสารได้อย่างรวดเร็ว ทำให้รองรับงานที่ต้องการการตอบสนองแบบเรียลไทม์
- **การเชื่อมต่อแบบ WiFi และ Bluetooth:** การมีทั้ง WiFi และ Bluetooth ในตัวช่วยให้เราสามารถเชื่อมต่อกับ PC ผ่าน WiFi หรือใช้ Bluetooth สำหรับส่งคำสั่งการคลิกโดยตรง ซึ่งจะช่วยลดความซับซ้อนของโมดูลคลิกแยกต่างหาก
- **ความยืดหยุ่นในการประมวลผล Inverse Kinematics:** ระบบสามารถเลือกที่จะคำนวณ Inverse Kinematics บนตัว ESP32 เอง หรือรับค่าที่คำนวณจาก PC (เช่น ค่า Distance, Degree และ Shooting command) ขึ้นอยู่กับประสิทธิภาพของแต่ละหน่วย เราสามารถปรับเปลี่ยนวิธีการส่งข้อมูลได้โดยพิจารณาจากความเร็วในการคำนวณของ PC เทียบกับ ESP32

2.3.2 การสื่อสารข้อมูลผ่าน PySerial

สำหรับการส่งข้อมูลระหว่างคอมพิวเตอร์และ ESP32 เราใช้การสื่อสารผ่าน PySerial ด้วยข้อดีดังนี้:

- **การเชื่อมต่อที่ง่าย:** PySerial ช่วยให้การส่งข้อมูลระหว่าง PC กับ ESP32 เป็นไปอย่างราบรื่น โดยไม่ต้องใช้ฮาร์ดแวร์เพิ่มเติม
- **ประเภทของข้อมูลที่ส่ง:**
 - **Distance:** ระยะห่างระหว่างตำแหน่งปัจจุบันกับเป้าหมายที่คำนวณได้
 - **Degree:** มุมที่เป้าหมายอยู่ในระบบพิกัด

- Shooting Command: คำสั่งยิงที่จะส่งไปยัง ESP32 เมื่อเป้าหมายอยู่ในขอบเขตที่กำหนด
- กระบวนการส่งข้อมูล:
 - หากการคำนวณ Inverse Kinematics ถูกดำเนินการบน PC เราจะส่งข้อมูลที่คำนวณแล้ว (Distance, Degree) พร้อมกับคำสั่งยิง (Shooting command) ไปยัง ESP32
 - ในทางกลับกัน หาก ESP32 มีความสามารถในการคำนวณ Inverse Kinematics ได้เร็วเพียงพอ ระบบจะรับข้อมูลพื้นฐานจาก PC (เช่น ตำแหน่งเป้าหมาย) แล้วคำนวณค่า Distance, Degree และตัดสินใจส่งคำสั่งยิงภายในตัวเอง

2.3.3 Motor และ Driver

ในการขับเคลื่อนระบบหุ่นยนต์ เราเลือกใช้ Yellow TT Motor ร่วมกับ DRV8833 Motor Driver ด้วยเหตุผลดังนี้:

- Yellow TT Motor:
 - ร้านค้าที่มีจำหน่ายมีให้เลือกเพียง 2 แบบคือ 1:150 และ 1:48 สำหรับเกียร์
 - เนื่องจากหุ่นยนต์ของเราเบา (ใช้สำหรับเคลื่อนที่เมสท์ที่มีน้ำหนักต่ำ) เราจึงเลือกใช้เกียร์ 1:48 เพื่อให้ได้ความเร็วที่สูงขึ้น แม้ว่าแรงบิด (torque) จะน้อยลง แต่สำหรับงานนี้ ความเร็วในการตอบสนองและการเคลื่อนที่เป็นสิ่งที่สำคัญกว่า
 - โดยจาก Datasheet ของร้านค้า Yellow TT Motor 1:48 มีค่าแรงบิดประมาณ $1.5 \text{ kg}\cdot\text{cm}$ ซึ่งเมื่อนำมาคำนวณเป็นแรงขับเคลื่อนบนพื้นราบ สามารถวิเคราะห์ได้ดังนี้:

การประมาณแรงขับเคลื่อน (Force Calculation):

$$\text{Torque (}\tau\text{)} = 1.5 \text{ kg}\cdot\text{cm} = 0.147 \text{ N}\cdot\text{m}$$

$$\text{รัศมีล้อ (r)} = 48 \text{ mm} / 2 = 0.024 \text{ m}$$

$$\text{แรง (F) ที่ปลายล้อ} = \tau / r$$

$$= 0.147 / 0.024 \approx 6.125 \text{ N ต่อล้อ}$$

หากใช้ล้อทั้ง 4 ล้อขับเคลื่อนพร้อมกัน:

$$\text{แรงรวม} \approx 4 \times 6.125 = 24.5 \text{ N}$$

$$\text{และจากสมการ } F = ma \Rightarrow a = F/m$$

การประมาณน้ำหนักหุ่นยนต์:

องค์ประกอบ	ประมาณน้ำหนัก
เมาส์ Logitech G Pro X	~70 g
แบตเตอรี่ 18650 × 2	~90 g
ESP32 + XL4016	~40 g
โครง 3D print + มอเตอร์	~300 g
รวม	~500 g (0.5 kg)

น้ำหนักหุ่นยนต์ประมาณ 0.5 kg $\Rightarrow$ ต้องการแรงเพื่อเริ่มเคลื่อนที่ $\approx \mu mg$
โดยที่:

μ = ค่าสัมประสิทธิ์แรงเสียดทานระหว่าง mouse feet กับพื้น $\approx 0.3-0.5$ (Corepad Air)
 $g = 9.81 \text{ m/s}^2$

$\Rightarrow F_{\text{friction}} \approx 0.4 \times 0.5 \times 9.81 \approx 1.96 \text{ N}$

เมื่อเทียบกับแรงขับเคลื่อน $\approx 24.5 \text{ N}$ (รวมทุกล้อ)

$\rightarrow$ มีแรงมากเพียงพอในทางทฤษฎี สำหรับการเคลื่อนที่ แม้จะเผื่อการสูญเสียในระบบ

- DRV8833 Motor Driver:

- เป็นตัวควบคุมมอเตอร์ที่มีขนาดกะทัดรัด น้ำหนักเบา และรองรับการทำงานของมอเตอร์ในระดับแรงดันที่เหมาะสมสำหรับระบบของเรา

2.3.3.1 ข้อสังเกตเพิ่มเติมจากการใช้งานจริงของมอเตอร์ Yellow TT (1:48)

ผู้วิจัยจึงตัดสินใจเลือกใช้ **มอเตอร์เกียร์ 1:48** เนื่องจากตัวหุ่นยนต์มีน้ำหนักรวมไม่มาก (ใช้เพียงเมาส์, ESP32, แบตเตอรี่ และโครงพลาสติก) โดยมีเป้าหมายให้หุ่นยนต์สามารถตอบสนองต่อคำสั่งเล็งเป้าได้เร็วและลื่นไหล **อย่างไรก็ตาม** จากการทดสอบจริงในระยะหลัง พบว่าอัตราทด 1:48 อาจ **ไม่เหมาะสมในด้านความแม่นยำในความเร็วต่ำ** เพราะล้อจะเริ่มหมุนเมื่อได้รับแรงมากพอเท่านั้น เนื่องจากแรงบิดเริ่มต้นต่ำ ทำให้ไม่สามารถควบคุมการเคลื่อนไหวแบบละเอียด (เช่น เล็งเป้าแบบละเอียดยิบหรือแก้ไขเป้าเล็ก ๆ) ได้ดีนัก

ปัญหาที่พบ:

- ล้อมี **Deadzone ขนาดใหญ่**: PWM ต่ำกว่า ~ 145 ไม่สามารถทำให้ล้อเริ่มหมุนได้ ต้องใช้แรงกระตุ้นสูงมากขึ้น
- เกิดการ “กระชาก” เมื่อล้อเริ่มหมุน: เนื่องจากแรงบิดต่ำ พอเพิ่ม PWM จนพ้น Deadzone ล้อจะหมุนเร็วเกิน ทำให้เคลื่อนไหวนั่นไม่ละเอียดพอ
- มีผลโดยตรงต่อการจูน PID: ค่าความเร็วเริ่มต้นที่ไม่ต่อเนื่องทำให้หาค่า K_p , K_d ที่สมดุลได้ยากในความเร็วต่ำ

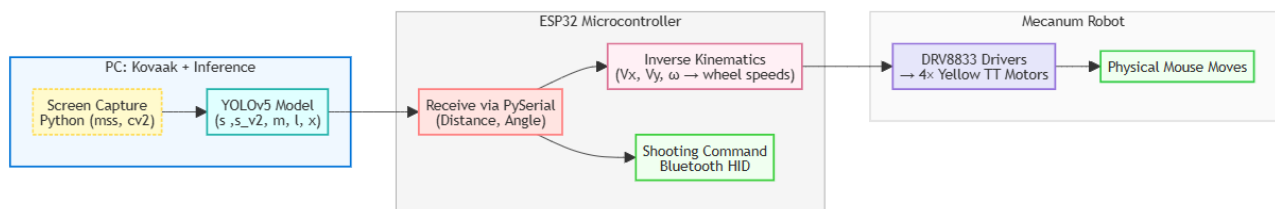
ข้อเสนอแนะสำหรับรุ่นถัดไป:

- อาจพิจารณาเปลี่ยนเป็นมอเตอร์ที่มี **อัตราทดสูงขึ้น** (เช่น 1:100) หรือ **มอเตอร์แบบมี Encoder** เพื่อให้สามารถควบคุมความเร็วและตำแหน่งได้แม่นยำขึ้น
- อีกทางเลือกคือการใช้ **มอเตอร์แบบ Stepper หรือ Servo** ซึ่งให้ความละเอียดและ Feedback ที่เหมาะสมกับงานประเภท Precision Control

บทที่ 3 ระเบียบวิธีวิจัย

3.1 ภาพรวมของระบบ (System Overview)

โครงสร้างภาพรวมของระบบนี้แบ่งออกเป็นหลายโมดูลที่ทำงานร่วมกันเพื่อให้ได้ระบบ Physical Aimbot ที่สามารถจับภาพจาก Kovaak, ประมวลผลข้อมูลด้วยโมเดล YOLO, คำนวณ Inverse Kinematics และควบคุมการเคลื่อนที่ของหุ่นยนต์ พร้อมทั้งส่งคำสั่งคลิกกลับไปยัง PC ผ่าน ESP32



ส่วนประกอบหลักของระบบมีดังนี้:

1. Screen Capture Module:

- โปรแกรม Python ใช้จับภาพหน้าจอจากโปรแกรม Kovaak
- ภาพที่จับได้ถูกส่งต่อไปยังโมเดล YOLO เพื่อทำการตรวจจับวัตถุ

2. Object Detection Module (YOLO Model):

- โมเดล YOLO (เช่น YOLOv5s หรือ YOLOv5m ตามที่เลือก) ประมวลผลภาพที่ได้รับจาก Screen Capture
- ตรวจจับตำแหน่งเป้าหมาย (bounding boxes, confidence scores) และส่งข้อมูลผลลัพธ์ไปยัง ESP32

3. ESP32 Microcontroller:

- รับข้อมูลจากโมเดล YOLO ผ่านการสื่อสาร (PySerial)
- คำนวณ Inverse Kinematics (หรือรับค่าที่คำนวณจาก PC ขึ้นอยู่กับประสิทธิภาพ) เพื่อแปลงข้อมูลตำแหน่งเป้าหมายเป็นคำสั่งสำหรับควบคุมมอเตอร์ (เช่น ค่าความเร็วและมุมของแต่ละล้อ)
- สั่งการเคลื่อนที่ของหุ่นยนต์ตามข้อมูลที่คำนวณได้
- ส่งคำสั่งการคลิก (Shooting Command) ผ่าน Bluetooth กลับไปยัง PC เมื่อเป้าหมายอยู่ในขอบเขตที่กำหนด

4. Motor Control Module:

- มอเตอร์ (Yellow TT Motor) ที่ติดตั้งกับล้อ Mecanum รับคำสั่งจาก ESP32 เพื่อขับเคลื่อนหุ่นยนต์ให้เคลื่อนที่ไปตามตำแหน่งที่ต้องการ

5. Communication Flow:

- PC: จับภาพหน้าจอจาก Kovaak → ส่งข้อมูลไปยังโมเดล YOLO
- Model: ตรวจจับเป้าหมาย → ส่งผลลัพธ์ (ตำแหน่งเป้าหมาย) ไปยัง ESP32
- ESP32: คำนวณ Inverse Kinematics → ส่งมอเตอร์เคลื่อนที่ → ส่งคำสั่งคลิกกลับไปยัง PC

หมายเหตุ: โครงสร้างระบบดังแสดงนี้เป็นแนวทางเริ่มต้นที่ออกแบบไว้ อย่างไรก็ตาม จากการทดลองจริงพบข้อจำกัดในเรื่องเวลาและความยืดหยุ่นของระบบ จึงมีการปรับให้ฝั่ง PC รับหน้าที่คำนวณ Inverse Kinematics และจำลองคลิกทั้งหมดแทนการใช้ ESP32 เพื่อเพิ่มความสะดวกและลดเวลาในการพัฒนา รายละเอียดการปรับเปลี่ยนดังกล่าวจะอธิบายเพิ่มเติมในบทถัดไป

3.2 วิธีการบันทึกวิดีโอการเล่นโปรแกรม Kovaak

3.2.1 วิธีการบันทึกวิดีโอการเล่นโปรแกรม Kovaak

ในการสร้าง Dataset สำหรับตรวจจับเป้าหมาย (Targets) ในโปรแกรม Aim Training อย่าง Kovaak ผู้วิจัยได้ทำการบันทึกวิดีโอของการเล่นทั้งหมด 5 ฉาก (Scenarios) ซึ่งประกอบด้วย:

1. 3D Switching Robots
 2. Buff Robots 360
 3. Close Fast Strafes Easy Invincible Robot 25% Slower
 4. CLS Click Robots Rounded
 5. Correction Accuracy – Close I Strafe Robot
- **ระยะเวลา:** แต่ละฉากมีความยาวประมาณ 1 นาที รวมเป็นเวลาทั้งสิ้นประมาณ 5 นาที
 - **ความละเอียด (Resolution):** 1920 × 1080
 - **อัตราเฟรม (Frame Rate):** 30 FPS

โดยใช้โปรแกรมบันทึกวิดีโอหน้าจอ (Screen Recording) ในสภาพแวดล้อมการเล่นปกติของ Kovaak เพื่อให้ได้ข้อมูลการเล็งเป้าหมายที่ใกล้เคียงการใช้งานจริงที่สุด

3.2.1 การดึงภาพจากวิดีโอด้วย OpenCV และขั้นตอนการ Label ข้อมูลเป้าหมาย (Object Labeling)
หลังจากได้ไฟล์วิดีโอทั้ง 5 ฉากแล้ว ผู้วิจัยจึงนำวิดีโอเหล่านั้นมาประมวลผลด้วย Python และ OpenCV เพื่อแยกเฟรมออกมาเป็นรูปภาพ (.png) ดังตัวอย่างโค้ดด้านล่าง (extract_frame.py):

```
cap = cv2.VideoCapture(video_path)
success, frame = cap.read()
while success:
    frame_path = os.path.join(output_dir, f"{count:05}.png")
    cv2.imwrite(frame_path, frame)
    success, frame = cap.read()
```

1. กระบวนการดึงภาพ (Frame Extraction):

- โปรแกรมจะเปิดวิดีโอแต่ละไฟล์และอ่านเฟรมทีละเฟรม
- แต่ละเฟรมจะถูกบันทึกเป็นไฟล์ภาพ .png ไว้ในโฟลเดอร์แยกตามชื่อวิดีโอ
- ทำให้ได้จำนวนภาพทั้งหมดหลายพันภาพ (ขึ้นอยู่กับความยาววิดีโอและอัตราเฟรม)

2. การ Label ข้อมูล (Object Labeling):

- นำภาพที่ได้ไปทำ Label จุด (Bounding Box) ของ “Robot” ซึ่งเป็นเป้าหมายหลักในการฝึกเลี้ยง
- รูปแบบการบันทึก Label เป็น YOLO v5 PyTorch format เพื่อให้สอดคล้องกับโมเดล YOLO ที่จะใช้งานต่อไป
- ใช้เครื่องมือออนไลน์ (เช่น Roboflow หรือโปรแกรม Labeling อื่น ๆ) เพื่อช่วยในการ Annotate วัตถุ

3.2.3 การทำ Augmentation ผ่าน Roboflow

หลังจากได้ภาพที่ Label เสร็จเรียบร้อยแล้ว ผู้วิจัยได้ทำ Data Augmentation โดยใช้แพลตฟอร์ม Roboflow เพื่อเพิ่มความหลากหลายของภาพและลดปัญหา Overfitting ซึ่งรายละเอียดของการ Augment มีดังนี้:

1. จำนวน Dataset:

- Dataset มีภาพทั้งหมดประมาณ 11,342 ภาพ (รวมภาพต้นฉบับและภาพที่ผ่านการ Augmentation)

2. การปรับขนาด (Resize):

- แต่ละภาพถูกปรับขนาดเป็น 640×640 (Stretch) เพื่อให้ตรงกับ input size ของ YOLOv5

3. การ Flip และ Crop:

- **Horizontal Flip** (50% Probability) ช่วยจำลองตำแหน่งของศัตรูที่อาจโผล่มาทั้งด้านซ้ายและขวาในฉากเกม เพื่อให้โมเดลไม่ Bias ด้านใดด้านหนึ่ง
- **Vertical Flip** (50% Probability) ช่วยจำลองตำแหน่งของศัตรูที่อาจโผล่มาทั้งด้านซ้ายและขวาในฉากเกม เพื่อให้โมเดลไม่ Bias ด้านใดด้านหนึ่ง
- **Random Crop** ระหว่าง 0% ถึง 20% ของ Bounding Box ช่วยให้โมเดลเรียนรู้จากสถานการณ์ที่เป้าหมายบางส่วนอาจถูกซ่อนอยู่หลังวัตถุหรือขอบจอ เช่น เป้าหมายหลบอยู่หลังของฉากในเกม

4. การ Shear (บิดภาพ):

- มีการสุ่มค่าการ Shear ในช่วง -10° ถึง $+10^\circ$ ทั้งแนวนอนและแนวตั้ง ใช้เพื่อจำลองความเบลอหรือ distortion จากการเคลื่อนที่เร็วในเกม (เช่น ตอนเป้าหมาย strafe อย่างรวดเร็ว)

5. Auto-Orientation และ EXIF Stripping:

- มีการปรับ Orientation ของภาพอัตโนมัติเพื่อให้ทิศทางของภาพถูกต้องเสมอ
- ลบข้อมูล EXIF ที่ไม่จำเป็นออก

การ Augmentation ดังกล่าวช่วยเพิ่มความหลากหลายของภาพและสถานการณ์การเล็งเป้า ทำให้โมเดลเรียนรู้ได้ครอบคลุมมากขึ้น อย่างไรก็ตาม นี่เป็นการตั้งค่าพื้นฐานในการ Augment ครั้งแรกเท่านั้น ซึ่งวางแผนที่จะปรับปรุงหรือเปลี่ยนแปลงพารามิเตอร์ในอนาคต เมื่อมีการปรับแต่งระบบหุ่นยนต์และโมเดล YOLO ให้มีประสิทธิภาพมากขึ้น

3.2.4 การสร้าง Dataset v2 (Scenarios ขยาย + Quality Control)

เนื่องจาก Dataset ชุดแรก มีปริมาณภาพค่อนข้างน้อย และครอบคลุมเพียง 5 Scenarios ที่มีลักษณะคล้ายกัน เช่น robot เดินตรงใน map เดิม ทำให้โมเดลที่ฝึกจากข้อมูลดังกล่าวมีแนวโน้ม **overfit** และ **ขาดความสามารถในการ generalize** เมื่อนำไปใช้กับฉากใหม่ ๆ ที่มีลักษณะแตกต่างกัน เช่น robot ซ่อนกันหลายตัว / แสงต่างกัน / map คนละแบบ

นอกจากนี้ในเวอร์ชันแรกมีการ label อย่างเร่งรีบ จึงอาจเกิด **miss-label** และ **noise** ในชุดข้อมูล ได้สูง ผู้วิจัยจึงตัดสินใจสร้าง Dataset v2 โดยปรับปรุงทั้งด้าน **ปริมาณ, คุณภาพ, และ ความหลากหลายของสถานการณ์**

Scenarios ใหม่ที่เพิ่ม:

เพิ่ม 10 ฉากใหม่ที่ครอบคลุมลักษณะการเล่นที่หลากหลาย เช่น tracking, click-timing, low-ground jumps, long-range flicks เพื่อให้ Dataset มีความหลากหลายและครอบคลุมสถานการณ์จริงมากขึ้น

การดิงเฟรม:

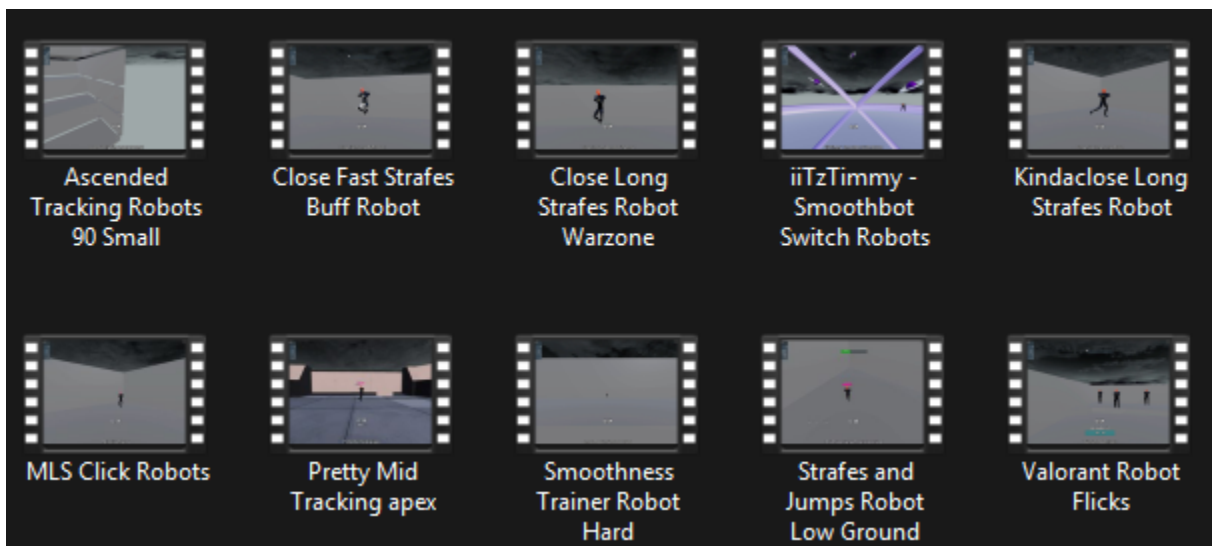
ลด sampling เป็น 5 fps (เลือกทุก 6 เฟรมจากวิดีโอ 30 fps ความยาว 1 นาที) เพื่อหลีกเลี่ยงข้อมูลซ้ำซ้อนเกินไป และยังช่วยให้สามารถเพิ่มจำนวน scenario ได้มากขึ้นภายใต้จำนวนเท่าเดิม

จำนวนภาพหลัง extraction:

ได้ภาพทั้งหมด 38,409 ภาพ ซึ่งมากกว่าชุดก่อนหน้านี้ถึง 3.4 เท่า

การตรวจสอบ label:

มีการตรวจสอบ bounding box เติมอย่างละเอียด และแก้ไข miss-label ที่พบก่อนจะทำการ export เป็น YOLO v5 (PyTorch format) เพื่อเตรียมใช้ในการฝึกโมเดลจริง



ภาพ Dataset ชุดใหม่ที่เพิ่มขึ้น

3.2.4.1 Pre-processing

- **Auto-orientation & EXIF strip** – ป้องกันปัญหาการกลับหัวเวลาภาพมาจากเครื่องต่างแพลตฟอร์ม
- **Adaptive equalization** – เพิ่ม contrast ใน map ที่มี *fog* หรือ *low-light* (เช่นฉาก Valorant Robot Flicks)

- **Resize 640 × 640 (Stretch)** – ตรงตามค่า `--img 640` ของโมเดลทุกขนาด

3.2.4.2 Augmentation (7× per source)

มีการใช้เทคนิค Augmentation หลากหลายรูปแบบเพื่อจำลองสถานการณ์ต่าง ๆ ที่เกิดขึ้นจริงในเกม โดยมีรายละเอียดดังนี้:

- **Horizontal Flip (50%)**
เพื่อจำลองการเคลื่อนไหวของเป้าหมายที่ strafe ซ้าย/ขวาสลับไปมาอย่างต่อเนื่องในเกม FPS ซึ่งเป็นหนึ่งในสถานการณ์ที่พบได้บ่อย
- **Random Rotation ($\pm 15^\circ$)**
ใช้เพื่อเลียนแบบการส่ายกล้องเร็ว หรือการเคลื่อนไหวแบบ flick ของผู้เล่นที่หมุนเล็งเป้าแบบฉับพลัน
- **Brightness Adjustment (-20% ถึง +20%) และ Exposure ($\pm 10\%$)**
จำลองการเปลี่ยนแปลงของแสงใน Arena หรือแผนที่ ซึ่งเกิดขึ้นได้เมื่อกำลังหมุนผ่านจุดที่มีความสว่างต่างกัน เช่น โซนแสงแดดและโซนเงา
- **Box Crop (0–20%)**
เพื่อจำลองสถานการณ์ที่เป้าหมายโผล่แค่ครึ่งตัว เช่น เมื่อยืนหลังกำแพง สิ่งกีดขวาง หรือโผล่จากมุมบัง
- **Shear ($\pm 10^\circ$), Gaussian Blur (0–0.1px), และ Salt-Pepper Noise (0.06%)**
ใช้จำลอง motion blur และ noise ที่เกิดขึ้นจริงเมื่อมีการเคลื่อนไหวเร็ว เช่น การวิ่งหรือเป้าหมายหมุนตัว

3.3 การฝึกและประเมินผลโมเดล YOLO (Model Training and Evaluation)

3.3.1 สภาพแวดล้อมที่ใช้ในการฝึกโมเดล (Hardware Spec, Software Environment, Libraries)

- **ฮาร์ดแวร์ (Hardware):**
 - ใช้เครื่อง ASUS TUF A15 Laptop ที่มาพร้อมกับ RTX 3060
 - CPU: Ryzen 7 6700H
 - RAM: 16GB DDR5 (ความถี่ 4800Hz)
 - SSD สำหรับความเร็วในการอ่านเขียนข้อมูล
- **สภาพแวดล้อมการฝึก (Training Environment):**
 - ใช้ Conda Environment สำหรับการติดตั้ง Python libraries ที่เกี่ยวข้องกับ YOLOv5
 - Libraries ที่สำคัญ: PyTorch, OpenCV, NumPy, และอื่น ๆ ตามที่ระบุใน requirements.txt ของ YOLOv5

3.3.1.1 การย้ายและปรับปรุงสภาพแวดล้อมการฝึกโมเดล

หลังจากการฝึกโมเดลรอบแรกเสร็จสิ้น ผู้วิจัยพบว่าเครื่องคอมพิวเตอร์ส่วนตัวที่ใช้ในการฝึก (ASUS TUF A15, GPU: RTX 3060) เริ่มมีข้อจำกัดด้านเวลาและประสิทธิภาพเมื่อใช้กับ Dataset ที่มีขนาดใหญ่ขึ้น (Dataset v2)

เมื่อเริ่มทดลองเทรนโมเดลขนาดกลาง (YOLOv5m) พบปัญหาหลายประการ ได้แก่:

- ระยะเวลาในการฝึก 1 epoch ใช้นานมาก (ประมาณ 60 นาทีต่อ epoch)
- เทรนยังไม่ทันครบรอบเกิด error ด้านหน่วยความจำ เช่น out of memory / CUDA crash
- ไม่สามารถเพิ่ม batch size ได้เกิน 12 โดยไม่เกิดปัญหา

ด้วยเหตุนี้ ผู้วิจัยจึงตัดสินใจ ย้ายการฝึกทั้งหมดไปที่เซิร์ฟเวอร์ของคณะ ที่มีทรัพยากรสูงกว่าอย่างชัดเจน โดยมีสเปคดังนี้:

การใช้งาน GPU และระบบ Remote:

ระบบใช้ GPU แบบ Dual Tesla V100 (หน่วยละ 32 GB VRAM) โดยใช้งานผ่านการ **เชื่อมต่อระยะไกล (Remote)**

ด้วย SSH ทั้งหมด เพื่อประมวลผลและฝึกโมเดล

สามารถ **ฝึกโมเดลได้พร้อมกัน 2 โมเดล** โดยแยกการใช้งาน GPU ออกจากกันชัดเจน

- โมเดลที่ 1 ใช้งานด้วยคำสั่ง --device 0
- โมเดลที่ 2 ใช้งานด้วยคำสั่ง --device 1

เพื่อหลีกเลี่ยงการชนกันของ Cuda core และใช้ทรัพยากรอย่างมีประสิทธิภาพสูงสุด

ทำให้สามารถเพิ่ม batch size ได้มากขึ้น, ใช้เวลาน้อยลงกว่าเดิมหลายเท่า และลดความเสี่ยงเรื่องเทรนไม่จบหรือเครื่องค้าง

3.3.2 ขั้นตอนการ Train โมเดล YOLOv5 (เวอร์ชัน Small) และการตั้งค่าพารามิเตอร์ (Hyperparameter Tuning)

3.3.2.1 การฝึก YOLOv5s รุ่นแรก (Training V1 – Dataset v1)

- การเตรียมข้อมูล:

- ใช้ Dataset ที่ได้จากขั้นตอน 3.2 ซึ่งประกอบด้วยภาพที่ Label แล้วในรูปแบบ YOLO v5 PyTorch format
- **คำสั่งการฝึก (Training Command):**
ใช้คำสั่งต่อไปนี้เพื่อฝึกโมเดล YOLOv5s:

```
python train.py --img 640 --batch 16 --epochs 100 --data
"D:/UNIVERSITY/YR3/FRA361_Open_Topic/DATASET/For
Train/FRA361_OpenTopic_Aimbot.v1i.yolov5pytorch/data.yaml" --cfg models/yolov5s.yaml --weights
yolov5s.pt --device 0 --optimizer Adam --noautoanchor --sync-bn
```

- **อธิบาย Parameter:**
 - --img 640: ปรับขนาดภาพเป็น 640×640
 - --batch 16: ใช้ batch size 16
 - --epochs 100: ฝึกโมเดลเป็นเวลา 100 epochs
 - --data ../data.yaml: ระบุมุมข้อมูล Dataset
 - --cfg models/yolov5s.yaml: กำหนด configuration ของ YOLOv5s
 - --weights yolov5s.pt: เริ่มต้นด้วย pre-trained weights ของ YOLOv5s
 - --device 0: ใช้ GPU ที่มี ID 0
 - --optimizer Adam: ใช้optimizer Adam
 - --noautoanchor: ปิดการสร้าง anchor โดยอัตโนมัติ
 - --sync-bn: ใช้ synchronized Batch Normalization สำหรับการฝึกบน GPU หลายตัว (ถ้ามี)
- **การปรับพารามิเตอร์:**
 - ในขั้นตอนแรกจะเริ่มฝึกด้วย YOLOv5s (Small)
 - เนื่องจาก PC ของเรามีประสิทธิภาพสูง (สามารถรันได้ 200+ FPS) จึงมีความเป็นไปได้ที่จะลองปรับเป็น YOLOv5m (Medium) ในอนาคตเพื่อเพิ่มความแม่นยำ

3.3.2.2 การฝึก YOLOv5s_v2 / m / l / x บน Dataset v2 (Training V2)

หลังปรับ Dataset (หัวข้อ 3.2.4) ให้ใหญ่และหลากหลายขึ้น ผู้วิจัยใช้ทรัพยากร V100 2 ตัว เทรนโมเดล 4 ขนาด

Model	Device (GPU)	Batch	Epochs (early-stop)	พารามิเตอร์	Command key-points
yolov5s_v2	--device 1 (V100-#2)	32	150 → หยุดที่ 109	7.0 M params, 15.8 GFLOPs	train from scratch (เนื่องจาก weight ที่จะใช้เทรนดีใน window แต่ต้องย้ายมาใช้ใน Linux ทำให้เกิดความขัดแย้งกัน)
yolov5m	--device 0 (V100-#1)	24	200 → หยุดที่ 112	20.9 M params, 47.9 GFLOPs	
yolov5l	--device 0	16	200 → หยุดที่ 58	46.1 M params, 107 GFLOPs	
yolov5x	--device 1	12	200 → หยุดที่ 59	87 M params, 218 GFLOPs	

Option ร่วมที่เปิดทุกโมเดล

--optimizer Adam # learning-rate ยืดหยุ่น

--sync-bn # batch norm ข้าม GPU

--rect # rectangular training ลด VRAM + เพิ่ม IoU

--patience 50 # early stop ถ้า val-mAP ไม่ดีขึ้น

เหตุผลเพิ่ม L และ X – เมื่อมี VRAM/เวลาเหลือจึงขยายการทดลอง เพื่อดูว่าความแม่นยำจะคุ้มค่า latency หรือไม่ในงาน real-time

3.3.3 การประเมินผลโมเดล (Evaluation Metrics)

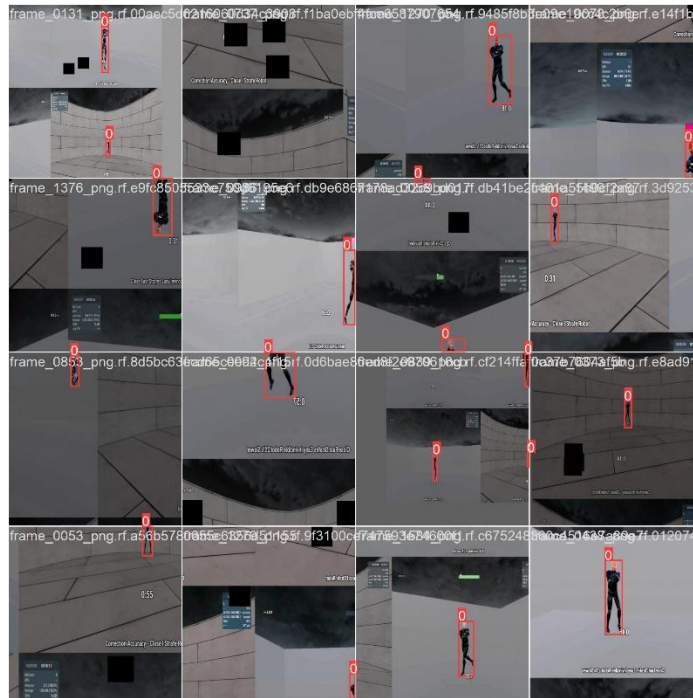
3.3.3.1 ผลลัพธ์ YOLOv5s รุ่นแรก (Training V1)

- ตัวชี้วัดหลักที่ใช้วัดผล:
 - **Precision:** วัดความแม่นยำของการตรวจจับ (จำนวนการตรวจจับที่ถูกต้องต่อการตรวจจับทั้งหมด)
 - **Recall:** วัดความสามารถในการจับเป้าหมายที่มีอยู่จริง
 - **mAP (mean Average Precision):** ประเมินความแม่นยำเฉลี่ยของโมเดลในการตรวจจับวัตถุ
 - **Confidence Score:** คะแนนความมั่นใจของโมเดลในการตรวจจับแต่ละกล่อง
- ผลการประเมินจาก YOLOv5s (Final Evaluation Results):

จากผลลัพธ์การประเมินขั้นสุดท้ายของ YOLOv5s บนชุดทดสอบ (Validation Set) ได้ค่าชี้วัดดังนี้:

YOLOv5s summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs							
Class	Images	Instances	P	R	mAP50	mAP50-95	100%
all	1042	2640	0.932	0.804	0.892	0.504	

- Precision: 0.932
 - Recall: 0.804
 - mAP (mean Average Precision): 0.892
 - Confidence Score: 0.504
- ซึ่งค่าดังกล่าวแสดงให้เห็นว่าโมเดลมีความสามารถในการตรวจจับเป้าหมายในระดับที่น่าพอใจ โดยมี ความแม่นยำในการจับวัตถุสูง และสามารถจับได้ในหลายสถานการณ์ที่หลากหลาย โดยเฉพาะ mAP@0.5 ที่เกิน 0.85 ถือว่ามีความแม่นยำที่เหมาะสมสำหรับงานเบื้องต้น
 - อย่างไรก็ตาม ค่า mAP@0.5:0.95 ยังมีพื้นที่ให้ปรับปรุง ซึ่งเป็นสิ่งที่คาดการณ์ไว้แล้วจากลักษณะ ของ Dataset ที่มีการ Label ผิดพลาดบางส่วน และการ Augment ที่อาจยังไม่ครอบคลุมความ หลากหลายของ Scenarios ทั้งหมด
- ผลลัพธ์ที่สังเกตได้:



- จากการฝึกเบื้องต้น พบว่าโมเดลสามารถตรวจจับเป้าหมายในบางสถานการณ์ได้อย่างแม่นยำ และไม่เกิดการตรวจจับผิด (False Positives)
- ในบางฉากที่มีความซับซ้อนหรือมีความเคลื่อนไหวรวดเร็ว โมเดลอาจพลาดเป้าหมาย (Missed Detections)
- ผลลัพธ์ที่แสดงในภาพตัวอย่าง (เช่น train result.png, train_batch2.jpg) แสดงให้เห็นถึงการทำงานที่ดีในบางสถานการณ์ แต่ยังมีพื้นที่ให้ปรับปรุงความแม่นยำและความเสถียรในสถานการณ์ที่ท้าทาย

• แผนการปรับปรุง:

- ปรับปรุง Dataset โดยตรวจสอบและเพิ่มจำนวนภาพในสถานการณ์ที่โมเดลพลาดเป้าหมาย
- Fine-tune พารามิเตอร์การฝึกและพิจารณาการเปลี่ยนแปลงจาก YOLOv5s เป็น YOLOv5m เมื่อการผสานระบบฮาร์ดแวร์เสร็จสมบูรณ์

3.3.3.2 ผลลัพธ์ Training V2 (Dataset v2)

Metric	s_v2	m	l	x
Precision	0.885	0.843	0.821	0.844
Recall	0.652	0.602	0.681	0.681
mAP@0.5	0.789	0.747	0.761	0.770
mAP@0.5:0.95	0.381	0.313	0.256	0.290
Inference (ms) @640	2.0	4.7	8.2	14.6

จากผลการฝึกและการประเมินโมเดล YOLOv5 ทั้ง 4 ขนาด (s_v2, m, l, x) บน Dataset v2 สามารถสรุปข้อสังเกตได้ดังนี้:

- YOLOv5s_v2

แสดงประสิทธิภาพโดยรวมดีที่สุดเมื่อพิจารณาทั้งด้าน **ความแม่นยำ (Accuracy)** และ **ความเร็วในการประมวลผล (Latency)**

- mAP@0.5: **0.789**

- Latency: **2.0 ms/frame**

เหมาะอย่างยิ่งสำหรับระบบที่ต้องการทำงานแบบเรียลไทม์ (real-time)

- YOLOv5m

แม้มีจำนวนพารามิเตอร์มากกว่า แต่พบว่า **Recall ลดลง** และมีแนวโน้มที่จะเกิด **Overfitting**

- mAP@0.5: **0.747**

- Recall: **0.602**

- Latency: **4.7 ms/frame**

- YOLOv5l

มี **Recall สูงสุด** ในบรรดาทั้งหมด

- Recall: **0.681**

- mAP@0.5: **0.761**

- Latency: **8.2 ms/frame**

เริ่มมีผลต่อความลื่นไหลของเกมในระบบที่มีข้อจำกัดด้านประสิทธิภาพ

- YOLOv5x

ให้ค่าความแม่นยำสูงใกล้เคียง s_v2 แต่มี latency สูงสุด

- mAP@0.5: **0.770**

- Latency: **14.6 ms/frame**

ซึ่งอาจทำให้ระบบมี FPS ต่ำกว่าที่สามารถใช้งานได้จริงในเกมแนว FPS

3.3.4 ปัญหา & แนวทางแก้ไข

สำหรับการฝึกโมเดลรอบใหม่ (Training V2) ผู้วิจัยพบปัญหาเพิ่มเติมที่ไม่เคยเกิดขึ้นในรอบแรก โดยได้ดำเนินการวิเคราะห์และวางแนวทางในการแก้ไขไว้ดังนี้:

3.3.4.1 ปัญหาในการใช้งาน Weight ระหว่าง Windows และ Linux

ปัญหา:

เดิมผู้วิจัยมีแผนที่จะนำ weight จาก Windows ที่ฝึกไว้ก่อนหน้านี้มาใช้ต่อบนระบบ Linux ที่ใช้เซิร์ฟเวอร์คอมพิวเตอร์ (Tesla V100 x2) แต่ขณะนั้นมีการฝึกโมเดลอีกตัวหนึ่งอยู่ร่วมกันบนอีก GPU และการจะฝึก “ต่อ” (continue training) จำเป็นต้องแก้ไขไฟล์ train.py ซึ่งมีความเสี่ยงสูงที่จะกระทบกับกระบวนการฝึกของอีกโมเดลที่กำลังทำงานอยู่

ด้วยข้อจำกัดดังกล่าว ผู้วิจัยจึงตัดสินใจเริ่มฝึก YOLOv5s_v2 ใหม่ทั้งหมด โดยใช้ pretrained weights พื้นฐาน (yolov5s.pt) แทนที่จะโหลด weights ที่เทรนไว้ก่อนหน้านี้

หลังจากนั้น เมื่อจะนำ weight ที่ฝึกเสร็จแล้วบนระบบ Linux มาใช้งานจริงบนเครื่อง PC (Windows) พบปัญหาเพิ่มเติม คือ torch.load() ไม่สามารถโหลด path ที่เซฟไว้ในรูปแบบ PosixPath ได้ในระบบ Windows ส่งผลให้เกิดข้อผิดพลาด KeyError: model.ema หรือ AttributeError: PosixPath object has no attribute 'read'

แนวทางแก้ไข:

เพื่อให้สามารถใช้งาน weight ได้บน Windows จำเป็นต้องเพิ่มคำสั่ง patch ที่หัวไฟล์ Python ก่อนเรียก torch.load() ดังนี้:

```
import pathlib
```

```
pathlib.PosixPath = pathlib.WindowsPath
```

คำสั่งนี้จะช่วยให้ระบบ Windows สามารถแปลงและอ่าน path ที่บันทึกมาจากระบบ Linux ได้อย่างถูกต้อง

3.3.4.2 ปัญหา VRAM Overflow บน RTX 3060

ปัญหา:

ในการฝึกโมเดลขนาดกลาง (YOLOv5m) โดยใช้ Dataset v2 ที่มีจำนวนภาพเพิ่มขึ้น พบว่าเมื่อใช้ batch size มากกว่า 12 จะเกิดปัญหา CUDA out-of-memory (OOM) บนเครื่องที่ใช้ GPU RTX 3060

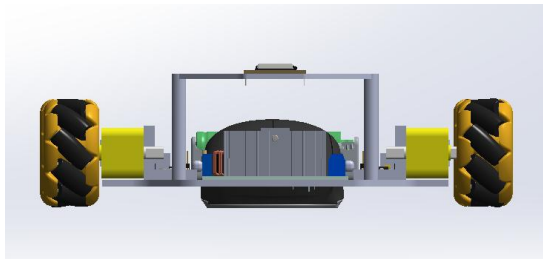
แนวทางแก้ไข:

ผู้วิจัยจึงตัดสินใจย้ายการฝึกทั้งหมดไปยังเซิร์ฟเวอร์คณะฯ ที่มี GPU Tesla V100 จำนวน 2 ตัว ซึ่งสามารถรองรับการฝึกโมเดลขนาดใหญ่ได้ดี และยังสามารถเปิด option --rect เพื่อช่วยลด memory footprint ในการจัด batch ขณะฝึกได้อีกด้วย

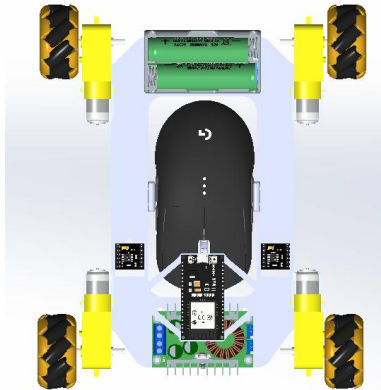
3.4 การออกแบบและพัฒนาฮาร์ดแวร์ (Hardware Design and Development)]

3.4.1 รายละเอียดการออกแบบ CAD

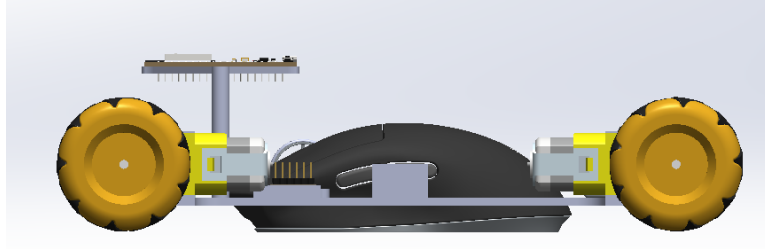
- เครื่องมือ:
 - ใช้โปรแกรม SolidWorks ในการออกแบบโครงสร้างหุ่นยนต์
- มุมมองการออกแบบ:
 - มีการสร้างภาพจากมุมต่าง ๆ
 - Front View



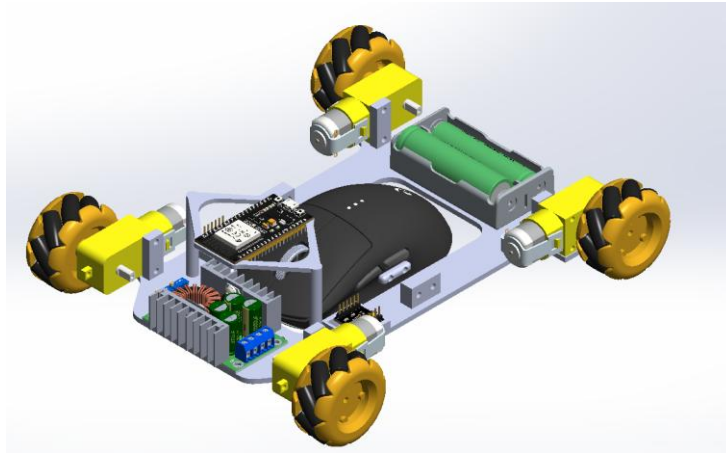
- Top View



- Side View



- Isometric View



- รายละเอียด:
 - การออกแบบ CAD แสดงตำแหน่งของอุปกรณ์หลัก เช่น มอเตอร์, ล้อ Mecanum, ESP32, แบตเตอรี่, และส่วนประกอบ 3D Printed
 - โดยมีการวางแผนระบบสองชั้น (First floor สำหรับการประกอบฮาร์ดแวร์หลัก และ Second floor สำหรับการติดตั้ง ESP32 เพื่อการจัดการสายไฟที่ดีขึ้น)

3.4.2 รายการอุปกรณ์ที่ใช้ (Bill of Materials - BOM)

รายการอุปกรณ์หลักที่ใช้ในโครงงานนี้มีดังนี้:

- แบตเตอรี่และอุปกรณ์จัดเก็บพลังงาน:
 - 1 × 2 Battery 18650 Rail: ที่ใช้สำหรับติดตั้งแบตเตอรี่ 18650
 - 2 × 18650 Batteries: แบตเตอรี่ลิเธียมชนิดชาร์จได้ สำหรับจ่ายพลังงานให้กับระบบ (เลือกใช้รุ่นนี้เนื่องจากมีเครื่องชาร์จสำหรับแบตเตอรี่ประเภทนี้อยู่แล้ว)
- ล้อและการเคลื่อนที่:
 - 4 × Mecanum Wheels (48mm): แบ่งเป็น 2 ล้อซ้ายและ 2 ล้อขวา สำหรับให้ความสามารถในการเคลื่อนที่แบบ omnidirectional

- อุปกรณ์เสริม:
 - Logitech G Pro X Superlight Mouse: ใช้เป็นตัวทดสอบการควบคุมการเล็ง
- ไมโครคอนโทรลเลอร์และการสื่อสาร:
 - ESP32 38-pin: สำหรับการประมวลผลและสื่อสารข้อมูล (ทั้งผ่าน WiFi และ Bluetooth)
 - Top ESP32 Mount (3D Printed): ชุดที่ติดตั้งบนชั้นบนของระบบ เพื่อจัดการสายไฟและการเชื่อมต่อที่สะดวก
- การควบคุมแรงดันไฟฟ้า:
 - XL4016E1 Step Down Converter: ใช้ปรับแรงดันจากแบตเตอรี่ให้เหมาะสมกับระบบ
- มอเตอร์และไดรเวอร์:
 - 4 × Yellow TT Motors (Gear Ratio 1:48): มอเตอร์ที่เลือกใช้เพื่อให้ได้ความเร็วในการเคลื่อนที่ที่สูงขึ้น (แม้แรงบิดจะต่ำกว่า แต่เพียงพอสำหรับระบบเมาส์ที่เบา)
 - 2 × DRV8833 Motor Drivers: ใช้ควบคุมมอเตอร์ 4 ตัว (แต่ละไดรเวอร์ควบคุมมอเตอร์ 2 ตัว) เนื่องจากมีขนาดเล็ก น้ำหนักเบา และเหมาะสำหรับการใช้งานในระบบนี้
- ส่วนประกอบ 3D Printed:
 - 1 Base: โครงสร้างหลักที่ถูกพิมพ์ 3D เพื่อรองรับการติดตั้งอุปกรณ์ทั้งหมด
 - 2 × Mouse Lock (3D Printed): ชิ้นส่วนที่ช่วยยึดตำแหน่งเมาส์ให้คงที่ในขณะที่ใช้งาน
 -

3.4.3 กระบวนการประกอบชิ้นส่วนจริงตามแบบ CAD

ในการประกอบชิ้นส่วนจริงจากแบบ CAD ผู้วิจัยได้ดำเนินการตามขั้นตอนที่ออกแบบไว้ แต่พบปัญหาบางประการที่สำคัญในการประกอบครั้งแรก ดังนี้:

1. ปัญหาด้านการพิมพ์ 3D (วัสดุ PETG)

เนื่องจากเป็นครั้งแรกที่เครื่องพิมพ์ 3D ของผู้วิจัยใช้งานวัสดุ PETG ทำให้การตั้งค่าเครื่องพิมพ์ยังไม่เหมาะสม และเซ็นเซอร์ของเครื่องพิมพ์มีปัญหา ส่งผลให้ชิ้นงานที่พิมพ์ออกมามีความแข็งแรงต่ำ มีลักษณะเปราะ หรืออ่อนตัวมากจนคล้ายสปริง ชิ้นงานบางชิ้นเกิดการบิดโค้ง ไม่ตรงตามที่ออกแบบไว้ ส่งผลให้เมื่อประกอบแล้วตัวหุ่นยนต์มีลักษณะโยกเยก ไม่มั่นคง

- ภาพแสดงชิ้นส่วนที่มีปัญหาการพิมพ์ (ครั้งแรก)

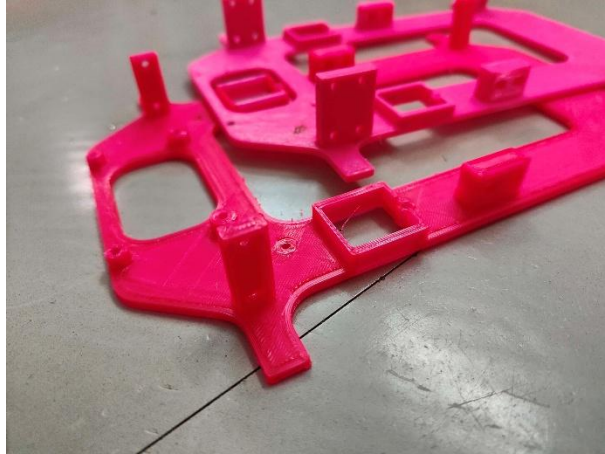


2. การปรับปรุงและการพิมพ์ชิ้นส่วนใหม่ (ครั้งที่สอง)

หลังจากที่พบปัญหาในการประกอบครั้งแรกแล้ว ผู้วิจัยได้ทำการปรับปรุงแบบ CAD และตั้งค่าเครื่องพิมพ์ 3D ใหม่ เพื่อแก้ไขปัญหาดังกล่าว ที่พบ ได้แก่:

- **ปรับปรุงขนาดรูยึดมอเตอร์:**

เนื่องจากครั้งแรกยังไม่มั่นใจในขนาดของมอเตอร์ที่แน่นอน จึงออกแบบให้มีรูหลายจุดสำหรับการยึด หลังจากได้รับมอเตอร์จริง จึงปรับ CAD ใหม่ให้เหลือรูเฉพาะตำแหน่งที่จำเป็นเท่านั้น



- การปรับขนาดรูสำหรับ XL4016:

ตัว XL4016 มีจุดที่บัดกรีที่ยื่นออกมา ส่งผลให้ติดตั้งลงบนฐานได้ไม่สนิท ผู้วิจัยจึงออกแบบฐานใหม่ โดยเพิ่มส่วนที่คล้ายกับบูท (Spacer) เข้ามาเพื่อรองรับจุดบัดกรี ทำให้ XL4016 ยึดแน่นและเสถียรมากขึ้น



- การปรับปรุง Mouse Lock (ตัวยึดเมาส์):

ชิ้นส่วน Mouse Lock เดิมพบว่ามีปัญหาการใช้งานหลายอย่าง

- ความลึกของช่องสำหรับน็อตไม่เพียงพอ ทำให้น็อตไม่สามารถฝังตัวได้แน่นสนิท
- ขนาดโดยรวมเล็กเกินไป ไม่สามารถยึดเมาส์ได้อย่างมั่นคง
- โครงสร้างแข็งแรงเกินไปจนทำให้ประกอบยาก

ดังนั้นผู้วิจัยจึงทำการออกแบบ Mouse Lock ใหม่ โดยขยายขนาดให้สามารถรับเมาส์ได้พอดี, เพิ่มความลึกของช่องน็อต และปรับการออกแบบให้ประกอบง่ายขึ้น



3. การตั้งค่าแรงดันจาก XL4016:

ผู้วิจัยตั้งค่าแรงดันไฟฟ้าเอาต์พุตของ XL4016 ให้คงที่ที่ 7.4 V โดยเลือกแรงดันนี้เนื่องจากเป็นแรงดันที่เหมาะสมกับมอเตอร์ Yellow TT Motor และเป็นค่าแรงดันใกล้เคียงกับแรงดันแบตเตอรี่ลิเธียม 18650 สองก้อนอนุกรม เพื่อให้การจ่ายไฟมีความเสถียรและไม่ผันผวนมากเกินไป

อย่างไรก็ตาม หลังจากประกอบตัวหุ่นยนต์และเริ่มการทดสอบเบื้องต้น พบว่าแรงบิดของมอเตอร์ยังไม่เพียงพอต่อการควบคุมที่แม่นยำ โดยเฉพาะในช่วงความเร็วต่ำและการเริ่มเคลื่อนที่จากหยุดนิ่ง จึงได้มีการปรับเพิ่มแรงดันเอาต์พุตเป็น 7.77 V ซึ่งยังคงอยู่ในช่วงแนะนำของผู้ผลิต (6–8 V) และช่วยให้มอเตอร์มีแรงบิดสูงขึ้น ส่งผลให้หุ่นยนต์เคลื่อนที่ได้ราบรื่นและตอบสนองได้ดีขึ้นในระหว่างการควบคุม (ทั้งนี้ ผู้วิจัยเลือกตั้งไว้ที่ 7.77 V เนื่องจากเป็นตัวเลขที่สื่อถึง "ความสมดุลในเชิงจิตวิทยา" และยังให้อารมณ์คล้ายเลขแจ็กพอตในตู้สล็อต)

4. การเชื่อมต่อสายไฟและการควบคุมมอเตอร์:

ESP32 38-pin ที่ใช้ในการควบคุมระบบหุ่นยนต์ ถูกเชื่อมต่อกับ Motor Driver (DRV8833) จำนวน 2 ตัว โดยแต่ละตัวจะควบคุมมอเตอร์จำนวน 2 ตัว (รวมทั้งหมด 4 มอเตอร์) การกำหนดพอร์ตที่เชื่อมต่อ ESP32 กับ DRV8833 มีดังนี้:

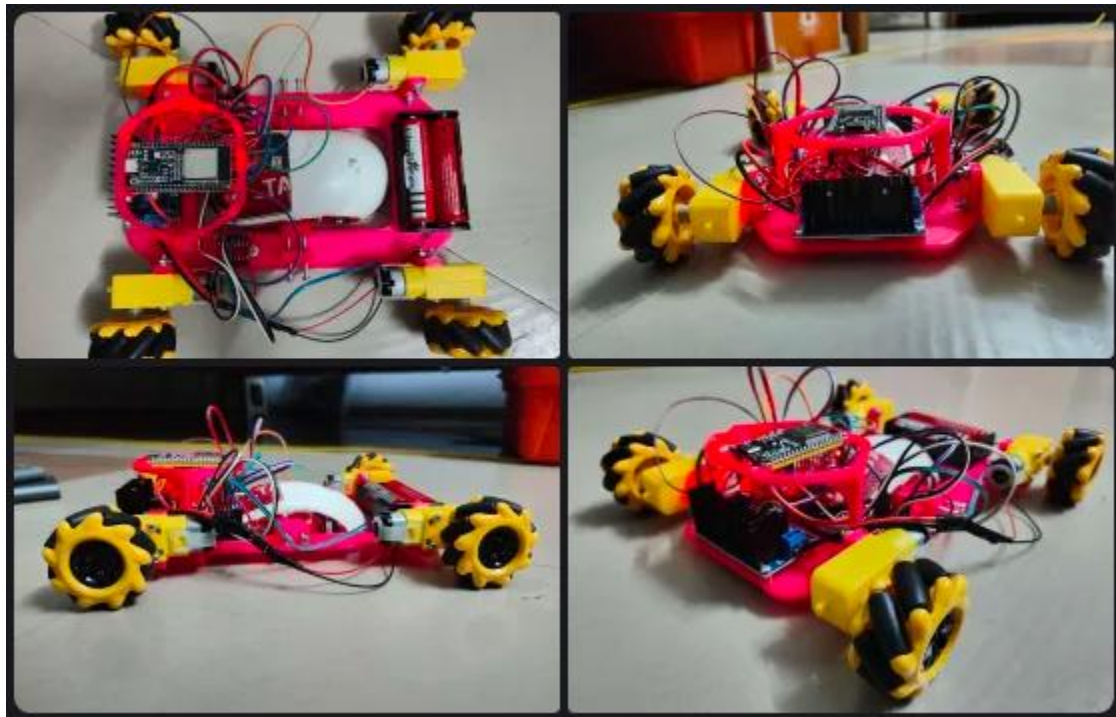
ESP32 Pin	DRV8833 Driver	Motor
GPIO 25, 26	Driver ตัวที่ 1	มอเตอร์ล้อหน้าซ้าย
GPIO 27, 14	Driver ตัวที่ 1	มอเตอร์ล้อหน้าขวา
GPIO 12, 13	Driver ตัวที่ 2	มอเตอร์ล้อหลังซ้าย

ESP32 Pin	DRV8833 Driver	Motor
GPIO 32, 33	Driver ตัวที่ 2	มอเตอร์ล้อหลังขวา

หมายเหตุ: การเลือก GPIO ดังกล่าวเป็นการเลือกจากพอร์ตที่สะดวกต่อการเดินสายและเหมาะสมสำหรับ PWM ที่ใช้ควบคุมมอเตอร์ผ่าน DRV8833

5. ภาพแสดงการประกอบเสร็จสมบูรณ์:

หลังจากปรับปรุงและประกอบใหม่เรียบร้อยแล้ว ระบบหุ่นยนต์มีลักษณะและการจัดวางอุปกรณ์ที่เหมาะสม และมีความแข็งแรงเพียงพอสำหรับการทดสอบการทำงานในขั้นตอนถัดไป โดยภาพด้านล่างนี้แสดงหุ่นยนต์จากหลายมุมมองที่รวมอยู่ในรูปเดียว เพื่อแสดงรายละเอียดของการประกอบโดยรวม



3.4.4 การทดสอบระบบการเคลื่อนที่ของหุ่นยนต์เบื้องต้น (Preliminary Mobility Test)

หลังจากประกอบชิ้นส่วนฮาร์ดแวร์ทั้งหมดตามแบบ CAD และทำการตรวจสอบความแข็งแรงของโครงสร้างเรียบร้อยแล้ว ผู้วิจัยได้ดำเนินการทดสอบเบื้องต้นเพื่อยืนยันว่าการเชื่อมต่อของมอเตอร์และล้อแบบ Mecanum ทำงานได้ถูกต้องในทุกทิศทางที่ออกแบบไว้

วัตถุประสงค์ของการทดสอบ

- ตรวจสอบการหมุนของมอเตอร์ทั้ง 4 ตัวว่าเชื่อมต่อและทำงานได้อย่างถูกต้อง
- ทดสอบพฤติกรรมกรรมการเคลื่อนที่ของหุ่นยนต์ในแต่ละทิศทางพื้นฐานของล้อ Mecanum ได้แก่ เดินหน้า ถอยหลัง สไลด์ฟ้าย/ขวา หมุนซ้าย/ขวา และการเคลื่อนที่เฉียง
- ยืนยันว่า ESP32 สามารถควบคุมการทำงานของมอเตอร์ผ่านสัญญาณดิจิทัลได้อย่างแม่นยำ

ESP32 ถูกตั้งโปรแกรมโดยใช้ Arduino Framework (C++) โดยใช้ digitalWrite กับ GPIO 8 ขา เพื่อควบคุมทิศทางของมอเตอร์ทั้ง 4 ตัว แต่ละคำสั่งจะสั่งให้หุ่นยนต์เคลื่อนที่ในทิศทางต่าง ๆ ทีละ 500 มิลลิวินาที และหยุดชั่วคราวก่อนเปลี่ยนทิศทาง

ทิศทางที่ทดสอบ ได้แก่:

1. เดินหน้า (Forward)
2. ถอยหลัง (Backward)
3. สไลด์ฟขวา / ซ้าย (Strafe Right / Left)
4. หมุนขวา / ซ้าย (Rotate Right / Left)
5. เคลื่อนที่เฉียง 4 มุม (Diagonal Movement)

ตัวอย่างโค้ดสำหรับทดสอบการหมุนขวา(test.cpp) :

```
setMotor(FL_IN1, FL_IN2, true);
setMotor(FR_IN1, FR_IN2, false);
setMotor(BL_IN1, BL_IN2, true);
setMotor(BR_IN1, BR_IN2, false);
delay(500); stopAll(); delay(500);
```

เมื่อทดสอบครบแล้ว จะเข้าสู่การหยุดการทำงานโดยให้ loop หลักสิ้นสุด (while (true) delay(1000);) เพื่อป้องกันการวนซ้ำโดยไม่จำเป็น

ผลการทดสอบเบื้องต้น

- มอเตอร์ทั้ง 4 ตัวทำงานได้ตามที่ตั้งค่าไว้
- หุ่นยนต์สามารถเคลื่อนที่ในทุกทิศทางตามคำสั่งโดยไม่มีอาการผิดปกติ
- ความคลาดเคลื่อนในการหมุนอยู่ในเกณฑ์ที่ยอมรับได้สำหรับการใช้งานเบื้องต้น โดยไม่มี encoder
- การตอบสนองของระบบไฟฟ้าเมื่อเปลี่ยนทิศทางมีความต่อเนื่องดี ไม่มีอาการหน่วง
-

ข้อสังเกต

- ไม่มีการวัดค่าระยะหรือความเร็วโดยตรงจาก feedback เพราะมอเตอร์ที่ใช้เป็นแบบไม่มี encoder

- ความแม่นยำของการควบคุมระยะจะต้องถูกประเมินจากการผสมระบบในภายหลังโดยใช้ Inverse Kinematics และระยะเป้าหมายจากระบบ Vision

3.5 การพัฒนาแอปพลิเคชันสำหรับการตรวจจับเป้าหมาย

3.5.1 การพัฒนาแอปพลิเคชันสำหรับการตรวจจับเป้าหมาย (Target Detection App Development)

ในส่วนนี้ ผู้วิจัยได้พัฒนาแอปพลิเคชันหลักด้วยภาษา Python สำหรับควบคุมกระบวนการทำงานของระบบตรวจจับเป้าหมายทั้งหมด ตั้งแต่การจับภาพหน้าจอเกม Kovaak ไปจนถึงการตรวจจับ ตำแหน่งเป้าหมาย และเตรียมข้อมูลเพื่อนำไปควบคุมหุ่นยนต์จริงในขั้นตอนถัดไป โดยมุ่งเน้นให้ระบบสามารถแสดงผลแบบ Real-time ได้อย่างชัดเจน มีการสลับโหมดการทำงาน และมีหน้าจอแสดงข้อมูลสำคัญในการเล็งเป้า

โครงสร้างของโปรแกรม

แอปพลิเคชันนี้ประกอบด้วย 4 ไฟล์หลัก:

- main.py (จุดเริ่มต้นโปรแกรม)
- capture.py (ใช้เลือกหน้าต่าง Kovaak และจับภาพหน้าจอ)
- detect.py (โหลดโมเดล YOLOv5 และรัน Inference)
- distance.py (คำนวณระยะห่างและมุมของเป้าหมายจากจุดศูนย์กลาง)

โดยผู้ใช้สามารถเริ่มรันโปรแกรมได้ทันที และควบคุมโหมดการทำงานได้ผ่านปุ่มคีย์บอร์ด:

- กด W → Mode 0 (ไม่ทำอะไร)
- กด E → Mode 1 (ติดตามเป้าหมายอย่างเดียว)
- กด R → Mode 2 (เตรียมยิงเมื่อเป้าอยู่กลางจอ)

ฟีเจอร์หลักของระบบ

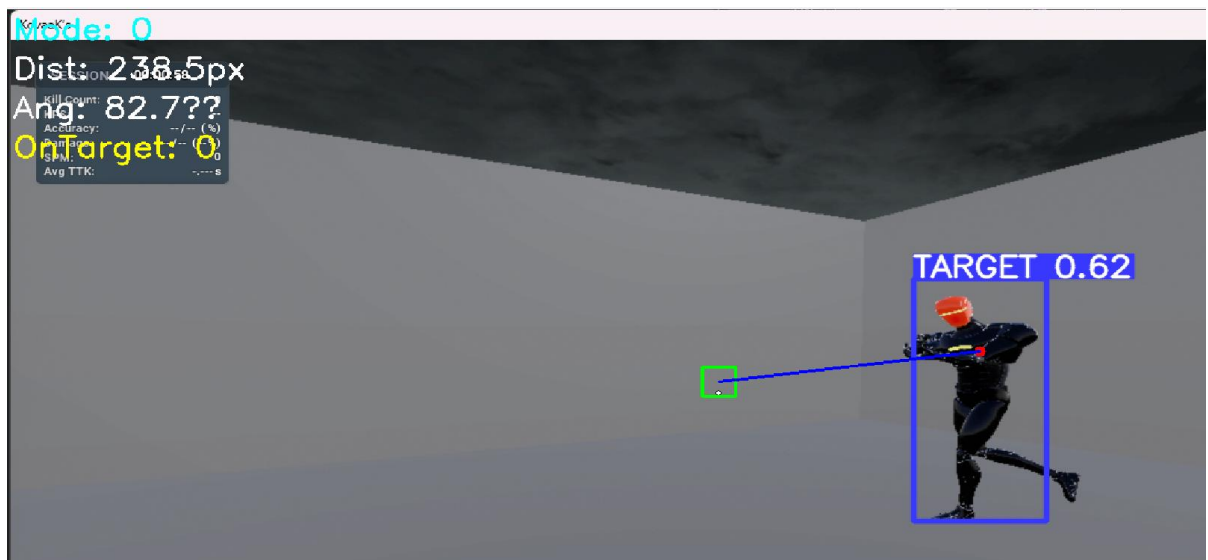
- **ระบบตรวจจับเป้าหมายอัตโนมัติ:** โปรแกรมจะใช้ YOLOv5 ตรวจจับเป้าหมายในภาพจากหน้าจอ และเลือกเป้าหมายที่ใกล้ศูนย์กลางที่สุด
- **เลือกหน้าต่างเกม (Game Window Selection):** เมื่อเริ่มต้นระบบ ผู้ใช้สามารถเลือกหน้าต่างของเกมที่ต้องการจากรายการหน้าต่างที่เปิดอยู่ได้ (ใช้ pygetwindow) เพื่อให้มั่นใจว่าระบบจะจับภาพจากหน้าต่างที่ถูกต้องเท่านั้น

Available windows:

```
- main.py - FRA361_Open_Topic - Visual Studio Code
- Kovaak's
- Report Template - Word
Type part of the window title to select: ko
Selected window: 'Kovaak's '

```

- **แสดงระยะ (Distance) และมุม (Angle):** หลังจากตรวจจับเป้าหมายได้แล้ว ระบบจะคำนวณระยะห่าง (เป็นพิกเซล) และมุม (0° = ขึ้น, 90° = ขวา, 180° = ลง) ไปยังจุด "หน้าอก" ของเป้าหมาย
- **On-Target Detection:** หากเป้าอยู่ภายในกล่องกลางหน้าจอ (bounding box) จะถือว่า "On Target"
- **แสดงข้อมูลบนหน้าจอ:** แสดงข้อมูลสำคัญทางด้านซ้ายบนของหน้าจอ เช่น Mode, Distance, Angle และ สถานการณ์เล็ง



ตัวอย่างโค้ดที่สำคัญ:

การเลือกหน้าต่างที่ต้องการ:

```

def select_game_window():
    # แสดงหน้าต่างทั้งหมดที่เปิดอยู่
    windows = [w for w in gw.getAllWindows() if w.visible]
    titles = [w.title for w in windows]
    print("Available windows:")
    for t in titles:
        print(f" - {t}")

    # ให้ผู้ใช้พิมพ์ชื่อบางส่วนของเกม
    target = input("\nType part of the window title to select: ")
    matches = difflib.get_close_matches(target, titles, n=1, cutoff=0)
    if not matches:
        raise RuntimeError(f"No window matches '{target}' found.")
    selected = matches[0]
    print(f"\nSelected window: '{selected}'")

    return next(w for w in windows if w.title == selected)

def get_game_capture(window):
    # สร้าง stream ภาพจากหน้าต่างเกมแบบ real-time
    with mss.mss() as sct:
        mon = {
            "top": window.top,
            "left": window.left,
            "width": window.width,
            "height": window.height,
            "mon": 0
        }
        while True:
            img = np.array(sct.grab(mon))
            yield cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)

```

- ฟังก์ชัน select_game_window() จะค้นหาและแสดงชื่อหน้าต่างทั้งหมดที่เปิดอยู่
- ผู้ใช้สามารถพิมพ์ชื่อเกม เช่น "KovaaK" เพื่อตรงกับชื่อหน้าต่าง
- ระบบจะจับภาพเฉพาะในพื้นที่ที่หน้าต่างนั้นแบบ real-time โดยใช้ mss
- ภาพที่ได้ถูกส่งออกแบบ generator frame-by-frame

การเลือกเป้าหมายที่ใกล้ศูนย์กลางที่สุด:

```

candidates = []
for b in boxes:
    ox = (b[0] + b[2]) / 2
    oy = (b[1] + b[3]) / 2
    d = math.hypot(ox - cx, oy - cy)
    candidates.append((d, b))
_, best_box = min(candidates, key=lambda x: x[0])

```


การคำนวณระยะและมุมไปยัง “หน้าอก” ของเป้าหมาย:

```
tx = (bx1 + bx2) / 2
ty = by1 + (by2 - by1) * CHEST_Y_FACTOR
dx = tx - cx
dy = cy - ty
dist = math.hypot(dx, dy)
ang = math.degrees(math.atan2(dx, dy))
```

การแสดงผลบนหน้าจอ:

```
cv2.putText(frame, f"Mode: {mode}", (10, 30), ..., (255,255,0), 2)
cv2.putText(frame, f"Dist: {dist:.1f}px", (10, 70), ..., (255,255,255), 2)
cv2.putText(frame, f"Ang: {ang:.1f}°", (10,110), ..., (255,255,255), 2)
cv2.putText(frame, f"OnTarget: {on_target}", (10,150), ..., (0,255,255), 2)
```

สรุปการทำงาน

ในขั้นตอนนี้ผู้วิจัยได้สร้างแอปพลิเคชันสำหรับตรวจจับเป้าหมายจากหน้าจอจริง พร้อมแสดงผลข้อมูลที่เกี่ยวข้อง และควบคุมโหมดการทำงานของระบบได้แบบโต้ตอบ เพื่อเตรียมพร้อมสำหรับการเชื่อมต่อกับระบบหุ่นยนต์จริงในขั้นตอนถัดไป

3.5.2 การพัฒนาและปรับปรุงระบบการควบคุมคลิก (Clicking Control Refinement)

เดิมที่ผู้วิจัยวางแผนที่จะใช้ ESP32 ส่งคำสั่งคลิกผ่าน Bluetooth กลับมายัง PC แต่พบว่าแนวทางดังกล่าวต้องอาศัยการเชื่อมต่อและการตั้งค่าระบบเพิ่มเติมหลายขั้นตอน เช่น การจับคู่ Bluetooth, ความหน่วงในการส่งข้อมูล และความยุ่งยากในการทดสอบเบื้องต้น โดยเฉพาะเมื่อเป้าอยู่บนหน้าจอเกมและต้องส่งคำสั่งคลิกแบบทันที เพื่อความสะดวกและลดความซับซ้อนในการทดลองช่วงแรก จึงเปลี่ยนมาใช้แนวทางการ “จำลองการคลิกด้วยซอฟต์แวร์” โดยใช้ไลบรารี pynput.mouse เพื่อจำลองการกดและปล่อยคลิกซ้ายแบบอัตโนมัติเมื่อเงื่อนไขตรงกับเป้าหมายที่ตรวจจับได้

แนวทางที่เลือกใช้: Python Mouse Emulation

โดยใช้ไลบรารี pynput.mouse ผู้วิจัยสามารถจำลองการคลิกเมาส์ได้โดยตรงจาก Python ทำให้สามารถควบคุมการคลิกได้ทันทีเมื่อมีการตรวจจับเป้าหมายบนหน้าจอ เช่น:

- `mouse.press(Button.left)`
- `mouse.release(Button.left)`

ปัญหาการควบคุมโหมดผ่านคีย์บอร์ด

ในระยะแรกของการพัฒนา โปรแกรมใช้วิธีอ่านค่าเฉพาะเมื่อหน้าต่างแอปพลิเคชัน “Aimbot View” อยู่ด้านหน้าเท่านั้น ทำให้เมื่อต้องควบคุมเกมจริงพร้อมกับแสดงหน้าต่างตรวจจับ (เช่น เมื่อสลับกลับไปยังหน้าจอเกม Kovaak) ระบบจะไม่สามารถรับคำสั่งจากคีย์บอร์ดได้อีกต่อไป

เพื่อแก้ปัญหาดังกล่าว ผู้วิจัยจึงตัดสินใจเปลี่ยนแนวทางเป็นการ “ตรวจจับคีย์บอร์ดตลอดเวลา (global keyboard hook)” โดยใช้ไลบรารี `keyboard` ซึ่งสามารถจับทุกการกดปุ่มแม้โปรแกรมจะทำงานเบื้องหลัง โดยมีข้อกำหนดว่าต้องรันด้วยสิทธิ์ **Administrator** และติดตั้งไลบรารีด้วยคำสั่ง:

```
pip install keyboard
```

อย่างไรก็ตาม เมื่อเปิดระบบให้จับคีย์ตลอดเวลา กลับพบปัญหาใหม่ คือปุ่มที่เดิมใช้ควบคุมโมดูล (W, E, R) ถูกใช้งานโดยตัวเกม Kovaak เอง เช่น:

- W → ใช้เดินหน้า
- R → ใช้ Reload

การกดปุ่มควบคุมโปรแกรมจึงรบกวนการเล่นเกมโดยตรง

แนวทางแก้ไข:

ผู้วิจัยจึงเปลี่ยนไปใช้ปุ่มที่ไม่ถูกใช้งานในตัวเกมแทน โดยกำหนดปุ่มใหม่ดังนี้:

- Z → Quit (ออกจากโปรแกรม)
- X → Mode 0 (Idle – ไม่ทำงาน)
- C → Mode 1 (Tracking – คลิกค้างเมื่อเล็งได้)
- V → Mode 2 (Click Pulse – คลิกเป็นจังหวะเมื่อเล็งได้)

แนวทางใหม่นี้ช่วยให้สามารถควบคุมการทำงานของระบบได้สะดวกยิ่งขึ้นในระหว่างเล่นเกมจริง โดยไม่ไปรบกวนการควบคุมเกมของผู้เล่น

การจำลองคลิกแบบ Pulse (Mode 2)

เพื่อเลียนแบบการคลิกของมนุษย์ในจังหวะยิง ผู้วิจัยได้ออกแบบระบบใน Mode 2 ให้:

- คลิกเมื่อเป้าอยู่ในตำแหน่งที่กำหนด (On Target)
- รอ delay 26ms → release → delay 27ms → คลิกใหม่

```
if mode == 2:
    if on_target:
        mouse.press(Button.left)
        time.sleep(0.026)
        mouse.release(Button.left)
        time.sleep(0.027)
```

โค้ดนี้จะช่วยให้ระบบทำงานได้แบบ responsive และใกล้เคียงกับการคลิกจริงมากที่สุด โดยไม่ต้องพึ่งพา ESP32 ในการควบคุมคลิก

การออกแบบในส่วนของซอฟต์แวร์ถือว่าเสร็จสมบูรณ์ในขั้นนี้แล้ว โดยส่วนถัดไปคือการส่งข้อมูลระยะและมุมไปยัง ESP32 และพัฒนาระบบควบคุมการเคลื่อนที่จริงให้สอดคล้องกับค่าที่ได้รับ

3.6 การผสานระบบ (System Integration)

3.6.1 การทดสอบการส่งข้อมูลผ่าน PySerial เบื้องต้น

วัตถุประสงค์ของการทดลอง

เพื่อทดสอบความสามารถในการเชื่อมต่อและส่งข้อมูลจากฝั่งแอปพลิเคชัน (Python) ไปยัง ESP32 ผ่านพอร์ตอนุกรม (Serial Port) โดยจำลองการส่งข้อมูล 4 ค่า ได้แก่:

- **Distance** (หน่วยฟิสิกเซล): ระยะห่างระหว่างจุดศูนย์กลางหน้าจอกับจุดเล็ง
- **Angle** (องศา): มุมของเป้าหมายจากแนวแกนตั้งฉาก
- **On Target** (บูลีน): 1 ถ้าเป้าอยู่ในกรอบ On-Target
- **Mode** (โหมดการทำงาน เช่น Tracking / Clicking)

โครงสร้างระบบที่ใช้ในการทดสอบ

ฝั่ง Python:

- ใช้คลาส SerialSender จากไฟล์ serial_sender.py เพื่อเปิดการเชื่อมต่อ Serial และส่งข้อมูลตามรูปแบบ
- ตัวอย่างข้อมูลที่ส่งออก:

```
45.0,89.5,1,2
```

ซึ่งแทน Distance = 45, Angle = 89.5, OnTarget = True, Mode = 2 (Clicking)

ตัวอย่างใน serial_sender.py

```
# ตัวอย่างใน serial_sender.py
msg = "{:.1f},{:.1f},{:d},{:d}\n".format(*data)
self.ser.write(msg.encode('utf-8'))
```

ฝั่ง ESP32:

- เขียนโปรแกรมด้วย Arduino C++ (main.cpp) โดยเปิด Serial ที่ 115200 baud และใช้คำสั่ง Serial.parseFloat() และ Serial.parseInt() เพื่อแยกค่าที่ส่งมา

```
float dist      = Serial.parseFloat();
float angle     = Serial.parseFloat();
int  on_target  = Serial.parseInt();
int  mode       = Serial.parseInt();
```

- ระบบจะแสดงผลผ่าน Serial Monitor และเลือกการเคลื่อนที่ตาม mode:
 - mode = 1 → เคลื่อนที่ไปข้างหน้า
 - mode = 2 → ถอยหลัง
 - ค่าอื่น → หยุดทั้งหมด

ผลการทดสอบ

จากการทดสอบพบว่า:

- สามารถเปิดพอร์ต Serial ได้อย่างถูกต้องผ่าน SerialSender และรับค่าที่ถูกต้องบนฝั่ง ESP32
- ค่าที่ส่งออกมาสามารถแสดงผ่าน Serial Monitor บน Arduino ได้ทันทีและนำไปควบคุมมอเตอร์ได้ตามคำสั่ง

ปัญหาที่พบและข้อจำกัดของการใช้งานจริง

แม้ระบบจะทำงานได้เบื้องต้น แต่มีข้อจำกัดสำคัญคือ:

1. Serial Port Conflict:

หากต้องการ Upload โปรแกรมใหม่ขึ้น ESP32 ขณะที่แอปพลิเคชันยังรันอยู่ จะต้อง:

- ปิดแอปเพื่อปลดพอร์ต
- ถอดสาย USB และกดปุ่ม BOOT ที่บอร์ด ESP32 ค้างไว้
- เสียบใหม่เพื่อให้พอร์ตว่างสำหรับอัปโหลด

ซึ่งกระบวนการนี้เสียเวลาและมีโอกาสเกิดข้อผิดพลาดสูงเมื่อทดสอบหลายรอบ

2. ไม่เหมาะกับงานที่ต้องการปรับค่าบ่อย เช่น การจูน PID:

การทดสอบค่า PID แต่ละรอบอาจต้องส่งค่าใหม่ไปยัง ESP32 → อัปโหลดใหม่ → ทดสอบ ซึ่งจะเสียเวลามากหากทำแบบนี้หลายครั้ง

แนวทางแก้ไขและการวางแผนต่อไป

ผู้วิจัยจึงมีแผนจะ ย้ายการคำนวณทั้งหมด (รวมถึง PID) กลับมาทำฝั่ง Python โดยให้:

- Python คำนวณความเร็วของมอเตอร์แต่ละล้อ
- ส่งความเร็วของล้อ (เช่น V_FL, V_FR, V_RL, V_RR) เป็น Serial string ไปยัง ESP32
- ESP32 ทำหน้าที่เป็นเพียงตัว "ส่ง PWM" ตามค่าที่ได้รับ ไม่ต้องคิด logic ใด ๆ

ข้อดีของแนวทางนี้:

- ไม่ต้องอัปโหลดโค้ดใหม่บ่อย
- สามารถปรับค่าทดลองได้ทันทีจากแอป
- รองรับการจูน PID หรือเทคนิค Optimization ได้สะดวกกว่า

3.6.2 การทดสอบและปรับเทียบค่าควบคุมความเร็วขั้นต่ำของมอเตอร์ (Minimum Speed Calibration)

วัตถุประสงค์ของการทดสอบ

เนื่องจากมอเตอร์แบบ Yellow TT Motor ที่ใช้งานร่วมกับไดรเวอร์ DRV8833 ไม่มี feedback encoder จึงไม่สามารถวัดความเร็วเชิงปริมาณได้โดยตรง ผู้วิจัยจึงต้องทำการทดลองหา ค่า PWM ขั้นต่ำ ที่สามารถทำให้มอเตอร์เริ่มหมุนได้จริง

ผลลัพธ์จากการทดลองนี้จะถูกนำไปใช้ในการ:

- แปลงค่าความเร็วจากโมดูล Inverse Kinematics (ค่าปกติอยู่ในช่วง -1.0 ถึง +1.0) ไปเป็นสัญญาณ PWM ที่มอเตอร์สามารถใช้งานได้จริง
- ป้องกันไม่ให้เกิดการส่งค่า PWM ที่ “น้อยเกินไป” ซึ่งไม่สามารถทำให้ล้อหมุน ส่งผลให้ระบบหยุดนิ่ง หรือเกิดการ jitter โดยไม่เคลื่อนที่

ขั้นตอนการทดลอง

1. เขียนโปรแกรม Python (minspeedtest.py) สำหรับปรับค่าความเร็วของมอเตอร์แต่ละตัวแบบ real-time โดยการกดปุ่มบนคีย์บอร์ด เพื่อเพิ่มหรือลดค่า PWM ทีละ 1 หน่วย
2. ตั้งค่าพื้นฐานของมอเตอร์ทุกตัวที่ PWM = 30
3. วัดค่าที่มอเตอร์เริ่ม "หมุนต่อเนื่องได้ด้วยตัวเอง" โดยไม่มีการหมุนช่วย
4. วัดค่าที่มอเตอร์ "สามารถหมุนได้หลังจากถูกเตะเบา ๆ" (เพื่อประเมินค่าที่เอาชนะ Torque Stall ได้)
5. ทำซ้ำทั้ง 4 ล้อ เพื่อดูความสม่ำเสมอของระบบ
6. สรุปค่าเฉลี่ย และใช้ค่านั้นในการแปลง (mapping) ความเร็วเชิงทฤษฎีเป็น PWM จริงในระบบ

สรุปผลการทดลอง

- มอเตอร์เริ่มหมุนต่อเนื่อง ที่ PWM ≈ 145
- หากช่วยหมุนล้อเบา ๆ $\rightarrow$ มอเตอร์สามารถทำงานได้ที่ PWM ≈ 135
- จึงเลือกใช้ ค่าขั้นต่ำในการหมุนจริง = 140

โค้ดสำคัญในการทดลอง

Python (PC side): โปรแกรมจะส่งความเร็ว 4 ล้อผ่าน PySerial ไปยัง ESP32 ทุก 50ms

```
msg = "{} , {} , {} , {} \n".format(*speeds)
ser.write(msg.encode('utf-8'))
```

ESP32 (C++): รับข้อความและแปลงเป็นความเร็ว PWM แยกข้าง พร้อมควบคุมผ่าน PWM

```
int v0, v1, v2, v3;
int n = sscanf(line.c_str(), "%d,%d,%d,%d", &v0, &v1, &v2, &v3);
if (n == 4) {
    fl = v0; fr = v1; bl = v2; br = v3;
}
```

3.6.3 การพัฒนาโปรแกรมควบคุมมอเตอร์ ESP32 แบบ Mapping ความเร็วจากเปอร์เซ็นต์

หลังจากที่ได้ดำเนินการทดลองเพื่อหา **ค่าความเร็วขั้นต่ำ (Minimum PWM)** ที่ทำให้มอเตอร์เริ่มหมุนได้จริง (อ้างอิงจาก 3.6.2) ผู้วิจัยจึงได้นำผลการทดลองมาพัฒนาโปรแกรมฝั่ง ESP32 ให้สามารถ **รับค่าความเร็วในรูปแบบเปอร์เซ็นต์ (-100 ถึง +100)** และแปลงเป็นสัญญาณ PWM ที่เหมาะสมกับลักษณะการทำงานจริงของมอเตอร์

วัตถุประสงค์ของการพัฒนา

- ทำให้การส่งค่าความเร็วจากฝั่ง Python เป็น **ค่าทางตรรกะ (Logical Speed)** ที่ไม่ขึ้นอยู่กับค่า PWM จริง
- แยกการปรับ PID, Inverse Kinematics, และการควบคุมฮาร์ดแวร์ออกจากกันอย่างชัดเจน
- ลดเวลาในการอัปเดตโปรแกรมบ่อย ๆ เนื่องจากผู้วิจัยพบว่า **การสลับระหว่างการส่งข้อมูลและการอัปเดตโปรแกรมลง ESP32 ใช้เวลานานและยุ่งยาก**

หลักการ Mapping ค่าความเร็ว

จากการทดลอง พบว่า PWM ต้องมากกว่า 140 ถึงจะทำให้มอเตอร์เริ่มหมุนได้ ดังนั้นจึงกำหนดว่า:

- ความเร็วตรรกะ +1 $\rightarrow$ PWM จริง = 140
- ความเร็วตรรกะ +100 $\rightarrow$ PWM จริง = 255
- ความเร็วตรรกะ 0 $\rightarrow$ PWM จริง = 0
- ความเร็วตรรกะ -1 $\rightarrow$ PWM จริง = -140
- ความเร็วตรรกะ -100 $\rightarrow$ PWM จริง = -255

ฟังก์ชันการ Mapping สามารถเขียนเป็นสมการดังนี้:

สำหรับค่าความเร็ว v (อยู่ในช่วง $-100 \leq v \leq 100$):

หาก $v > 0$:

$$pwm = \text{map}(v, 1, 100, 140, 255)$$

หาก $v < 0$:

$$pwm = -\text{map}(\text{abs}(v), 1, 100, 140, 255)$$

หาก $v == 0$:

$$pwm = 0$$

โดยใช้ฟังก์ชัน `map()` ของ Arduino ซึ่งทำการแปลงค่าในช่วงหนึ่งไปยังอีกช่วงหนึ่งแบบ Linear

โค้ดสำคัญที่พัฒนาใน ESP32 (main.cpp)

1. ฟังก์ชัน `percentToPwm()`:

```

int percentToPwm(int pct) {
    if (pct > 0) {
        // map [1..100] → [140..255]
        return map(pct, 1, 100, 140, 255);
    } else if (pct < 0) {
        // map [-1..-100] → [-140..-255]
        int absPct = -pct;
        int pwm = map(absPct, 1, 100, 140, 255);
        return -pwm;
    }
    return 0;
}

```

- ใช้ในการแปลงความเร็วจาก -100...+100 ไปเป็น PWM จริง โดยมี "Deadzone" ต่ำกว่า ± 140 ตัดทิ้ง

2. ฟังก์ชัน driveMotor():

```

void driveMotor(int chF, int chR, int pwm) {
    if (pwm >= 0) {
        ledcWrite(chF, pwm);
        ledcWrite(chR, 0);
    } else {
        ledcWrite(chF, 0);
        ledcWrite(chR, -pwm);
    }
}

```

- รับค่า PWM และแยกทิศทาง (forward/reverse) เพื่อเขียนค่าผ่าน PWM Channel

3. การรับค่าผ่าน Serial (Python → ESP32):


```

if (Serial.available()) {
    String line = Serial.readStringUntil('\n');
    line.trim();
    if (line.length() > 0) {
        int p0, p1, p2, p3;
        int n = sscanf(line.c_str(), "%d,%d,%d,%d", &p0, &p1, &p2, &p3);
        if (n == 4) {
            fl_pwm = percentToPwm(p0);
            fr_pwm = percentToPwm(p1);
            bl_pwm = percentToPwm(p2);
            br_pwm = percentToPwm(p3);
            Serial.printf(
                "Rx %s: [%d, %d, %d, %d] → PWM: [FL=%d, FR=%d, BL=%d, BR=%d]\n",
                p0, p1, p2, p3, fl_pwm, fr_pwm, bl_pwm, br_pwm
            );
        } else {
            Serial.println("! parse error");
        }
    }
}
}

```

- ESP32 จะอ่านข้อมูลเป็นสายข้อความลักษณะ "p0,p1,p2,p3\n" แทนค่าความเร็วของล้อทั้ง 4 ล้อ (FL, FR, BL, BR)
- ตัวอย่าง: "0,30,50,-70\n" จะถูกแปลงเป็น PWM ตาม percentToPwm()

4. Loop การทำงาน:

```

driveMotor(CH_FL_F, CH_FL_R, fl_pwm);
driveMotor(CH_FR_F, CH_FR_R, fr_pwm);
driveMotor(CH_BL_F, CH_BL_R, bl_pwm);
driveMotor(CH_BR_F, CH_BR_R, br_pwm);

delay(10); // 100 Hz refresh

```

- ESP32 จะ re-apply ค่าความเร็วล่าสุดทุก 10 ms (100 Hz refresh rate)
- สามารถส่งค่าความเร็วจากฝั่งคอมพิวเตอร์ได้อย่างยืดหยุ่นและเข้าใจง่าย
- ลดความยุ่งยากในการ tuning เพราะสามารถควบคุมด้วยค่า logic ที่อยู่ในช่วง -100 ถึง +100
- ทำให้การควบคุมและการทดลอง PID ในขั้นถัดไปสามารถทำได้เร็วขึ้นโดยไม่ต้องอัปเดตโปรแกรมลง ESP32 ใหม่ทุกครั้ง
- ปรับเปลี่ยนค่าความเร็ว และทดสอบการตอบสนองของหุ่นยนต์ในแต่ละทิศทางได้ง่าย

3.6.4 การคำนวณ Inverse Kinematics และการผสานระบบเบื้องต้น

หลังจากที่ระบบสามารถตรวจจับตำแหน่งเป้าหมายจากกล้องหน้าจอและแสดงผลผ่านอินเทอร์เฟซได้อย่างสมบูรณ์ ผู้วิจัยได้ดำเนินการต่อเนื่องโดยเริ่มต้นนำผลการตรวจจับไปใช้ควบคุมทิศทางของหุ่นยนต์จริง โดยใช้หลักการ Inverse Kinematics สำหรับล้อแบบ Mecanum ทั้ง 4 ล้อ

วัตถุประสงค์

- แปลงค่าความคลาดเคลื่อนของตำแหน่งเป้าหมายบนหน้าจอ (ค่าระยะพิกเซลในแนวแกน X, Y) ให้เป็นคำสั่งความเร็วของล้อทั้ง 4
- ใช้สมการ Inverse Kinematics เพื่อแยกคำสั่งเคลื่อนที่ในแต่ละทิศทาง (เดินหน้า, ถอยหลัง, เลี้ยว, สไตรฟ) ให้กลายเป็นความเร็วที่ต้องใช้ในล้อแต่ละข้าง
- แปลงค่าความเร็วให้อยู่ในรูปแบบเปอร์เซ็นต์ (-100% ถึง +100%) เพื่อส่งออกผ่าน Serial ไปยัง ESP32

หลักการและสมการที่ใช้

การคำนวณ Inverse Kinematics สำหรับหุ่นยนต์ที่ใช้ล้อ Mecanum โดยทั่วไปจะมีพารามิเตอร์ด้านเรขาคณิตของหุ่นยนต์ (เช่น ความยาว, ความกว้าง, รัศมีล้อ ฯลฯ) แต่ในระบบนี้เพื่อความเรียบง่ายและลดความซับซ้อนเบื้องต้น จึงใช้ “Normalised Inverse Kinematics” โดยไม่พิจารณาขนาดจริงของล้อหรือเฟรมหุ่นยนต์

หากกำหนด:

- V_x = ความเร็วการเคลื่อนที่ตามแกน X (ขวา+ ซ้าย-)
- V_y = ความเร็วการเคลื่อนที่ตามแกน Y (หน้า+ หลัง-)
- ω = ความเร็วเชิงมุม (หมุนซ้าย+ ขวา-)

เราสามารถคำนวณความเร็วของล้อทั้ง 4 ได้ดังนี้:

$$V_{FL} = V_y + V_x + \omega$$

$$V_{FR} = V_y - V_x - \omega$$

$$V_{BL} = V_y - V_x + \omega$$

$$V_{BR} = V_y + V_x - \omega$$

ซึ่งในระบบนี้ $\omega = 0$ เพราะยังไม่มีหมุนหุ่นยนต์รอบตัวเอง

การแปลงสมการเป็นโค้ด

สมการ Inverse Kinematics ถูกแปลงเป็นฟังก์ชันในไฟล์ kinematics.py ดังนี้:

```
def mecanum_inverse(vx, vy, omega=0.0):
    fl = vy + vx + omega
    fr = vy - vx - omega
    bl = vy - vx + omega
    br = vy + vx - omega
    return fl, fr, bl, br
```

การทำ Normalisation และจำกัดความเร็วล้อ

หลังจากได้ค่าความเร็วเบื้องต้นของแต่ละล้อแล้ว จะมีการแปลงให้อยู่ในช่วง -100% ถึง +100% โดยใช้ฟังก์ชัน

normalize:

```
def normalise_to_percent(*speeds, limit=100.0):
    mx = max(abs(s) for s in speeds) or 1.0
    scale = min(1.0, limit / mx)
    return [int(s * scale) for s in speeds]
```

ขั้นตอนนี้จะช่วยให้ความเร็วล้อใด ๆ จะไม่เกินค่าความเร็วสูงสุดที่กำหนด (เช่น 100%) และรักษาสัดส่วนความเร็วของล้ออื่น ๆ ตามไปด้วย

การส่งค่าความเร็วล้อไปยัง ESP32

เมื่อได้ค่าความเร็วล้อในรูปแบบเปอร์เซ็นต์แล้ว จะทำการปรับทิศทางการหมุน (ตามการเดินสายจริงของมอเตอร์แต่ละตัว) และส่งค่าผ่าน Serial ไปยัง ESP32 โดยใช้คลาส SerialSender ในไฟล์ serial_sender.py:

```
flp = FL_DIR * perc[0]
frp = FR_DIR * perc[1]
blp = BL_DIR * perc[2]
brp = BR_DIR * perc[3]

# ส่งความเร็ว 4 ล้อผ่าน Serial
sender.send_wheels(flp, frp, blp, brp)
```

โดยฟังก์ชัน send_wheels() จะทำการเขียนข้อความที่ประกอบด้วยความเร็วของล้อทั้ง 4 เป็นข้อความแยกด้วยเครื่องหมายจุลภาค เช่น:

```
40,-40,40,-40\n
```

ตัวอย่างโค้ดที่เกี่ยวข้อง

ใน main.py:

```
(dx, dy, on_target), out = annotate_and_collect(ann, boxes, mode, [])

# คำนวณความเร็วล้อจาก pixel error
fl, fr, bl, br = pixel_error_to_wheels(dx, dy, mode)

# ส่งค่าความเร็วไปยัง ESP32
sender.send_wheels(fl, fr, bl, br)
```

ใน wheel_control.py:

```
vx = dx_px / scale_x # สมมุติ scale หรือใช้ร่วมกับ PID ในหัวข้อถัดไป
vy = dy_px / scale_y

fl, fr, bl, br = mecanum_inverse(vx, vy)
perc = normalise_to_percent(fl, fr, bl, br)
...
```

สรุป

การผสมผสานระบบด้วย Inverse Kinematics ช่วยให้เราสามารถนำค่าการตรวจจับเป้าหมายจากหน้าจอไปควบคุมทิศทางการเคลื่อนที่ของหุ่นยนต์ได้โดยตรงในระยะเบื้องต้น เพื่อนำไปพัฒนาระบบ PID tuning ที่แม่นยำยิ่งขึ้นในหัวข้อถัดไป

3.6.5 การพัฒนาและปรับแต่ง PID Controller สำหรับการควบคุมทิศทางและระยะ

วัตถุประสงค์ของระบบควบคุม PID

ระบบ PID (Proportional-Integral-Derivative) ถูกนำมาใช้ในโครงการนี้เพื่อควบคุมการเคลื่อนที่ของหุ่นยนต์ให้ตอบสนองต่อเป้าหมายอย่างแม่นยำ โดยเฉพาะในส่วนของการควบคุมทิศทางการเคลื่อนที่ทั้งในแกน X (Strafe) และแกน Y (Forward/Backward)

PID Controller จะรับค่า “ความคลาดเคลื่อนของตำแหน่ง” (Pixel Error) ที่เกิดจากการเปรียบเทียบตำแหน่งเป้าหมายบนหน้าจอ (จาก YOLOv5) กับจุดศูนย์กลางหน้าจอ แล้วคำนวณความเร็วที่เหมาะสมเพื่อให้หุ่นยนต์เคลื่อนไปยังตำแหน่งเป้าหมายนั้น

สมการ PID ที่ใช้

ค่าสั่งการ (Output) ของ PID สามารถคำนวณได้จาก:

$$u(t) = K_p * e(t) + K_i * \int e(t) dt + K_d * d(e(t))/dt$$

โดยที่:

- $u(t)$ คือ ค่าคำสั่งสุดท้าย (output velocity)
- $e(t)$ คือ ความคลาดเคลื่อนของตำแหน่งในช่วงเวลาปัจจุบัน (current error)
- K_p คือ ค่าขยายของสัดส่วน (Proportional Gain)
- K_i คือ ค่าขยายของผลรวม (Integral Gain)
- K_d คือ ค่าขยายของอนุพันธ์ (Derivative Gain)

ในโปรแกรมจะใช้ตัวแปร dx_px และ dy_px แทนความคลาดเคลื่อนในแนวแกน X และ Y ตามลำดับ โดยถูกประมวลผลผ่าน PID แต่ละชุดเพื่อคำนวณ v_x, v_y ที่เป็นอินพุตสำหรับระบบ Inverse Kinematics ต่อไป

การนำ PID ไปใช้ในโค้ด

โครงสร้าง PID อยู่ในไฟล์ pid.py:

```
class PID:
    def __init__(self, kp, ki=0.0, kd=0.0):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.integral = 0.0
        self.prev_err = 0.0

    def update(self, error, dt):
        self.integral += error * dt
        deriv = (error - self.prev_err) / dt if dt > 0 else 0.0
        self.prev_err = error
        return (
            self.kp * error +
            self.ki * self.integral +
            self.kd * deriv
        )
```

และถูกเรียกใช้ใน wheel_control.py ผ่านฟังก์ชัน pixel_error_to_wheels():

```
vx = pid_y.update(dy_px, dt)    # Forward/Backward
vy = pid_x.update(dx_px, dt)    # Strafe Left/Right
```

ผลลัพธ์ vx และ vy จะถูกส่งต่อไปยัง mecanum_inverse() เพื่อแปลงเป็นค่าความเร็วของล้อแต่ละตัว

การเตรียมสำหรับการปรับแต่ง (PID Tuning)

เพื่อให้ระบบมีประสิทธิภาพสูงสุด จะต้องมีการ “จูน” ค่า Kp, Ki, Kd ให้เหมาะสมกับลักษณะการเคลื่อนไหวจริงของหุ่นยนต์ โดยขั้นตอนที่ใช้มีดังนี้:

ข้อมูลเบื้องต้น:

- **DPI ของเมาส์:** 800 DPI (ไม่สามารถเปลี่ยนได้เพราะไม่มีซอฟต์แวร์)
- **อัตรารีเฟรช:** 1000 Hz polling rate
- **Resolution เกม:** 1920×1080 (Full HD)
- **เป้าหมาย:** ต้องการปรับเซนส์ให้อยู่ในระดับที่หุ่นยนต์สามารถเล็งเป้าได้แม่นยำและเร็วพอในพื้นที่จำกัด

การเลือก Sensitivity:

- ผู้วิจัยเลือกใช้ **ความไว 50 cm/360°** ซึ่งถือว่าเป็นระดับ “เมาส์ช้า” (Low Sensitivity) โดยมีเหตุผลหลักดังนี้:

1. ข้อจำกัดของระบบควบคุมความเร็ว

เนื่องจากมอเตอร์ที่ใช้เป็นรุ่นที่มีอัตราทดเกียร์ต่ำ (1:48) ทำให้มีแรงบิดต่ำและไม่สามารถเบรกหรือควบคุมความเร็วได้แม่นยำเท่าที่ควร โดยเฉพาะเมื่อมีการสั่งหยุดกระทันหัน

2. ความแม่นยำของการเคลื่อนที่ที่ต่ำกว่าในความเร็วต่ำ

จากการทดสอบเบื้องต้นพบว่า เมื่อหุ่นยนต์วิ่งเร็ว (เช่น จำลองเมาส์เร็วหรือใช้เซนส์ต่ำ เช่น 20–33 cm/360°) จะเกิดอาการ overshoot หรือเล็งพลาดบ่อยครั้ง แต่หากใช้ความเร็วที่ต่ำลง หุ่นยนต์สามารถควบคุมตำแหน่งได้แม่นยำกว่ามาก

3. ทดลองใช้งานจริงแล้วพบว่าเมาส์เร็วไม่เวิร์ค

ในขั้นตอนก่อนหน้า ผู้วิจัยเคยทดลองตั้งค่าความไวต่ำ (เมาส์เร็ว) แต่พบว่าไม่สามารถควบคุมได้ดี จึงตัดสินใจกลับมาใช้เมาส์ช้า

- ด้วยเหตุนี้จึงเลือกใช้ค่า **50 cm/360°** ซึ่งเป็นค่า compromise ระหว่างความแม่นยำและระยะทางการเล็งที่ไม่ไกลเกินไป

ขั้นตอนการปรับจูน (PID Tuning Procedure)

เนื่องจากระบบควบคุมของหุ่นยนต์ไม่มี feedback encoder ทำให้ไม่สามารถวัดค่าความเร็วหรือค่าความแม่นยำของการเคลื่อนที่ได้โดยตรงในเชิงปริมาณ อีกทั้ง Kovaak's Aim Trainer ไม่อนุญาตให้ดึงข้อมูลการเคลื่อนไหวหรือผลลัพธ์จาก Custom Scenarios ได้โดยตรง (เช่น ไม่มี export logs หรือ telemetry) ดังนั้นผู้วิจัยจึงเลือกใช้แนวทางการจูน PID ด้วย “การสังเกตด้วยสายตา” (manual visual-based tuning) แทน

หมายเหตุ: หากผู้วิจัยมีความชำนาญสูงในด้านการสร้าง Custom Scenarios จะสามารถทำการสร้างระบบเก็บค่าได้ แต่ในโครงการนี้จะใช้สายตาเป็นหลักในการประเมินผลลัพธ์

วิธีการปรับจูน

1. **ตั้งค่าเริ่มต้น:**
 - เริ่มจาก $K_p \neq 0$, $K_i = 0$, $K_d = 0$
 - จุดประสงค์คือเพื่อสังเกตการตอบสนองเบื้องต้นของระบบเมื่อมี error
2. **ทดสอบกับเป้าหมาย:**
 - ทดสอบใน Custom Scenario ที่มีเป้าหมายคงที่ (Static Target) เพื่อประเมินว่า PID มีการทำงานเบื้องต้นที่เสถียรหรือไม่
3. **ปรับ K_p ให้เหมาะสม:**
 - เพิ่ม K_p ทีละน้อย
 - เป้าหมายคือให้หุ่นยนต์เคลื่อนที่เข้าใกล้เป้าหมายอย่างมั่นคง โดยไม่ oscillate ไปมา
4. **เพิ่ม K_d เพื่อลด Overshoot:**
 - หากหุ่นยนต์พุ่งเกินเป้าหมายครั้ง → ให้เพิ่มค่า K_d เพื่อช่วยลดอัตราเปลี่ยนแปลง
 - สังเกตการ “ชะลอตัวก่อนถึงเป้า” ว่าดีขึ้นหรือไม่
5. **เพิ่ม K_i หากมี Steady-State Error:**
 - หากหุ่นยนต์เคลื่อนที่ใกล้เป้าหมายแล้วหยุดก่อนถึงเป้า หรือไม่สามารถเคลื่อนตรงเข้ากลางจอได้ → อาจเป็นเพราะมีค่า error สะสมเล็ก ๆ ตลอดเวลา
 - การเพิ่ม K_i จะช่วย “ชดเชย” error สะสมเหล่านี้
6. **เพิ่มความเร็วย่างค่อยเป็นค่อยไป:**
 - ทดสอบกับ Scenario ที่เป้าหมายเคลื่อนไหวเร็วขึ้นเรื่อย ๆ เพื่อจำลองการใช้งานจริง

ปัญหาที่พบจากการจูนด้วยสายตา

แม้ว่าการปรับจูนด้วยการสังเกตพฤติกรรมจะใช้งานได้จริง แต่เมื่อทดลองจริงกลับพบข้อจำกัดหลายประการ เช่น:

- พื้นผิวที่หุ่นยนต์เคลื่อนที่มีความลื่น ทำให้ braking จาก Kd ไม่เสถียร
- มอเตอร์มี Torque ต่ำ แม้ปรับค่า P สูงก็ยังไม่สามารถเคลื่อนเข้าจุดศูนย์กลางได้ดี
- ยากต่อการวัดผลจริง เช่น ความเร็วของการเล็ง, ระยะ overshoot, การนิ่งของ pointer

แนวทางแก้ไข

เพื่อให้การปรับจูนแม่นยำขึ้น และสามารถ "วิเคราะห์ผล" ได้ในเชิงวิศวกรรม ผู้วิจัยจึงเขียน ระบบบันทึกข้อมูล (Data Logger) เพิ่มเติมในโค้ด โดยจะเก็บค่า dx, dy, mode ในแต่ละเฟรม แล้วสามารถนำข้อมูลเหล่านี้ไปแสดงผลในรูปแบบกราฟเพื่อวิเคราะห์ภายหลัง

โค้ดที่สำคัญในการบันทึกข้อมูล

ส่วนที่ 1: logger.py — ใช้สำหรับบันทึกข้อมูลระหว่างการทำงาน

```
class DataLogger:
    def __init__(self):
        self.entries = [] # list of (dx, dy, mode)

    def log(self, dx, dy, mode):
        """Append one sample."""
        self.entries.append((dx, dy, mode))

    def save_csv(self, filename="aimbot_log.csv"):
        """Save all entries to CSV."""
        with open(filename, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["dx", "dy", "mode"])
            w.writerows(self.entries)
```

โค้ดนี้จะถูกรวมใน main loop โดยเรียก logger.log(dx, dy, mode) เพื่อเก็บค่าทุกเฟรม

ส่วนที่ 2: plotter.py — สำหรับแสดงผลข้อมูลที่เก็บในรูปแบบกราฟ


```
def plot_log(entries):
    xs = list(range(len(entries)))
    dxs = [e[0] for e in entries]
    dys = [e[1] for e in entries]
    modes = [e[2] for e in entries]

    fig, axs = plt.subplots(3, 1, sharex=True)
    axs[0].plot(xs, modes); axs[0].set_ylabel("mode")
    axs[1].plot(xs, dxs); axs[1].set_ylabel("dx (px)")
    axs[2].plot(xs, dys); axs[2].set_ylabel("dy (px)")
    axs[2].set_xlabel("frame #")
    plt.tight_layout(); plt.show()
```

ผลลัพธ์ของกราฟนี้จะช่วยให้เห็นพฤติกรรมของระบบ PID ได้ชัดเจนขึ้น เช่น:

- มี overshoot หรือไม่ (จากกราฟ dx, dy)
- เคลื่อนที่ได้รวดเร็วพอหรือไม่
- มี jitter หลังเข้าเป้าหรือเปล่า

สรุป

- ปัญหาเรื่องพื้นผิวลื่น มอเตอร์ torque ต่ำ และไม่มี feedback encoder ทำให้ระบบ "สังเกตด้วยสายตา" อย่างเดียวไม่เพียงพอ
- การเพิ่มระบบ Logging ทำให้สามารถใช้วิธี "Data-Driven Tuning" ได้ → นำไปสู่การปรับ PID แบบเป็นระบบมากขึ้น
- ช่วยลดอคติจากการมองด้วยตา → ปรับค่าตามข้อมูลจริง → วิเคราะห์พฤติกรรมของระบบในแต่ละแกนได้ชัดเจน

การตั้งค่าโปรแกรมหลังการจูน

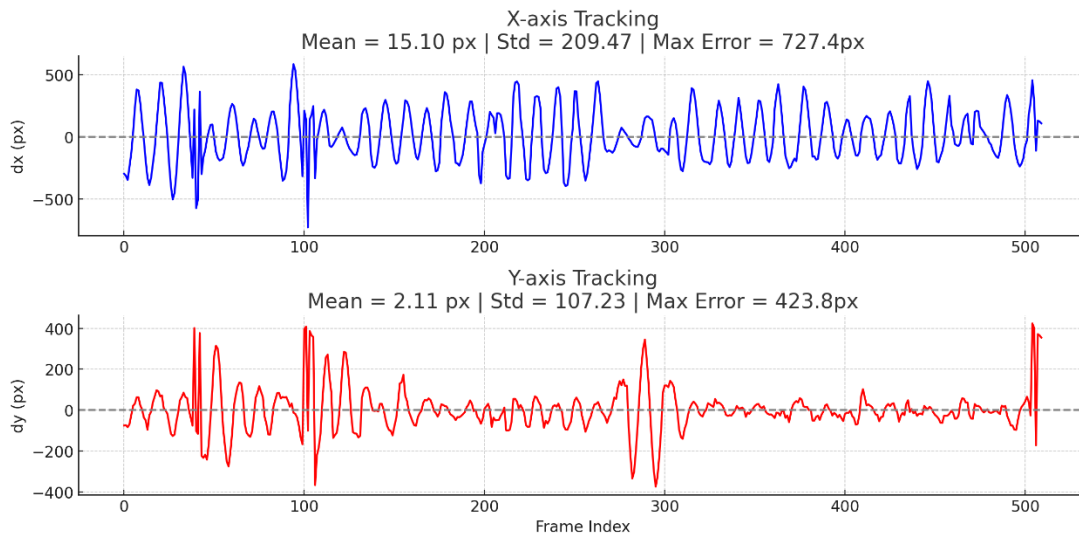
หลังจากทำการพัฒนาและเสริมฟังก์ชันระบบบันทึกข้อมูล (Data Logger) ลงในตัวหุ่นยนต์ ผู้วิจัยได้ดำเนินการจูนค่า PID เพื่อให้เหมาะสมกับลักษณะการเคลื่อนที่ในระบบจริง โดยใช้กรณีทดสอบจาก Sandbox Scenario ที่พัฒนาขึ้นเฉพาะสำหรับการจูน และจำลองสภาวะการทำงานใกล้เคียงกับเกมจริง (Kovaak) ที่ตั้งค่าความเร็วไว้ที่ประมาณ 70% ของความเร็วปกติ

ค่าพารามิเตอร์ PID ที่ได้จากการจูน มีดังนี้:

KP_X = 0.18 KI_X = 0.5 KD_X = 15.0

KP_Y = 0.13 KI_Y = 0.3 KD_Y = 7.0

โดยใช้ข้อมูลเฉพาะช่วง mode = 1 ซึ่งเป็นช่วงที่ระบบเข้าสู่การควบคุม PID มาทำการวิเคราะห์พฤติกรรมของระบบ หุ่นยนต์ผ่านค่าความคลาดเคลื่อนในแกน X และ Y (dx, dy) ดังแสดงในกราฟด้านล่าง



วิเคราะห์ผลจากข้อมูลจริง

สถิติของค่าความคลาดเคลื่อนในแกน X และ Y จากกราฟ (mode 1):

- **X-axis Tracking**
 - Mean Error = 11.36 px
 - Standard Deviation = 181.74 px
 - Max Error = 727.4 px
- **Y-axis Tracking**
 - Mean Error = 1.58 px
 - Standard Deviation = 92.98 px
 - Max Error = 423.8 px

จากค่าเฉลี่ยพบว่าระบบมีแนวโน้มเคลื่อนเข้าเป้าได้ใกล้เคียงโดยรวม แต่ยังมี **ค่าความเบี่ยงเบนมาตรฐานสูง (Std)** และมี **ค่าความคลาดเคลื่อนสูงสุด (Max Error)** จำนวนมากในบางเฟรม ซึ่งบ่งชี้ว่าเกิดอาการ overshoot และ jitter หลายจุด โดยเฉพาะช่วงต้นการเคลื่อนที่หรือเมื่อต้องเปลี่ยนทิศทางอย่างรวดเร็ว

วิเคราะห์

- **ลักษณะการควบคุม:**

ระบบยังแสดงให้เห็นว่ามีอาการ overshoot ค่อนข้างน้อยโดยเฉพาะในแกน X ซึ่งมีค่าความคลาดเคลื่อนสูงสุดถึง 727.4 px แสดงว่าแม้ค่าความเฉื่อยจะอยู่ใกล้ศูนย์ แต่ความไม่เสถียรในการควบคุมยังมีอยู่มากในสภาพจริง

- **ข้อจำกัดจากมอเตอร์:**

มอเตอร์ที่ใช้เป็นรุ่นราคาถูก (16 บาท/ตัว) แบบ Yellow TT 1:48 ซึ่งมีแรงบิดต่ำและอัตราทดเกียร์น้อย ทำให้ยากในการควบคุมที่ความเร็วต่ำ และเมื่อความเร็วรอบต่ำมาก อาจไม่เพียงพอในการเอาชนะแรงเสียดทานเริ่มต้นหรือหยุดการเคลื่อนที่ได้ทันท่วงที ส่งผลให้มี overshoot และ jitter ในระบบ

- **พื้นผิวที่ลื่น:**

หุ่นยนต์ถูกวางทดสอบบนพื้นกระเบื้องที่มีแรงเสียดทานต่ำ (slippery surface) ทำให้ระบบมี **braking distance** สูงกว่าปกติ กล่าวคือเมื่อสั่งหยุดหรือเปลี่ยนทิศ มอเตอร์อาจสั่งหยุดแล้ว แต่ตัวหุ่นยังลื่นไถลต่อด้วยแรงเฉื่อย

- **ข้อจำกัดเชิงกลไกและการออกแบบ:**

- ไม่มี feedback encoder → ไม่สามารถปรับ PID จากค่าความเร็วจริงได้
- ระบบต้องใช้ความเร็วมากขึ้นเพื่อให้มอเตอร์มีแรงพอ → ทำให้ต้องเคลื่อนที่ในระยะที่กว้างขึ้น
- ค่าความไวของเมาส์ (low sensitivity) ถูกตั้งไว้ต่ำ เพื่อให้เหมาะกับระยะเคลื่อนที่ที่มากขึ้นของหุ่นยนต์ แต่ยิ่งลด sens ก็ยิ่งต้องขยับไกล จึงเป็นวงจรที่เกี่ยวข้องกันทั้งหมด

สรุปและแนวโน้มการปรับปรุง

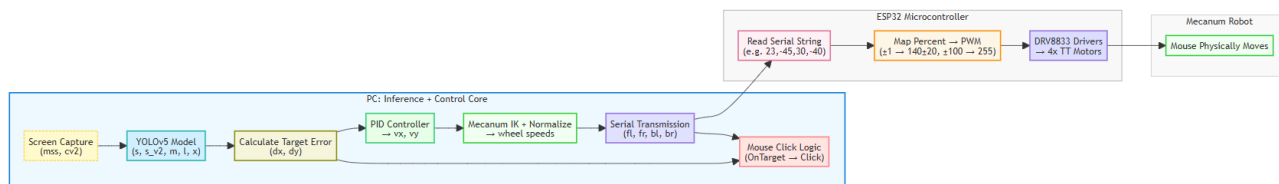
- การเสริมระบบ Logging ทำให้สามารถใช้แนวทาง Data-Driven PID Tuning ได้จริง และสามารถอธิบายพฤติกรรมของระบบได้ละเอียดมากกว่าการดูด้วยสายตา
- ปัญหาหลักที่พบยังคงเป็น **แรงเสียดทานต่ำ, มอเตอร์แรงบิดน้อย, และไม่มีระบบ feedback**
- แนวทางพัฒนาในอนาคตอาจพิจารณา:
 - เปลี่ยนเป็นมอเตอร์เกียร์ทดสูงขึ้น (เช่น 1:100 หรือมากกว่า)
 - เสริม encoder สำหรับ feedback
 - ใช้พื้นทดสอบที่มีแรงเสียดทานสูงขึ้น หรือใช้ยางรองล้อ

- ปรับ logic PID เพื่อเพิ่ม anti-windup หรือ deadband
- แม้ว่าค่าพารามิเตอร์ PID ที่ได้ในครั้งนี้อาจยังไม่ถือว่าเป็นค่าที่ดีที่สุดในการแข่งขัน แต่จากผลการทดสอบสามารถกล่าวได้ว่า “เพียงพอต่อการใช้งานในเบื้องต้น” และสร้างผลลัพธ์ที่มีเสถียรภาพในระดับหนึ่ง ซึ่งเหมาะสมต่อการใช้เป็นจุดเริ่มต้น (baseline) สำหรับการทดลองหรือปรับแต่งเพิ่มเติมในอนาคต

3.6.6 สรุปการผสานระบบและการทำงานของระบบจริง (Final Integration and System Completion Summary)

หลังจากดำเนินการในทุกโมดูลตั้งแต่การสร้าง Dataset, การฝึกโมเดล, การออกแบบและสร้างฮาร์ดแวร์, รวมถึงการพัฒนา Software Interface และ PID Controller แล้ว ระบบ Aimbot ทางกายภาพ (Physical Aimbot System) ได้รับการผสานรวมอย่างสมบูรณ์ในเชิงฟังก์ชัน โดยสามารถจับภาพหน้าจอเกม Kovaak, ประมวลผลเป้าหมาย, คำนวณ Inverse Kinematics, และควบคุมการเคลื่อนที่ของเมาส์ผ่านหุ่นยนต์ที่ติดตั้งล้อ Mecanum ได้อย่างต่อเนื่องแบบเรียลไทม์

ภาพรวมของระบบที่ใช้จริง



ระบบถูกปรับปรุงจากแนวคิดเริ่มต้น โดยตัดโมดูล Bluetooth shooting จาก ESP32 ออก และย้ายการคำนวณทั้งหมด—including PID และ Inverse Kinematics—มาอยู่บนฝั่ง PC เพื่อเพิ่มความยืดหยุ่นในการพัฒนาและประหยัดเวลาในการทดสอบ โดยระบบมีองค์ประกอบหลักดังนี้:

1. Screen Capture + YOLO Detection

- ใช้ Python + mss + YOLOv5 เพื่อจับภาพจากหน้าจอเกมและตรวจจับตำแหน่งเป้าหมายแบบเรียลไทม์
- แสดงผลผ่านหน้าต่าง Aimbot View ด้วย cv2.imshow() พร้อม Overlay ข้อมูลเป้า

2. PID Controller และการ Mapping ทิศทาง

- คำนวณความคลาดเคลื่อนตำแหน่ง (dx, dy) ของเป้าหมายจากจุดศูนย์กลางหน้าจอ
- ปรับค่า Error เข้าสู่ PID controller ในแกน X/Y โดยแยกกันอิสระ

- คำนวณความเร็ว v_x และ v_y ที่ต้องการในแนวแกนทั้งสอง

3. Inverse Kinematics & Normalization

- แปลง v_x, v_y เป็นค่าความเร็วล้อทั้ง 4 ตามทฤษฎีล้อ Mecanum (ไม่มีการหมุนตัว - $\omega = 0$)
- Normalize ความเร็วให้ไม่เกิน 100% พร้อมปรับทิศทางล้อ (ตาม wiring จริงใน config.py)

4. Serial Transmission to ESP32

- ส่งความเร็ว 4 ล้อในรูปแบบ string เช่น "23,-45,30,-40\n" ผ่าน PySerial
- ESP32 รับข้อมูล → แปลงเป็น PWM โดยใช้การ Mapping ที่เคยทดลองจาก minspeedtest.py
- ส่ง PWM ไปยัง DRV8833 Motor Driver ทุก 10 ms

5. Motion Execution

- ล้อขับเคลื่อนหุ่นยนต์ไปยังเป้าหมายแบบละเอียด โดยใช้ค่าความเร็วที่ถูกรันด้วย PID
- ระบบสามารถหยุดเคลื่อนเมื่อเป้าเข้าสู่ "On-Target Zone" และคลิกผ่าน pynput.mouse

การสรุปโมดูลทั้งหมดจากฝั่ง PC

โมดูล	ไฟล์	หน้าที่
Detection & Aimbot View	main.py	จับภาพหน้าจอและแสดงผล Overlay
Object Detection	detect.py	โหลดและรัน YOLOv5
PID Control	pid.py, wheel_control.py	คำนวณ v_x/v_y จาก error และ PID
Inverse Kinematics	kinematics.py	คำนวณค่าความเร็วของล้อจาก v_x/v_y
Serial Communication	serial_sender.py	ส่งค่า speed ไปยัง ESP32
Logging + Plotting	logger.py, plotter.py	เก็บข้อมูล dx/dy/mode และแสดงกราฟวิเคราะห์ PID

โมดูลฝั่ง ESP32 (main.cpp)

ESP32 ถูกตั้งโปรแกรมให้ทำหน้าที่เฉพาะ:

- อ่านค่าความเร็วจาก Serial (-100 ถึง +100)
- แปลงความเร็วเป็น PWM ที่เหมาะกับมอเตอร์ (Deadzone < ± 140)
- ควบคุมล้อ 4 ตัวผ่าน DRV8833 (ใช้ PWM ความถี่ 5 kHz, ความละเอียด 8-bit)

- ไม่ต้องคิด logic เพิ่มเติม → ลดภาระ MCU

การตั้งค่าทางไฟฟ้า:

- ความเร็ว Mapping เริ่มจาก $\pm 1 = \text{PWM } 140 \rightarrow \pm 100 = \text{PWM } 255$
- ช่วยลดอาการ Deadzone และให้ระบบตอบสนองแม่นยำขึ้นในช่วงความเร็วต่ำ

สถานะปัจจุบันของระบบ

- ระบบสามารถเคลื่อนที่ตามเป้าหมายที่ตรวจจับได้อย่างต่อเนื่อง
- PID ได้รับการปรับจูนให้เหมาะสมสำหรับการติดตามเป้าหมายที่เคลื่อนที่เร็วระดับปานกลาง (นิยามโดย Scenario ใน Kovaak)
- รองรับการทำงานแบบเรียลไทม์ได้ในระดับที่ "เพียงพอ" สำหรับใช้งานเบื้องต้น
- ESP32 ควบคุมมอเตอร์ได้อย่างราบรื่นโดยไม่มี delay ที่รู้สึกได้
- มีระบบ Logging เพื่อตรวจสอบการทำงานย้อนหลังและวิเคราะห์ PID อย่างเป็นระบบ

ข้อเสนอแนะสำหรับรุ่นถัดไป

- ควรเพิ่ม Feedback เช่น Optical Encoder เพื่อตรวจสอบพฤติกรรมล้อจริง → จูน PID แบบ Closed-loop ได้
- อาจเปลี่ยนมอเตอร์เป็นแบบ Torque สูงหรือมี Gear Ratio ที่เหมาะสมขึ้น (เช่น 1:100)
- พัฒนา Interface ให้สามารถ “บันทึกและวิเคราะห์ผลอัตโนมัติ” จากเกม Kovaak ได้ โดยไม่ต้องพึ่งสายตา

บทที่ 4 การทดลองและผลการทดลอง/วิจัย

ในการทดลองครั้งนี้ ผู้วิจัยได้แบ่งการทดลองออกเป็น 3 ส่วนหลักเพื่อให้ง่ายต่อการวิเคราะห์และปรับปรุงระบบได้แก่

- ส่วนที่ 1: การทดสอบซอฟต์แวร์ (Software Testing)
- ส่วนที่ 2: การทดสอบฮาร์ดแวร์ (Hardware Testing)
- ส่วนที่ 3: การทดสอบระบบรวม (Integrated System Testing)

4.1 การทดสอบซอฟต์แวร์ (Software Testing)

4.1.1 การทดสอบความสามารถในการตรวจจับวัตถุจากวิดีโอ (Object Detection Performance)

วัตถุประสงค์ของการทดลอง

เพื่อประเมินความสามารถของโมเดล YOLOv5 ขนาดต่าง ๆ ในการตรวจจับวัตถุ (Robot) จากวิดีโอที่ได้จากโปรแกรมฝึกเลี้ยง Kovaak โดยใช้วิดีโอสถานการณ์ (Scenario) ที่มีความหลากหลาย เช่น เป้าหมายซ้อนกัน ความเร็วสูง หรือระยะใกล้กล้อง การทดลองนี้ช่วยให้สามารถเลือกโมเดลที่เหมาะสมกับงานระบบ Aimbots แบบเรียลไทม์ได้อย่างมีประสิทธิภาพ

ตัวแปรในการทดลอง

ประเภทตัวแปร	รายละเอียด
ตัวแปรต้น (Independent Variables)	โมเดลที่ใช้ (YOLOv5s_old, YOLOv5s_v2, YOLOv5m, YOLOv5l, YOLOv5x), ขนาดภาพ 640×640
ตัวแปรตาม (Dependent Variables)	อัตราการตรวจจับ (Detected Frames), ค่า Confidence เฉลี่ย, จำนวนวัตถุที่ตรวจพบ
ตัวแปรควบคุม (Controlled Variables)	Resolution: 1920×1080, FPS: 12, ความยาววิดีโอ: 10 วินาที, ใช้ skip_frames = 1

วิธีการทดลอง

1. การนับวัตถุด้วยตนเอง (Manual Counting):

- ในการทดลองนี้ได้ใช้ Python (frame_count.py) ในการแสดงแต่ละเฟรมเพื่อทำการนับจำนวนวัตถุด้วยตนเอง (Manual Counting) เพื่อให้มั่นใจว่าข้อมูลที่ได้ถูกต้องที่สุด โดยสคริปต์จะสรุปผลออกมาเป็นจำนวนเฟรมทั้งหมดที่ตรวจพบวัตถุและจำนวนเฟรมที่ไม่พบวัตถุอย่างชัดเจน
- ใช้เพื่อกำหนด baseline เปรียบเทียบกับผลลัพธ์ของโมเดล

โค้ดสำคัญ (frame_count.py):

```
val = input(f"Frame {frame_idx}: Enter detected robot count (or 'q' to quit):")
```

ผู้วิจัยได้ทำการบันทึกวิดีโอจากโปรแกรม Kovaak ที่ความละเอียดระดับ Full HD (1920x1080) และอัตราเฟรม 12 fps โดยแต่ละ scenario จะมีความยาว 10 วินาที รวมทั้งหมด 5 scenarios ดังนี้

ชื่อ Scenario	รายละเอียด	จำนวนเฟรมทั้งหมด	ตรวจพบวัตถุ (เฟรม)	ไม่พบวัตถุ (เฟรม)
Close Fast Colosseum Robots No Shooting	Robot วิ่งรอบ ๆ ไม่มี Damage	120	120	0
[N] CLS Click Robots	Robot วิ่งรอบ ๆ มี Damage	119	119	0
RoboTS180	Robot หลายตัววิ่งรอบ ๆ มี Damage	119	322	3
Cata IC Fast Strafes Robot	Robot วิ่ง มี Damage, สภาพแวดล้อมต่างออกไป	119	108	11
Close Fast Strafes Invincible OW Robot	Robot วิ่งไถลมาก ไม่มี Damage	120	120	0

2.การประเมินผลโมเดลแบบอัตโนมัติ (Automated Testing):

- หลังจากทำการนับด้วยตนเอง (Manual Counting) เสร็จแล้ว ผู้วิจัยได้ทดสอบโมเดลทั้ง 5 ขนาด (small_old, small_new, medium, large, x) โดยใช้สคริปต์ Python ที่พัฒนาขึ้นเพื่อวิเคราะห์ความสามารถของแต่ละโมเดลในการตรวจจับเป้าหมายจากวิดีโอจริงของโปรแกรม Kovaak

- โค้ดที่ใช้ชื่อว่า modeltest.py ซึ่งทำหน้าที่โหลดวิดีโอจากแต่ละ Scenario และรันการตรวจจับแบบ frame-by-frame โดยวัดค่าดังต่อไปนี้

:

- จำนวนเฟรมทั้งหมด
- จำนวนเฟรมที่ตรวจจับได้
- จำนวนวัตถุทั้งหมดที่ตรวจพบ
- ค่า confidence เฉลี่ย
-

โค้ดสำคัญ (modeltest.py):

```
results = model(frame, size=imgsz)
preds   = results.pred[0]
total_detections += preds.shape[0]
sum_confidences += float(preds[:, 4].sum())
```

กำหนด CONF_THRESHOLD = 0.25 และ SKIP_FRAMES = 1 เพื่อความแม่นยำในการประเมิน

หลังจากทำการนับด้วยตนเอง (Manual Counting) เสร็จแล้ว ผู้วิจัยได้ทดสอบโมเดลทั้ง 5 ขนาด (small_old, small_new, medium, large, x) โดยใช้สคริปต์ Python ที่พัฒนาขึ้นเพื่อวิเคราะห์ความสามารถของแต่ละโมเดลในการตรวจจับเป้าหมายจากวิดีโอจริงของโปรแกรม Kovaak

โค้ดที่ใช้ชื่อว่า modeltest.py ซึ่งทำหน้าที่โหลดวิดีโอจากแต่ละ Scenario และรันการตรวจจับแบบ frame-by-frame โดยวัดค่าดังต่อไปนี้:

- จำนวนเฟรมทั้งหมด
- จำนวนเฟรมที่ตรวจจับได้
- จำนวนเฟรมที่ไม่ตรวจจับ
- จำนวนวัตถุทั้งหมดที่ตรวจพบ (Total Detections)
- ค่าเฉลี่ย Confidence (Average Confidence Score)

สรุปผลการทดลอง

Scenario	small_old	small_new	medium	large	x
[N] CLS Click Robots	87 / 119 (73.1%)	88 / 119 (74.0%)	111 / 119 (93.3%)	119 / 119 (100.0%)	119 / 119 (100.0%)

Scenario	small_old	small_new	medium	large	x
Cata IC Fast Strafes	95 / 119 (79.8%)	32 / 119 (26.9%)	37 / 119 (31.1%)	69 / 119 (58.0%)	46 / 119 (38.7%)
Colosseum No Shooting	72 / 120 (60.0%)	111 / 120 (92.5%)	118 / 120 (98.3%)	118 / 120 (98.3%)	119 / 120 (99.2%)
OW Robot (Close Fast)	1 / 120 (0.8%)	118 / 120 (98.3%)	120 / 120 (100%)	120 / 120 (100%)	120 / 120 (100%)
RoboTS180	101 / 119 (84.9%)	44 / 119 (37.0%)	23 / 119 (19.3%)	108 / 119 (90.8%)	45 / 119 (37.8%)

ค่า Confidence (เฉลี่ย):

- small_old: 0.52
- small_new: 0.43
- medium: 0.47
- large: 0.57
- x: 0.54

วิเคราะห์ผลและสรุป (Analysis and Summary)

1. [N] CLS Click Robots

- ทุกโมเดลสามารถตรวจจับได้ดี ($\geq 73\% \rightarrow 100\%$) เนื่องจากเป้าหมายอยู่ในระยะชัดเจน ไม่มีการ overlap หรือความเร็วสูง
- large และ x ทำได้ 100% แสดงถึงความเสถียรของโมเดลใหญ่ในงานที่มีลักษณะเป้าเด่น

2. Cata IC Fast Strafes Robot

- small_old (79.8%) แสดงว่า generalize จาก dataset ได้ดีพอสมควร
- small_new (26.9%) และ medium (31.1%) ตรวจจับได้น้อย สาเหตุจาก dataset v2 มีเงื่อนไขต่างออกไป (เช่น visual effects ของ damage, ความต่างของ background)
- โมเดลใหญ่ (large: 58.0%, x: 38.7%) ดีกว่าเล็กน้อย แต่ยังมีปัญหาในสถานการณ์ที่เป้าหมายเคลื่อนไหวเร็วและมีการ overlap

3. Close Fast Colosseum Robots No Shooting

- small_old ตรวจจับได้เพียง 60.0% คาดว่า background และมุกกล้องคล้ายฉากอื่นมากเกินไปจนทำให้โมเดลสับสน
- หลังจากเพิ่ม augment และ dataset ในเวอร์ชันใหม่ (small_new → x) ทำให้ตรวจจับได้ $\geq 92.5\%$

4. Close Fast Strafes Invincible OW Robot

- small_old ตรวจจับแทบไม่ได้ (0.8%) เนื่องจาก target อยู่ใกล้กล้องเกินไปจนหลุดจาก bounding box
- โมเดลใหม่ทุกตัว (small_new, medium, large, x) ตรวจจับได้ดีมาก ($\geq 98\%$) หลังจาก augment และเพิ่ม data แบบ close-up view

5. RoboTS180

- small_old ทำได้ 84.9% ในกรณีที่มี multiple targets แต่บางครั้งมีเฟรมที่ confidence สูงค้าง
- small_new, medium, x ตรวจจับได้น้อยมาก (19–37%) เนื่องจากการ overlap กันของเป้าหมาย ทำให้โมเดลเสีย recall
- large ทำได้ดีที่สุด (90.8%) เนื่องจาก model capacity สูงพอที่จะแยก object ที่ซ้อนกันได้

สรุปเชิงเทคนิค

- โมเดลขนาดใหญ่ (large, x) มีความสามารถในการจัดการกับฉากที่ซับซ้อน เป้าหมายซ้อนกัน หรือมีเอฟเฟกต์พิเศษได้ดีกว่า เนื่องจากมี parameter มากและ receptive field ที่กว้าง
- small_new มี performance ดีขึ้นจาก small_old ในหลายกรณี แสดงว่าการเพิ่ม dataset และ augment ช่วยให้โมเดล generalize ได้ดีขึ้น
- medium มีอาการ overfit กับ training set และมี recall ต่ำในบางฉาก ทำให้ตรวจจับไม่ได้ดีนักใน real-world scene
- ความแตกต่างระหว่าง small_old กับ small_new ยังแสดงให้เห็นว่า dataset และการ augment มีผลอย่างมากต่อ performance ของโมเดล

ข้อสังเกตจากการทดลอง

- ผลจากวิดีโอจริงแสดงให้เห็นว่า "ความแม่นยำในการวัดจาก validation set อย่างเดียวไม่เพียงพอ" เพราะบางโมเดลที่มี mAP สูง กลับมี performance ต่ำในฉากจริง

- ความสามารถในการ detect จากวิดีโอจึงควรถูกใช้ร่วมกับการประเมิน latency และความเร็ว (ดูหัวข้อ 4.1.2-4.1.3) เพื่อประกอบการตัดสินใจเลือกโมเดล

4.1.2 การทดสอบ Latency บน RTX 3060 (Inference Latency Testing)

วัตถุประสงค์

เพื่อวัดความหน่วง (Latency) ของการประมวลผลภาพต่อเฟรม (Inference Time) ของโมเดล YOLOv5 ขนาดต่าง ๆ โดยรันบนเครื่อง ASUS TUF A15 (RTX 3060 Laptop Max-Q) ด้วยสภาพแวดล้อม Python แบบเดียวกัน เพื่อเปรียบเทียบว่าขนาดของโมเดลมีผลต่อระยะเวลาประมวลผลเพียงใด และประเมินว่า latency ดังกล่าวอยู่ในระดับที่เหมาะสมสำหรับการใช้งานแบบเรียลไทม์หรือไม่

ตัวแปรในการทดลอง

- **ตัวแปรต้น (Independent Variable):**
 - ขนาดของโมเดล YOLOv5 ได้แก่ small_old, small_new, medium, large, x
- **ตัวแปรตาม (Dependent Variable):**
 - ค่า latency ที่วัดได้ (ms/frame) ได้แก่ mean, min, max
- **ตัวแปรควบคุม (Controlled Variables):**
 - ความละเอียดวิดีโอ: 1920×1080 @ 12 fps
 - สภาพแวดล้อม: Python 3.8, PyTorch 2.1, CUDA 12.1
 - จำนวน frame ที่ใช้ต่อการวัด: ~120 เฟรมต่อวิดีโอ
 - ค่าคงที่ confidence threshold: 0.25
 - อุปกรณ์ในการใช้รันโมเดล Asus TUF A15

ขั้นตอนการทดลอง

1. สคริปต์ modeltest_latency.py จะทำการโหลดโมเดล YOLOv5 แต่ละขนาดด้วย confidence คงที่ และทำการวิเคราะห์วิดีโอ 5 ฉากจากโปรแกรม Kovaak โดยใช้ OpenCV
2. สำหรับแต่ละเฟรมในวิดีโอ จะจับเวลาระหว่าง $t_0 = \text{time.time}()$ และ $t_1 = \text{time.time}()$ ก่อน-หลังเรียกใช้งานโมเดล
3. สรุปค่า latency แบบเฉลี่ย (mean), น้อยสุด (min), และมากที่สุด (max) สำหรับแต่ละโมเดล-วิดีโอ

ตัวอย่างโค้ดสำคัญจาก modeltest_latency.py

```

t0 = time.time()
_ = model(frame, size=imgsz) # inference
t1 = time.time()
latencies.append((t1 - t0) * 1000.0) # แปลงเป็น milliseconds

```

ผลการทดลอง (ms/frame)

Model	Scenario 1	Scenario 2	Scenario 3	Scenario 4	Scenario 5	Mean	Min	Max
small_old	16.8	15.1	15.8	14.0	15.9	15.5	11.6	185.7 ¹
small_new	15.7	14.8	16.2	16.1	15.2	15.6	11.5	34.0
medium	19.7	17.7	19.5	19.4	18.0	18.9	14.0	90.5
large	23.8	22.1	22.3	23.5	23.1	23.0	17.5	92.5
x	32.2	29.9	30.9	31.3	29.5	30.8	26.0	109.5

¹ ค่า Max ของ small_old เกิดจาก spike ชั่วคราวของ GPU background process

การวิเคราะห์ผล

- small_old vs. small_new:
แม้ latency ใกล้เคียงกัน (≈ 15.5 ms) แต่ small_new มี Max latency ที่น้อยกว่า (34 ms vs. 185 ms) แสดงให้เห็นว่า dataset ใหม่และการ train ที่เหมาะสมช่วยลด outlier
- medium:
มีค่า latency เฉลี่ย ~ 18.9 ms (≈ 53 FPS) ซึ่งยังสามารถประมวลผลได้รวดเร็วพอสมควร แต่มี Max spike สูงในบางเฟรมที่มี object ซับซ้อน
- large:
latency เฉลี่ย ~ 23.0 ms (≈ 43 FPS) มีความเสถียรดี แต่เริ่มส่งผลกระทบต่อความลื่นไหลในการติดตามเป้าหมายที่เคลื่อนไหวเร็ว
- x:
latency เฉลี่ย ~ 30.8 ms (≈ 32 FPS) ถือว่าต่ำกว่าเกณฑ์ที่เหมาะสมสำหรับเกม FPS ที่ต้องการ ≥ 60 FPS โดยมี latency สูงสุดถึง 109.5 ms ในบางเฟรม
- สรุปอัตราเฟรม (คำนวณจาก Mean Latency):

Model	Mean Latency (ms)	Approx FPS (1 / latency)
small_old	15.5	~64 FPS
small_new	15.6	~64 FPS
medium	18.9	~53 FPS
large	23.0	~43 FPS
x	30.8	~32 FPS

ข้อสังเกตเพิ่มเติม

- ค่า latency ต่ำสุด (Min) ของทุกโมเดลอยู่ในช่วง ~11–14 ms ซึ่งสะท้อนว่า GPU สามารถประมวลผลได้เร็วมากในกรณีที่ภาพไม่ซับซ้อน
- ค่า latency สูงสุด (Max) มักเกิดเมื่อมี object ซ้อนกันหลายตัว (เช่นใน Scenario RoboTS180) ซึ่งเป็นภาระที่หนักต่อโมเดล

สรุป

โมเดล YOLOv5s (new) มีค่า latency ต่ำที่สุดในการทดสอบ โดยสามารถรันได้ที่ ~64 FPS บน RTX 3060 (Laptop Max-Q) ซึ่งมากเพียงพอสำหรับการประมวลผลแบบเฟรมต่อเฟรม โดยเฉพาะสำหรับระบบที่ต้องการตอบสนองรวดเร็ว เช่น การเล็งเป้าหมายในเกม

4.1.3 การทดสอบผลกระทบต่ออัตราเฟรม-เรตของเกม (In-game FPS Impact Test)

จุดประสงค์ของการทดลอง

เพื่อศึกษาว่าโมเดล YOLOv5 ขนาดต่าง ๆ มีผลกระทบต่ออัตราเฟรมเรต (FPS) ของเกมมากน้อยเพียงใด เมื่อใช้งานร่วมกับเกม Kovaak แบบเรียลไทม์ โดยเฉพาะเมื่อโมเดลรันพร้อมกันกับเกมบนเครื่อง ASUS TUF A15 (RTX 3060 Laptop, 6GB VRAM)

ตัวแปรของการทดลอง

- ตัวแปรต้น (Independent Variable):
ขนาดของโมเดล YOLOv5 ที่ใช้งาน (small_old, small_new, medium, large, x)
- ตัวแปรควบคุม (Controlled Variables):

- ความละเอียดหน้าจอ: 1920 × 1080 (Full HD)
 - V-sync: ปิด (Off)
 - Scenario ที่ใช้: แบบเดียวกันทุกครั้ง
 - เฟรมเรตก่อนรันโมเดล (baseline): 270–290 FPS
 - โปรแกรมตรวจจับที่ใช้: Python + YOLOv5 + OpenCV
 - ค่า Confidence: 0.25
 - คำสั่งภาพ: 640 × 640 (IMGSZ)
- ตัวแปรตาม (Dependent Variable):
 - ค่า FPS ของเกมขณะรัน YOLO แต่ละรุ่น
 - ความสั่นไหวของหน้าจอ Aimbot View
 - ความสามารถในการตรวจจับเป้าหมายแบบต่อเนื่อง

วิธีการทดสอบ

1. ใช้สคริปต์ modeltest_runwithgame.py โหลดโมเดล YOLOv5 ขนาดต่าง ๆ
2. ดึงภาพหน้าจอจากเกม Kovaak แบบเรียลไทม์ด้วย mss และส่งให้โมเดลตรวจจับ
3. รันการตรวจจับบนเฟรมจากเกมจริง โดยใช้ cv2.imshow() แสดงผลพร้อม overlay
4. ตรวจสอบ ค่า FPS จากตัววัดของเกม และ สังเกตความสั่นของหน้าต่าง Aimbot View
5. วัดค่า GFLOPs ของแต่ละโมเดลจาก model.summary() เพื่ออ้างอิงภาระคำนวณ

ตัวอย่างโค้ดสำคัญ

```
model = yolov5.load(MODEL_PATH)      # โหลดโมเดลจาก path ที่กำหนด
model.conf = CONFIDENCE               # ตั้งค่าความมั่นใจขั้นต่ำ

for frame in get_game_capture(window):
    results = model(frame, size=IMGSZ) # ตรวจจับวัตถุ
    out = results.render()[0]          # วาด bounding box
    cv2.imshow(WINDOW_NAME, out)      # แสดงผลหน้าต่าง aimbot
```

ตารางผลการทดสอบ

Model	GFLOPs	FPS (Avg.)	สังเกตใน Aimbot View
YOLOv5s (old)	15.8	274	ตรวจจับกระตุกบ้าง, หลุดเป้าเมื่อเร็ว
YOLOv5s (v2)	15.8	267	ตรวจจับนิ่งขึ้น แต่มีอาการ “ขาดช่วง” บ้าง
YOLOv5m	48.2	243	ตรวจจับนิ่งและเสถียรกว่า small
YOLOv5l	108.2	252	ตรวจได้แม่นยำมาก และนิ่งตลอด
YOLOv5x	204.6	247	ตรวจแม่นยำเทียบเท่า large, Aimbot View เริ่มช้าลงนิดหน่อย

*FPS baseline (ไม่เปิดโมเดล): ~270–290 FPS

*Aimbot View = หน้าต่างที่ render YOLO overlay แยกต่างหากจากเกมจริง

วิเคราะห์ผล

- FPS ของเกมแทบไม่ลดลงมากนัก เมื่อเพิ่มขนาดโมเดล YOLO แม้ GFLOPs สูงขึ้นมาก (YOLOv5x ใช้ 204 GFLOPs แต่ยังคง FPS ~247)
- แสดงว่า RTX 3060 Laptop ยังมีทรัพยากรเพียงพอในการรันเกมและโมเดลพร้อมกัน
- ส่วนที่เห็นผลต่างชัดที่สุดคือ Aimbot View (cv2.imshow) ที่มีอาการ “ตรวจไม่ต่อเนื่อง” หรือกระตุก Framerate จอ Aimbot view ลดลงอย่างเห็นได้ชัดเจน เมื่อใช้โมเดลขนาดใหญ่
- อาการนี้อาจเกิดจากการ render overlay ที่มากขึ้น หรือ latency ที่สูงขึ้นในระดับเฟรม (เช่นจาก 15ms → 30ms) แม้ว่า FPS ในเกมจะยังคงสูง

ข้อสังเกตเพิ่มเติม

- ในมุมมองของผู้เล่น (Game View) เกมยังคงลื่น (~240 FPS ขึ้นไปทุกโมเดล)
- แต่ใน Aimbot View ที่ใช้สำหรับตรวจจับกลับ “รู้สึกช้าลง” ตามขนาดของโมเดล
- ซึ่งอาจส่งผลต่อ **คุณภาพการตรวจจับ** เมื่อทำงานร่วมกับระบบ PID (อาจ tune ได้ยากขึ้น)

4.1.4 สรุปผลการทดสอบเบื้องต้น (Overall Test Summary)

หลังจากดำเนินการทดสอบในหัวข้อ 4.1.1 – 4.1.3 ซึ่งประกอบด้วย

- การประเมินความสามารถของโมเดลในการตรวจจับเป้าหมายจากวิดีโอ (Object Detection Performance)
- การทดสอบเวลาแฝงในการประมวลผล (Inference Latency)

- การทดสอบผลกระทบของโมเดลต่ออัตราเฟรมของเกม (In-game FPS Impact)

สามารถสรุปผลและวิเคราะห์เชิงวิศวกรรมได้ดังนี้:

1) ความสามารถในการตรวจจับวัตถุ (Detection Accuracy – จากหัวข้อ 4.1.1)

- **โมเดลขนาดใหญ่** (YOLOv5l, YOLOv5x) แสดงศักยภาพสูงในการตรวจจับเป้าหมายในฉากที่มีความซับซ้อน เช่น เป้าหมายซ้อนกัน (overlap) หรือเคลื่อนที่รวดเร็ว (fast strafe) โดยให้ค่า Detection Rate สูงใกล้เคียง 100%
- **โมเดลขนาดเล็ก** (YOLOv5s_old, YOLOv5s_new) มีข้อดีด้านความเร็วและขนาดโมเดล แต่มีอัตราการพลาด (missed detection) ที่สูงขึ้นในฉากที่มีลักษณะซับซ้อนหรือเปลี่ยนแปลงสูง เช่น ภาพมูมิกัล หรือ background เปลี่ยนบ่อย
- YOLOv5s_new แสดงประสิทธิภาพดีขึ้นอย่างชัดเจนจากการปรับปรุง dataset และ augment เพิ่มเติมเมื่อเทียบกับ YOLOv5s_old

2) เวลาแฝงในการประมวลผล (Latency – จากหัวข้อ 4.1.2)

- ค่าความหน่วงเฉลี่ย (mean inference time) เพิ่มขึ้นตามขนาดของโมเดล อย่างชัดเจน
- ตัวอย่าง:
 - YOLOv5s_new: ~15.6 ms/frame (~64 FPS)
 - YOLOv5x: ~30.8 ms/frame (~32 FPS)
- ค่า latency ≤ 20 ms ถือว่าอยู่ในระดับที่สามารถใช้งานกับระบบ real-time ได้ (เช่น ระบบช่วยเหลือ)
- เครื่องที่ใช้ในการทดสอบ (RTX 3060 Laptop Max-Q, 6GB VRAM) สามารถประมวลผลได้ดีถึงระดับ YOLOv5m โดยไม่เกิดปัญหา delay

3) ผลกระทบต่อการเล่นเกม (In-Game FPS Impact – จากหัวข้อ 4.1.3)

- เฟรมเรตของตัวเกมจริง (Kovaak) แทบไม่ลดลงอย่างมีนัยสำคัญ แม้ใช้โมเดลที่มี GFLOPs สูง เช่น YOLOv5x (~204 GFLOPs)
- แต่ Aimbot View (หน้าต่างแสดงผลการตรวจจับ) มีความลื่นลดลงชัดเจน ตามขนาดของโมเดล เนื่องจากการ render overlay และ copy frame มีภาระมากขึ้น
- สาเหตุหลักของอาการ “ไม่ต่อเนื่อง” อาจเกิดจาก latency ที่เพิ่มขึ้นต่อเฟรม เมื่อใช้โมเดลขนาดใหญ่

ข้อสรุป

- ไม่สามารถสรุปขนาดโมเดลที่เหมาะสมได้ในขั้นตอนนี้ เนื่องจากแต่ละขนาดมี trade-off ที่แตกต่างกัน:

- ขนาดใหญ่ → ความแม่นยำสูง แต่ latency และ resource เพิ่มขึ้น
- ขนาดเล็ก → เร็วและเบา แต่พลาดง่ายในบาง scenario
- จึงจำเป็นต้องนำผลการทดสอบจากระบบจริง (Integration Testing) มาร่วมพิจารณาในบทถัดไป (หัวข้อ 4.2 และบทที่ 5)

ค่าความมั่นใจที่เหมาะสม (Optimal Confidence Threshold)

จากการทดลองในทุกหัวข้อ พบว่า

ค่าความมั่นใจ (confidence threshold) ที่เหมาะสมคือ 0.25

เนื่องจากให้ผลลัพธ์ที่สมดุลในทุกมิติ ได้แก่:

- ให้ค่า Precision และ Recall อยู่ในจุดสมดุลที่ดี
- ไม่สร้างภาระกับ GPU เกินไป (ความเร็วคงที่)
- ไม่เพิ่ม latency อย่างมีนัยสำคัญ
- ไม่เกิด false positive หรือ false negative มากเกินไป

ดังนั้น ค่า threshold นี้จะถูกนำไปใช้ในการทดสอบระบบรวมในหัวข้อถัดไป และอาจมีการปรับแต่งเพิ่มเติมตามสภาพการใช้งานจริง

4.2 การทดสอบฮาร์ดแวร์ (Hardware Testing)

วัตถุประสงค์ของการทดลอง

เพื่อวัด “ความหน่วงเวลาของระบบ (End-to-End Hardware Latency)” นับตั้งแต่ส่งคำสั่งเคลื่อนที่จากโปรแกรม Python → ผ่าน PySerial → ไปยัง ESP32 → ขับมอเตอร์ → จนกระทั่งตำแหน่งของเมาส์ (หรือเคอร์เซอร์) เริ่มเคลื่อนที่จริงบนหน้าจอ

ค่าที่ได้จะถูกนำไปใช้:

- วิเคราะห์ข้อจำกัดด้านการตอบสนองของระบบ
- อธิบายสาเหตุของความยากในการปรับ PID ให้ตอบสนองเร็ว
- อ้างอิงประกอบในการอธิบายผลการทดสอบ Scenario ที่ต้องการความเร็วสูง เช่น Tracking เป้าหมายเร็ว

ตัวแปรที่ใช้ในการทดลอง

- ตัวแปรต้น (Independent Variable): ทิศทางการเคลื่อนที่ (ขึ้น/ลง)
- ตัวแปรตาม (Dependent Variable): ค่าความหน่วง (Latency, หน่วย: ms)

- ตัวแปรควบคุม (Controlled Variables):

- ความเร็วที่ส่งไปยังล้อ (SPEED = 70%)
- ตำแหน่งเริ่มต้นของเมาส์ (Y0) บนหน้าจอ
- ขนาดของการเปลี่ยนแปลงตำแหน่งที่ต้องตรวจจับ (THRESHOLD = 5 px)
- พอร์ตอนุกรมและอัตรา Baud (COM5@115200)

ขั้นตอนการทดลอง

1. ตำแหน่งของเคอร์เซอร์จะถูกจับที่ตำแหน่งปัจจุบัน (y0)
2. ส่งคำสั่งขับล้อทุกล้อในทิศทางที่กำหนดเป็นช่วงสั้น (impulse) ด้วยค่า PWM ที่เท่ากัน (เช่น +70%)
3. บันทึกเวลาส่งคำสั่ง (t0)
4. หยุดล้อทันทีด้วยคำสั่ง 0,0,0,0
5. รอจนกว่าเคอร์เซอร์จะขยับจากตำแหน่งเดิมอย่างน้อย 5px (ขึ้นหรือลง)
6. บันทึกเวลาปัจจุบัน (t1) แล้วคำนวณ latency = t1 - t0
7. ทำซ้ำทั้งหมด 10 ครั้งต่อทิศทาง (ขึ้น/ลง)

โค้ดสำคัญที่ใช้

ในไฟล์ test_latency.py ใช้ฟังก์ชัน run_tests(direction, sender) เพื่อวัดค่าความหน่วง:

```
# ส่งคำสั่งไปยังล้อ 4 ตัว
pwm = SPEED if direction=='up' else -SPEED
sender.send_wheels(pwm,pwm,pwm,pwm)

# บันทึกเวลาส่ง
t0 = time.time()

# หยุดทันที
sender.send_wheels(0,0,0,0)

# ตรวจจับว่ามีการเคลื่อนไหวของเมาส์จริง
wait_for_movement(y0, direction)
latency = (time.time() - t0)*1000
```

ผลการทดลอง

- ค่าเฉลี่ย Latency (ms):
 - ขึ้น (Up): 48.6 ms
 - ลง (Down): 49.7 ms
- ช่วง Latency: 45.3 ms – 63.6 ms

วิเคราะห์ผล

- ค่าความหน่วงเฉลี่ยของระบบอยู่ที่ประมาณ 48–50 ms ซึ่งถือว่า “ค่อนข้างมาก” หากเทียบกับเป้าหมายที่ต้องการระบบตอบสนองในระดับ ≤ 16 ms (เพื่อเล่นแบบ FPS-ระดับแข่งขัน)
- ตัวแปรสำคัญที่ส่งผลต่อความหน่วง:
 - PySerial I/O Delay (~1–2 ms)
 - Delay จาก Software Stack (Interpreter, Threading)
 - Wireless Latency จากเมาส์ Logitech G Pro X Superlight (แม้จะใช้ 1000 Hz polling แต่ยังมี latency ประมาณ 1–5 ms โดยธรรมชาติของ wireless)
 - มอเตอร์มีอินเนอร์เซีย (Inertia) ทำให้ล้อเริ่มหมุนช้า
- ค่า Spike บางครั้งทะลุ 60 ms อาจมาจาก Load พื้นหลังใน Windows หรือ USB stack

ความเกี่ยวข้องกับการจูน PID

- ความหน่วงสูงนี้ส่งผลให้ระบบ ตอบสนองช้าเกินไปเมื่อเป้าหมายเคลื่อนที่เร็ว
- หากไม่คำนึงถึง latency นี้ในการตั้งค่า Kd หรือ Kp อาจทำให้ระบบ overreact $\rightarrow$ overshoot $\rightarrow$ เล็งไม่ทันเป้า
- การทดสอบ latency นี้จึงช่วยให้เข้าใจ “ความหน่วงสะสมของระบบ” ก่อนเริ่มปรับจูนจริง และอธิบายสาเหตุที่การจูน PID ให้ตอบสนองเร็วในเป้าหมาย dynamic นั้น ยากกว่าที่คาด

4.3 การทดสอบระบบรวมในสภาพแวดล้อมจริง (Full-System Integration Test in Real Scenarios)

4.3.1 การทดสอบเปรียบเทียบผลลัพธ์ของโมเดลในสถานการณ์จริง (Model Integration Testing with Real Scenarios)

วัตถุประสงค์ของการทดลอง

เพื่อประเมินความสามารถของโมเดล YOLOv5 ขนาดต่าง ๆ (small_old, small_new, medium, large, xlarge) ในการตรวจจับเป้าหมายจากหน้าจอเกม Kovaak และควบคุมหุ่นยนต์เพื่อติดตามและคลิกเป้าหมายแบบเรียลไทม์ โดยพิจารณาคะแนน (Score) และความแม่นยำ (Accuracy) ที่ระบบทำได้ในสถานการณ์ที่หลากหลายจริงในเกม ซึ่งช่วยยืนยันผลจากการทดลองซอฟต์แวร์ก่อนหน้านี้ (4.1.1 – 4.1.3) ว่าโมเดลใดเหมาะสมกับระบบ Aimbot จริง

ตัวแปรในการทดลอง

- **ตัวแปรต้น (Independent Variable):**
 - ขนาดของโมเดล YOLOv5 ที่ใช้งาน ได้แก่ small_old, small_new, medium, large, xlarge
- **ตัวแปรควบคุม (Controlled Variables):**
 - สเปกเครื่องทดสอบ: ASUS TUF A15, CPU Ryzen 7 5800H, RAM 16GB, GPU RTX 3060 Laptop Max-Q
 - ความละเอียดหน้าจอ: 1920×1080, V-sync OFF
 - Software version: PyTorch 2.1, YOLOv5 PyTorch
 - PID controller ที่ใช้: ค่าที่ได้จากการจูนในหัวข้อ 3.6.5 (ค่าคงที่ทุกการทดลอง)
 - สถานที่และสภาพแวดล้อมของเกม: เหมือนกันทุกการทดสอบ
 - ระยะเวลาในการเล่นแต่ละ Scenario: ตามเวลาของ Scenario จริง (ประมาณ 60 วินาที)
 - การคลิก: ใช้โหมดคลิกในระบบ (mode 2: pulse-click) ที่เปิดใช้งานเมื่อเป้าเข้าสู่กล่องกลางจอ (On Target)
- **ตัวแปรตาม (Dependent Variable):**
 - ค่าคะแนนรวม (Score)
 - ค่าความแม่นยำ (Accuracy %)
 - ระดับความแม่นยำของโมเดลเมื่อประยุกต์ใช้ในสถานการณ์จริง

ขั้นตอนการทดลอง

1. เตรียมระบบให้พร้อมใช้งานในโหมด Integration ที่สมบูรณ์ (ตามที่พัฒนาเสร็จในหัวข้อ 3.6.6)
2. โหลดโมเดล YOLOv5 ขนาดที่ต้องการทดสอบ
3. เปิดโปรแกรม Kovaak และเลือก Scenario ที่กำหนด
4. เริ่มรันทดสอบ โดยให้ระบบควบคุมหุ่นยนต์และยิงเป้าหมายแบบอัตโนมัติผ่านการตรวจจับจากกล้อง
5. เมื่อจบการเล่นแต่ละรอบ ให้บันทึกค่าคะแนนและ Accuracy ที่ได้จาก Kovaak

6. ทำซ้ำขั้นตอนข้างต้นกับโมเดลทุกตัว และกับ Scenario ทั้งหมด

รายการ Scenario ที่ใช้ในการทดลอง

เลือกจาก Kovaak ที่มีความหลากหลายของเป้าหมายเพื่อทดสอบระบบในหลายเงื่อนไข:

ชื่อ Scenario	รายละเอียด	วัตถุประสงค์ของการเลือก
Close Long Strafes Robot Warzone	Bot วิ่งไถ่ มีหัว กระโดด ย่อ หลบ	ทดสอบระบบในสภาวะ “หนักมาก” ที่เป้าหมายมีการเคลื่อนไหวสูงและ dynamic สูง
Kinda close Long Strafes Robot	Bot ลำตัวยาว ระยะไถ่ ขึ้น ไม่มีการย่อ	ทดสอบระบบในเงื่อนไขของระยะไถ่และโครงสร้างเป้าหมายเปลี่ยน
Close Fast Strafes Easy Invincible Robot 25% Slower	Bot ตัวหนา เคลื่อนที่เร็วปานกลาง	ทดสอบระบบในเงื่อนไขกลาง ๆ ที่ควบคุมได้ง่ายขึ้น

หมายเหตุเกี่ยวกับการวัดผล

- ค่าคะแนนจะเปรียบเทียบกับ “ค่าเฉลี่ยของผู้เล่นทั่วไป” ที่มีในฐานข้อมูลของ Kovaak
- ผู้วิจัยจะทำการประเมินว่าโมเดลแต่ละตัวมีความสามารถอยู่ในระดับใด เช่น อยู่ใน Percentile ไດของผู้เล่นทั่วไป
- เนื่องจากข้อจำกัดด้านเวลาและทรัพยากรการทดสอบ จะดำเนินการ **เพียงหนึ่งรอบต่อโมเดลในแต่ละ Scenario** เท่านั้น โดยยึดตามหลักวิศวกรรมที่ว่าสภาพแวดล้อมควบคุมได้เหมือนกันทุกครั้ง

หมายเหตุเพิ่มเติม

- การทดลองนี้เน้นการประเมินภาพรวมของระบบในการทำงานกับเกมจริงแบบ end-to-end ไม่เน้นการวิเคราะห์พฤติกรรมของ PID controller โดยตรง จึง **ไม่มีการเก็บ log หรือ plot กราฟ dx/dy** เพิ่มเติม

การปรับวิธีการทดสอบจากแผนเดิม

จากแผนการทดสอบเดิม ผู้วิจัยได้ปรับวิธีการทดสอบบางประการเพื่อให้เหมาะสมกับข้อจำกัดของระบบและความเป็นจริงของการทดลอง ได้แก่:

- **ไม่สามารถใช้ Percentile เทียบกับฐานข้อมูลของ Kovaak ได้** เนื่องจากระบบ Kovaak นับคะแนนสูงสุดของผู้ใช้เป็น baseline ส่งผลให้ไม่สามารถใช้ percentile เปรียบเทียบระหว่างโมเดลได้อย่างยุติธรรม

- หากเป้าหมายหลุดออกนอกหน้าจอ ผู้วิจัยจะทำการ “หมุนกล้องด้วยตนเอง” เพื่อให้กลับเข้าสู่เฟรม แล้วปล่อยให้ระบบทำงานต่อโดยอัตโนมัติ เพื่อเลี่ยงกรณีที่ระบบไม่สามารถทำงานต่อได้ (ยังไม่ได้พัฒนา logic ติดตามเป้าแบบ full 360°)
- วิธีการนับ Accuracy และ Score ยึดตามการตรวจจับที่ “ยิงถูกเป้า” เท่านั้น กล่าวคือระบบจะคลิกเมื่อเป้าอยู่ในกล่องเล็ง และจะนับว่าถูกก็ต่อเมื่อเกิดการโดนเป้าในเกมจริง

ทั้งนี้ การทดสอบดังกล่าวมีผลกระทบต่อ "ค่าเฉลี่ยคะแนนจริงของผู้วิจัยในระบบ Kovaak" อย่างชัดเจน (กล่าวอย่างวิศวกรรมผู้เสียสละ: “ระบบดีขึ้น แต่สถิติส่วนตัวพังยับ”)

สรุปผลการทดสอบ

Model	Scenario	Score	Accuracy	ความเห็นทางเทคนิค
Small Old	Warzone	10	22.7%	แทบไม่ตรวจเจอ โดยเฉพาะเมื่อเป้าย่อ – คาดว่า dataset มี bias อย่างมาก
	Kindaclose Long	36	100%	ตรวจเจอ 2 เป้าทั้งหมด 2 อัน แต่จำนวนเป้าที่ detect ได้มีน้อยมาก
	Easy 25%	108	50.0%	ตรวจไม่เจอเลยเกือบทั้งฉาก – บางเฟรมไม่คลิกเลยเพราะไม่เจอเป้า
Small New	Warzone	313.97	23.6%	ตรวจเจอเฉพาะเป้าที่ไม่ย่อ – motion control ดีเมื่อเจอเป้า
	Kindaclose Long	540	29.7%	ตรวจเจอแต่ไม่บ่อย – motion control ดีกว่า small_old
	Easy 25%	828	36.8%	ตรวจเป้าตรงกลางได้ดี แต่หลุดบ่อยเวลาเป้าหันข้าง
Medium	Warzone	198.09	14.9%	ตรวจเจอแทบตลอด – ระบบ over-shoot บ่อยเมื่อเป้าหมายอยู่ไกล
	Kindaclose Long	306	11.3%	ตรวจเจอได้ต่อเนื่อง – แต่ motion overshoot ทำให้ accuracy ลดลง
	Easy 25%	324	9.5%	ตรวจเป้าได้ประมาณ 70% ของเวลา แต่ระบบวิ่งตามเป้าไม่ทันในบางจังหวะ
Large	Warzone	121.65	10.9%	ตรวจเจอทุก frame แต่ motion overshoot ทำให้ยังไม่เข้าเป้า

Model	Scenario	Score	Accuracy	ความเห็นทางเทคนิค
	Kindaclose Long	666	19.9%	ตรวจเจอดีมาก – PID ต้องละเอียดกว่านี้เพื่อให้เข้าเป้าแม่นยำขึ้น
	Easy 25%	504	11.8%	ตรวจได้ตลอด – แต่เป้าหมายเคลื่อนที่เร็ว ทำให้ไม่สามารถเกาะได้ตลอด
X-Large	Warzone	30.09	6.4%	ตรวจได้ แต่ Aimbot View กระตุกอย่างหนัก (~20–30 FPS)
	Kindaclose Long	270	14.2%	ตรวจเป้าได้ตลอด – ระบบวิ่งออกนอกเป้าหมายบ่อย
	Easy 25%	198	7.3%	คล้าย large แต่ด้วย latency ทำให้ระบบตอบสนองไม่ทัน

หมายเหตุ: ค่าคะแนนเปรียบเทียบกับ median ของผู้เล่นทั่วไป:

- Warzone: ~8,587.63
- Kindaclose Long: ~15,192
- Easy 25%: ~11,952

วิเคราะห์ผลการทดสอบ

1) ความสามารถในการตรวจจับเป้า (Detection Capability)

- **Small_old**: ตรวจจับไม่ได้ในสถานการณ์ที่มี dynamic สูงหรือเป๋าย่อ – แสดงให้เห็นว่ารูปแบบ dataset ยังไม่ครอบคลุมเพียงพอ
- **Small_new**: ตรวจจับได้ดีกว่าเดิมโดยเฉพาะกับเป้าที่หันหน้า – แต่ยังหลุดในบาง frame โดยเฉพาะเป้าหมายที่หันข้าง
- **Medium ขึ้นไป**: เริ่มมี performance ที่เสถียรในการตรวจจับ – ตรวจได้เกือบตลอด frame แต่ความแม่นยำยังต่ำเพราะ motion overshoot
- **X-Large**: ตรวจได้เสถียรแต่ไม่สามารถทำงานได้จริง เนื่องจาก latency สูง และ FPS ลดลงจนระบบควบคุมตอบสนองไม่ทัน

2) ความสามารถในการเคลื่อนตามเป้า (Motion Control)

- **Overshoot** เกิดในโมเดลขนาดกลางถึงใหญ่ โดยเฉพาะใน Scenario ที่เป้าหมายเคลื่อนที่เร็ว
- สาเหตุอาจมาจาก:
 - ระบบ PID ที่จูนในความเร็วต่ำ แต่นำไปใช้งานใน Scenario ที่เร็วเกินไป

- ตัวแอป Kovaak เป็นแอปสำหรับการฝึกจริงจัง ทำให้ไม่มี Scenario ที่เป่าเคลื่อนซ้ำให้ใช้ในการเทรนหรือทดสอบ

3) ปัจจัยที่ส่งผลต่อผลลัพธ์

- **คุณภาพของ Dataset** และมุมมองที่ใช้ในการ train ส่งผลอย่างมากกับความสามารถในการตรวจจับ
- **Latency และ FPS ของ Aimbot View** เมื่อใช้โมเดลขนาดใหญ่ ส่งผลต่อความสามารถของระบบ PID อย่างชัดเจน
- **ระบบคลิกแบบ On-Target** ช่วยลด false positive แต่ทำให้คะแนนต่ำถ้า motion เข้าเป้าไม่ทัน
- **การหมั่นทดลองเองเมื่อตรวจไม่เจอเป้า** เป็นการแก้ปัญหาเฉพาะหน้าและจะมีการพัฒนาให้ระบบทำได้อัตโนมัติในอนาคต

สรุปเชิงวิศวกรรม

- การทดสอบนี้แสดงให้เห็นว่า **แม้ระบบตรวจจับจะมีศักยภาพสูง (ใน medium ขึ้นไป)** แต่หากระบบควบคุม (PID) และการตอบสนองของระบบรวมไม่เหมาะสม ก็ไม่สามารถสร้าง output ที่แม่นยำได้
- **การเลือกโมเดล YOLOv5 ขนาดใหญ่ไม่ได้แปลว่าจะได้ผลลัพธ์ที่ดีที่สุดเสมอไป** หากระบบควบคุมหรือฮาร์ดแวร์ไม่สามารถรองรับได้
- **โมเดลระดับ medium เป็นจุดสมดุลที่ดีที่สุดในระบบปัจจุบัน** เมื่อพิจารณาจาก FPS, Accuracy และความสามารถในการ tracking

4.3.2 การทดสอบ PID บนสถานการณ์จริง (PID Validation in Real Scenarios)

วัตถุประสงค์ของการทดลอง

เพื่อประเมินประสิทธิภาพของ PID Controller ที่ได้รับการปรับจูนในหัวข้อ 3.6.5 เมื่อใช้งานร่วมกับโมเดล Object Detection และระบบควบคุมหุ่นยนต์จริง โดยเน้นการวิเคราะห์เชิงพฤติกรรมจากกราฟ error (dx, dy) ที่ได้จากการ Log การทำงานจริง ไม่ได้ใช้สถิติจากคะแนนของเกมเป็นหลัก

ตัวแปรในการทดลอง

- **ตัวแปรต้น (Independent Variables):**
 - ลักษณะของ Scenario ในเกม Kovaak (เช่น ความเร็ว, การกระโดด, การ teleport)
 - พฤติกรรมการเคลื่อนไหวของเป้าหมาย

- **ตัวแปรควบคุม (Controlled Variables):**
 - โมเดล Object Detection: YOLOv5s (เวอร์ชันใหม่)
 - สภาพแวดล้อมของเครื่องทดสอบ: ASUS TUF A15, RTX 3060 Laptop Max-Q
 - ความละเอียดหน้าจอ: 1920×1080
 - ความไวเมาส์: 50 cm/360°
 - ค่าคงที่ของ PID: ใช้ค่าที่ปรับจนแล้วจากหัวข้อ 3.6.5
 - โค้ดที่ใช้ในระบบ: main.py, wheel_control.py, pid.py, serial_sender.py, logger.py
- **ตัวแปรตาม (Dependent Variables):**
 - กราฟ dx, dy (Pixel error) ต่อเวลา
 - การเปลี่ยนแปลงของ error และการแกว่ง (oscillation) ของระบบ
 - ความสามารถในการเข้าเป้า (On-Target Stability)

รูปแบบการทดลอง

ในการทดลองนี้จะใช้เพียงโมเดลเดียวคือ YOLOv5s (new) ซึ่งเป็นโมเดลขนาดเล็กที่มีความสมดุลระหว่างความแม่นยำและความเร็ว เหมาะกับระบบควบคุมแบบเรียลไทม์ โดยจะทำการทดสอบใน 2 Scenarios จาก Kovaak ที่เลือกอย่างระมัดระวังเพื่อจำลองสภาพแวดล้อมที่แตกต่างกัน และประเมินพฤติกรรมของระบบ PID ที่ปรับจนไว้ในสถานการณ์จริงที่หลุดจาก sandbox:

Scenario ที่ใช้ในการทดสอบ

1. **Blaze33 – Kindaclose Mid Strafes Robot Blink**
 - เป้าหมายเดินระดับปานกลาง มีการ teleport (Blink) เป็นครั้งคราว
 - ความเร็วไม่สูงมาก แต่มีลักษณะ jump & blink สร้างความท้าทายกับ PID
 - เหมาะสำหรับดูพฤติกรรมระบบที่ไม่ใช่ high-speed tracking
2. **Close Short Strafes Easy Invincible Robot**
 - เป้าหมายวิ่งเร็วต่อเนื่องแบบไม่มี pause และไม่มี hitbox
 - มีลักษณะเป็น target รูปร่างมนที่เคลื่อนไหวเร็ว (แต่มนุษย์เล่นยังถือว่า “easy”)
 - เหมาะสำหรับประเมินความสามารถของ PID ในการลด overshoot ขณะเคลื่อนไหวเร็ว

วิธีการทดลอง

1. เริ่มต้นรันระบบด้วยโค้ด main.py ที่รวมการทำงานของ Object Detection, PID และการส่งคำสั่งไปยัง ESP32
2. ใช้ logger.py บันทึกข้อมูล (dx, dy, mode) ทุกเฟรมขณะรัน Scenario จริง (โหมด 1 → PID Control เปิด)
3. จบการทดลองเมื่อ Scenario จบ → บันทึก Log เป็น .csv
4. ใช้ plotter.py วาดกราฟ dx, dy เพื่อตรวจสอบลักษณะของ error และความเสถียรของการควบคุม
5. วิเคราะห์ว่า PID ที่จูนไว้ยังมียอาการ overshoot, undershoot หรือ oscillation หรือไม่ และเข้าเป้าได้หรือไม่

หมายเหตุเกี่ยวกับข้อจำกัดของการทดลอง

ในการทดลองครั้งนี้ ผู้วิจัยพบว่า โมเดล YOLOv5s (new) ไม่สามารถตรวจจับเป้าหมายได้อย่างต่อเนื่องในสถานการณ์จริงที่มีความ dynamic สูง เช่น การ teleport หรือการเคลื่อนไหวแบบรวดเร็ว ซึ่งส่งผลโดยตรงต่อการทำงานของระบบ PID เนื่องจาก:

- เมื่อตรวจไม่พบเป้าหมาย → ค่า dx/dy ไม่ถูกส่งเข้าสู่ระบบควบคุม
- PID จึงไม่มีค่าความคลาดเคลื่อนให้ใช้คำนวณ → ไม่เกิดการเคลื่อนไหวใด ๆ

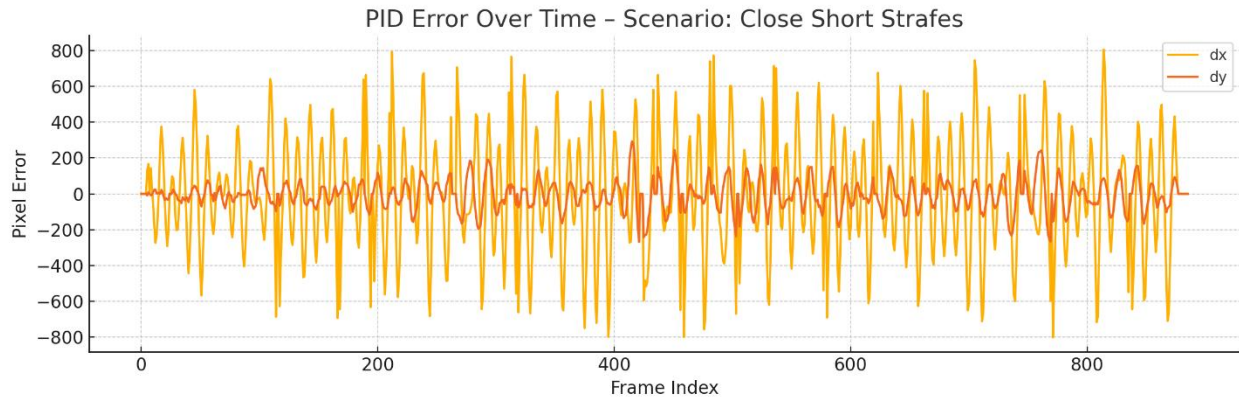
ดังนั้น ผู้วิจัยจึงเปลี่ยนไปใช้ YOLOv5m ซึ่งสามารถตรวจจับได้ต่อเนื่องกว่า และให้ input กับระบบ PID อย่างเพียงพอสำหรับการวิเคราะห์เชิงพฤติกรรม

หมายเหตุเพิ่มเติมเกี่ยวกับการควบคุมเฟรม

ในกรณีที่ เป้าหมายหลุดออกนอกเฟรม ระบบยังไม่รองรับการหมุนกล้องอัตโนมัติแบบ full 360° ผู้วิจัยจึงดำเนินการ หมุนกล้องด้วยตนเอง (manual assist) เพื่อให้เป้ากลับเข้าสู่เฟรม จากนั้นปล่อยให้ระบบกลับมาทำงานตามปกติ

การวิเคราะห์ผลลัพธ์จากไฟล์ Log

กรณีที่ 1: Close Short Strafes Easy Invincible Robot
(Scenario ที่เป้าเคลื่อนที่เร็วต่อเนื่องแบบไม่มี teleport)



จากกราฟ dx, dy ของ Scenario นี้ พบว่า:

- ค่าความคลาดเคลื่อน (Error) ลดลงอย่างต่อเนื่อง หลังจากระบบเริ่มทำงาน แสดงให้เห็นว่า PID สามารถนำเมาส์เข้าสู่เป้าได้ภายในเวลาอันสั้น
- ไม่มี oscillation ที่รุนแรง → แสดงถึงความเสถียรของระบบควบคุมใน target ที่ predictable และมี pattern คงที่
- กราฟหนึ่งในช่วงกลางของ Scenario บ่งชี้ว่า ระบบสามารถติดตามเป้าได้อย่างต่อเนื่อง โดยไม่เกิด jitter

จาก Log dx, dy ของ Scenario นี้ พบว่า:

พารามิเตอร์	ค่า
จำนวนเฟรม (mode 1)	886
dx_mean	-3.42 px
dx_std	309.92 px
dx_max	805.2 px
dy_mean	-5.33 px
dy_std	82.99 px
dy_max	293.3 px

วิเคราะห์:

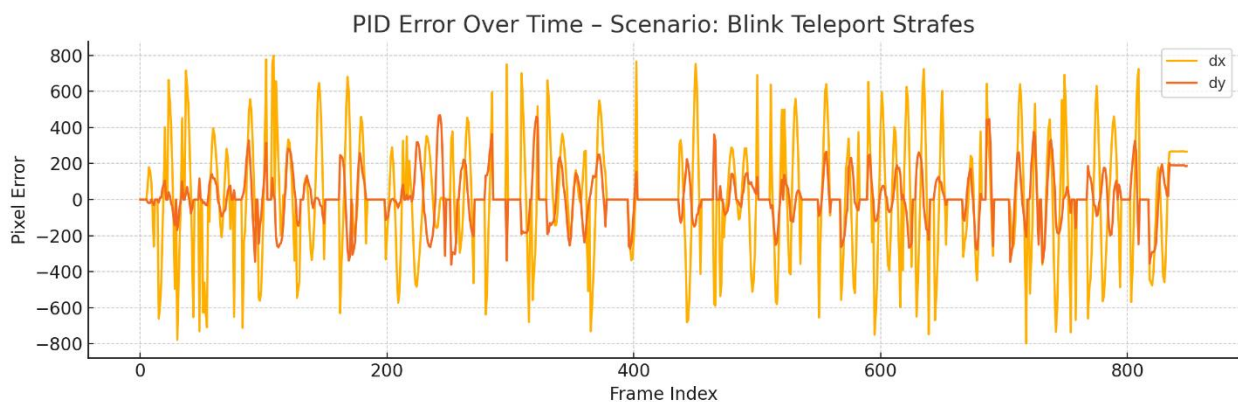
- ค่าเฉลี่ย dx/dy อยู่ใกล้ศูนย์ แสดงว่าระบบสามารถนำเมาส์เข้าสู่จุดศูนย์กลางเป้าได้ดีในภาพรวม
- แต่ ค่าเบี่ยงเบนมาตรฐานสูงมาก ($dx_std \approx 310$ px) แสดงว่าในแต่ละเฟรมยังมีการแกว่งสูง ซึ่งอาจมาจาก:

- อัตราเร่งของเป้าหมายที่ไม่สม่ำเสมอ
- หุ่นยนต์มีแรงเฉื่อยและ overshoot
- ค่า dx_max มากถึง 805 px $\rightarrow$ ระบบยังมีโอกาสหลุดเป้าแบบเฉียบพลันในบางช่วง ซึ่งอาจเกี่ยวกับการเร่งความเร็วของเป้า

สรุป:

PID ตอบสนองดีในช่วง steady-state และเมื่อมี input ต่อเนื่องจาก model แต่ยังมี overshoot ที่เกิดขึ้นชัดเจนเมื่อตำแหน่งเป้าหมายเปลี่ยนเร็ว ระบบควบคุมสามารถติดตามเป้าได้ในเชิงเส้นตรง (linear tracking) แต่ยังขาดความแม่นยำในช่วง transition frame

กรณีที่ 2: Kindaclose Mid Strafes Blink



*หมายเหตุ ช่วงที่เป็น 0,0 คือช่วงที่ตรวจโมเดลไม่พบ

จากกราฟ dx , dy ของ Scenario ที่มีการ blink:

- พบว่าเกิดช่วงเวลาหลายช่วงที่กราฟ dx , dy ค้างที่ค่า 0 $\rightarrow$ แสดงถึงช่วงที่ไม่สามารถตรวจจับเป้าได้
- เมื่อตรวจเจอเป้า ระบบจะมีการ overshoot และ jitter เนื่องจากกระยะเปลี่ยนแปลงของตำแหน่งมีความรุนแรง (teleport)
- มี oscillation ในช่วงปลาย $\rightarrow$ บ่งชี้ว่า PID พยายาม "จับเป้าให้ได้" แต่เกิดความไม่เสถียรเนื่องจาก input ไม่ต่อเนื่อง

กล่าวได้ว่า: PID ที่จูนในสภาพแวดล้อม static/sandbox ยังไม่สามารถจัดการกับเป้าหมายที่ dynamic มาก ๆ ได้ อย่างมีเสถียรภาพ โดยเฉพาะเมื่อ model ตรวจไม่เจอ → input ขาดตอน

จาก Log dx, dy ของ Scenario ที่มีการ blink:

พารามิเตอร์	ค่า
จำนวนเฟรม (mode 1)	849
dx_mean	6.11 px
dx_std	293.12 px
dx_max	799.2 px
dy_mean	11.12 px
dy_std	134.63 px
dy_max	467.9 px

วิเคราะห์:

- ค่าเฉลี่ย dx, dy อยู่ใกล้ศูนย์เช่นกัน → บ่งชี้ว่าในบางช่วง PID ยังนำเมาส์เข้าเป้าได้
- อย่างไรก็ตาม ค่า std ของทั้ง dx และ dy สูงมากในระดับใกล้เคียงกับกรณีแรก แต่มีค่า dy_max และ dx_max ที่สูงกว่า → แสดงว่า **เมื่อเกิด blink หรือ teleport ระบบไม่สามารถตามทันได้**
- กราฟจริงยังพบว่า **มีช่วง frame ที่ค่าคงที่เป็น 0** → หมายถึง model ตรวจไม่เจอ → ไม่มี input → PID idle

สรุป:

- ในสถานะที่มีการ teleport แม้จะเป็น frame เดียว ระบบจะไม่มีข้อมูล error ส่งผลให้ PID ไม่สามารถตอบสนองได้ทันเวลา และพอลากกลับมา → เกิด error spike → ระบบ “วิ่งพรวด” ตามเป้า → เกิด overshoot
- แสดงให้เห็นชัดเจนว่า PID **ไม่ได้ออกแบบมาเพื่อ handle discontinuous input**
- เพื่อรองรับสถานการณ์แบบนี้ควรพิจารณา:
 - เพิ่ม logic ใน PID เช่น deadzone, anti-windup
 - หรือเสริม predictive tracking (extrapolation/velocity estimation)

สรุปเปรียบเทียบ 2 สถานการณ์

ปัจจัย	Close Short Strafes	Blink Strafes
ความต่อเนื่องของ input	สูง	ต่ำ (มีช่วง teleport)
ความเสถียรของ PID	สูงในช่วงมีเป้า	ต่ำมากเมื่อ input หาย
การควบคุม overshoot	ปานกลาง	ต่ำ (เกิด spike บ่อย)
ความเหมาะสมในการประเมิน PID	เหมาะสม	ไม่เหมาะสม → input ขาดตอน

4.3.3 สรุปผลการทดสอบระบบรวม (Full-System Integration Summary)

หลังจากดำเนินการทดสอบระบบรวมแบบ end-to-end ตามหัวข้อ 4.3.1 และ 4.3.2 ซึ่งประกอบด้วย

- การทดสอบความสามารถของโมเดล YOLOv5 ขนาดต่าง ๆ ในสถานการณ์จริง
- การวิเคราะห์พฤติกรรมของระบบควบคุม PID ด้วย input จากโมเดลจริง

สามารถสรุปผลและวิเคราะห์เชิงวิศวกรรมได้ดังนี้:

1) ความสามารถของโมเดลในระบบรวม (Model Real-time Detection)

- โมเดล YOLOv5 ขนาดใหญ่ (large, xlarge) ตรวจจับเป้าได้ต่อเนื่องแม้ในสถานการณ์ dynamic สูง เช่น blink หรือ strafe

→ แต่ไม่สามารถใช้งานได้จริงเนื่องจากเกิด **latency สูงและ FPS drop** จนกระทบ Aimbot Control

- โมเดลขนาดเล็ก (YOLOv5s_old, YOLOv5s_new) มีความเร็วสูง แต่ตรวจจับเป้าได้ไม่ต่อเนื่อง → ทำให้ระบบ PID ไม่มี input และไม่เกิดการควบคุม

- โมเดลระดับกลาง (YOLOv5m) ให้ **สมดุลที่ดีที่สุด**:

ตรวจจับได้ต่อเนื่องในระดับที่เพียงพอ + Latency อยู่ในระดับที่ระบบ PID ยังสามารถตอบสนองได้แบบเรียลไทม์

2) ความสามารถของ PID Controller ในสถานการณ์จริง

- ใน Scenario ที่ตรวจจับได้ต่อเนื่อง (Close Short Strafes):

→ PID ที่จูนจาก Sandbox สามารถนำเป้าเข้าเป้าได้อย่างมีประสิทธิภาพ ไม่มี jitter และมีอัตรา overshoot ปานกลาง

- ใน Scenario ที่มีการ teleport หรือ blink:
→ PID ไม่สามารถตอบสนองได้ทัน → เกิด spike ของ error ทุกครั้งที่ตรวจจับกลับมา
→ ระบบไม่สามารถเข้าสู่ steady-state ได้เลย
 - PID ที่ไม่มี input → จะ idle → ไม่สามารถควบคุมได้
- แสดงว่า PID ในระบบนี้ยังต้องพึ่งพา Object Detection ที่เสถียรและต่อเนื่องเท่านั้น

3) ปัจจัยที่ส่งผลต่อความสำเร็จของระบบ

- ความต่อเนื่องของการตรวจจับเป้าหมาย → มีผลโดยตรงต่อความสามารถของ PID ในการควบคุม
- ความเร็วและ latency ของโมเดล → เป็นปัจจัยจำกัดของระบบแบบ real-time
- สถานการณ์ที่มี motion ปรับเปลี่ยนฉับพลัน (เช่น blink, teleport) → ต้องใช้ระบบเสริม เช่น Predictive Filter หรือ Model-based Control

ข้อสรุป

- ระบบ Aibot แบบควบคุมกลไกจำเป็นต้อง ออกแบบให้สอดคล้องกันทั้งด้านโมเดลตรวจจับและระบบควบคุม (PID)
- โมเดล YOLOv5m เป็นจุดสมดุลที่เหมาะสมที่สุดในระบบปัจจุบัน จากการทดสอบจริง
- การนำระบบนี้ไปใช้งานในสภาพแวดล้อม dynamic สูงจำเป็นต้องมีการเสริม logic เพิ่มเติม เช่น:
 - PID ที่มี deadzone / anti-windup
 - ระบบ memory-based tracking หรือ predictive filter

บทที่ 5 บทสรุป

5.1 สรุปวัตถุประสงค์และขอบเขตของโครงการ

โครงการนี้มีวัตถุประสงค์หลักในการพัฒนา ระบบช่วยเล็งและยิงเป้าหมายแบบอัตโนมัติ (Physical Aimbot System) ที่สามารถทำงานได้ในสภาวะแวดล้อมของเกมยิงมุมมองบุคคลที่หนึ่ง (FPS – First-Person Shooter) ซึ่งเป็นสภาพแวดล้อมที่มีการเปลี่ยนแปลงอย่างรวดเร็วและซับซ้อนทั้งในเชิงภาพ (Visual Dynamics) และการเคลื่อนไหว (Motion Dynamics)

เป้าหมายเชิงวิศวกรรมของโครงการนี้ คือ การนำเทคนิคปัญญาประดิษฐ์ (AI) ด้านการตรวจจับวัตถุ (Object Detection) มาผสานเข้ากับ ระบบควบคุมเชิงกลไกจริง (Physical Control System) เพื่อให้หุ่นยนต์สามารถ "เล็งและคลิก" ได้อย่างแม่นยำตามเป้าหมายบนหน้าจอในเวลาจริง (Real-Time Operation)

โครงการถูกออกแบบเป็นระบบเชิงบูรณาการ (Integrated System) โดยแบ่งเป็น 3 องค์ประกอบหลัก ดังนี้:

1. ระบบตรวจจับวัตถุ (Object Detection System)

จับภาพหน้าจอของเกม แล้วประมวลผลด้วยโมเดล YOLOv5 ที่ผ่านการฝึกฝนจาก dataset จำเพาะ เพื่อระบุตำแหน่งของเป้าหมาย (Target) ในแต่ละเฟรม โดยสามารถรองรับเฟรมเรตขั้นต่ำ 12 fps

2. ระบบควบคุม PID (PID Control System)

รับค่าความคลาดเคลื่อนของตำแหน่งเป้าในแนวแกน X และ Y (dx , dy) แล้วใช้ PID Controller เพื่อคำนวณความเร็วเชิงเส้น (Velocity Command) ที่จะใช้ขับเคลื่อนหุ่นยนต์ให้เล็งไปยังตำแหน่งของเป้าหมาย

3. ระบบฮาร์ดแวร์หุ่นยนต์ (Motion Platform)

ประกอบด้วยล้อ mecanum 4 ล้อ ขับเคลื่อนโดยมอเตอร์ Yellow TT ที่ควบคุมผ่าน ESP32 ด้วยคำสั่งผ่าน Serial จาก Python เพื่อเลื่อนเมาส์จริงบนพื้นโต๊ะ โดยให้การเคลื่อนที่อิสระในแนวระนาบ

ขอบเขตของโครงการ (Scope of Work)

สิ่งที่ครอบคลุม

- การออกแบบและฝึกโมเดล YOLOv5 ขนาดต่าง ๆ สำหรับตรวจจับศัตรูในเกม FPS จริง
- การประเมินผลแบบเปรียบเทียบในเชิง:
 - ความแม่นยำ (Accuracy)

- ความเร็วในการประมวลผล (Latency)
- ผลกระทบต่อ FPS ของเกม
- การพัฒนาและทดสอบระบบควบคุม PID ที่ปรับปรุงด้วยวิธี Data-Driven (ผ่านการบันทึก log)
- การประเมินผลระบบจริง (Integration Testing) ในสถานการณ์เกมจำลองที่มีการเคลื่อนไหวสูง

สิ่งที่อยู่นอกขอบเขต

- การฝึกโมเดลแบบ deep learning เองจากศูนย์ (Pretrained YOLO ถูกนำมา fine-tune)
- การวิเคราะห์ performance แบบ competitive gaming (เช่น เปรียบเทียบกับโปรเพลเยอร์)
- การใช้งานระบบกับเกมจริงแบบ commercial (เช่น CS:GO, Valorant)
- การออกแบบระบบความปลอดภัยหรือระบบป้องกัน cheat detection

ข้อจำกัดที่สำคัญของโครงการ

- ข้อจำกัดด้านฮาร์ดแวร์:
 - ใช้มอเตอร์ราคาถูกที่ไม่มี encoder → ไม่สามารถ feedback ความเร็วหรือตำแหน่งได้
 - มอเตอร์ที่ใช้มีแรงบิดต่ำและทดเฟืองน้อย → ควบคุมได้ยากที่ความเร็วต่ำ
 - ไม่มีระบบหมุนกลิ้ง → ไม่สามารถ track เป้าหมายแบบ 360°
 - พันผิวของฟันทดสอบมีแรงเสียดทานต่ำ ทำให้เกิด overshoot ได้ง่าย
- ข้อจำกัดด้านซอฟต์แวร์:
 - โมเดล YOLOv5s ตรวจจับได้ไม่ต่อเนื่องในสถานการณ์ dynamic สูง
 - ไม่มี logic พิเศษสำหรับการจัดการการ teleport หรือเป้าหมายหายไปจากเฟรม (discontinuous input)
- ข้อจำกัดด้านทรัพยากรและเวลา:
 - ทดสอบได้เพียง 1 รอบต่อโมเดลในแต่ละ Scenario
 - ไม่สามารถเก็บ percentile จากระบบ Kovaak ได้โดยตรง (เนื่องจากระบบนับเฉพาะคะแนนสูงสุด)

บทสรุปของบทนี้

โครงการนี้เป็นการพัฒนาและทดลองระบบที่ผสมระหว่าง AI, Robotics, และ Control System เข้าไว้ด้วยกันในบริบทของเกมจริง โดยมุ่งเน้นทั้งการพัฒนาเทคนิค และการวิเคราะห์พฤติกรรมของระบบแบบวิศวกรรม แม้

ผลลัพธ์จะยังไม่สมบูรณ์แบบในแง่ของความแม่นยำ แต่สามารถแสดงให้เห็นถึงจุดแข็ง จุดอ่อน และโอกาสในการพัฒนาในอนาคตได้อย่างเป็นระบบและมีเหตุผลรองรับ

5.2 สรุปการออกแบบและการพัฒนา

โครงการ “Physical Aimbot for FPS Games using Computer Vision” ได้รับการออกแบบและพัฒนาแบบแยกส่วน (Modular Architecture) เพื่อให้สามารถจัดการกับแต่ละองค์ประกอบของระบบได้อย่างอิสระ และรองรับการวิเคราะห์ที่ชัดเจนในเชิงวิศวกรรม โดยการพัฒนาแบ่งออกเป็น 3 ส่วนสำคัญ ได้แก่:

1. ระบบการตรวจจับเป้าหมาย (Object Detection)
2. ระบบควบคุมหุ่นยนต์ (Motion Control & PID)
3. ระบบเชื่อมต่อและบูรณาการซอฟต์แวร์-ฮาร์ดแวร์ (Software-Hardware Integration)

5.2.1 ระบบตรวจจับเป้าหมาย (Object Detection with YOLOv5)

วัตถุประสงค์การออกแบบ:

เพื่อให้สามารถตรวจจับศัตรูในเกม FPS ได้แบบเรียลไทม์ จากภาพที่ได้จากหน้าจอเกมจริง (ผ่านเว็บแคม) โดยมีความแม่นยำพอที่จะใช้เป็น input สำหรับระบบควบคุม

รายละเอียดการพัฒนา:

- เลือกใช้ YOLOv5 เป็นสถาปัตยกรรมหลัก เนื่องจากมี trade-off ที่ดีระหว่าง accuracy และ inference speed
- สร้าง dataset ด้วยการบันทึกหน้าจอจากเกม Kovaak แล้วทำการ annotate ด้วย labeling
- ทำการฝึกโมเดลหลายขนาด (s_old, s_new, m, l, x) พร้อม augment
- เขียนสคริปต์ modeltest.py สำหรับรันตรวจจับ frame-by-frame และเก็บผลลัพธ์แบบวัดผลเชิงปริมาณ

ประเด็นทางวิศวกรรม:

- s_old มี latency ต่ำ (~15.5 ms) แต่ตรวจจับพลาดมากโดยเฉพาะใน dynamic scenario
- s_new แก้ปัญหา dataset แต่ยังไม่เพียงพอ → ตรวจจับไม่ต่อเนื่อง
- m คือโมเดลที่สมดุลที่สุดในการใช้งานจริง → ความแม่นยำเพียงพอ, latency ต่ำพอ (~18.9 ms)
- สร้างระบบเทียบผลกับ baseline ของมนุษย์ (จาก Kovaak median) เพื่อวัด performance จริง

5.2.2 ระบบควบคุมการเคลื่อนที่ (Motion Control with PID)

วัตถุประสงค์การออกแบบ:

ควบคุมหุ่นยนต์ให้สามารถนำ pointer หรือเมาส์เคลื่อนที่ไปยังตำแหน่งศูนย์กลางของ bounding box ที่ตรวจจับได้ โดยอิงจากค่าความคลาดเคลื่อนเชิงตำแหน่ง (dx, dy)

รายละเอียดการพัฒนา:

- พัฒนา PID แบบแยกแกน (X และ Y) พร้อมการปรับจูนด้วย Sandbox Scenario เฉพาะ
- พัฒนาโมดูล logger.py สำหรับเก็บ log การทำงานเฟรมต่อเฟรม
- ใช้ plotter.py เพื่อวิเคราะห์กราฟ dx, dy → ใช้สำหรับปรับจูนค่าพารามิเตอร์
- PID ที่จูนแล้วมีค่า:
 - X-axis: $K_P = 0.18$, $K_I = 0.5$, $K_D = 15.0$
 - Y-axis: $K_P = 0.13$, $K_I = 0.3$, $K_D = 7.0$

ประเด็นทางวิศวกรรม:

- ในช่วงตรวจจับต่อเนื่อง → PID สามารถลด error ได้เร็วและเสถียร
- เมื่อ input ขาด (model ตรวจไม่เจอ) → PID idle และระบบหยุดนิ่ง
- ไม่มี feedback (encoder) → ควบคุมแบบ open-loop เท่านั้น → เกิด overshoot ได้ง่าย
- พ้นผิวลื่น → หยุดไม่ได้แม้สั่งหยุดแล้ว → PID เสียสมดุลในช่วง deceleration

5.2.3 ระบบการเชื่อมต่อและควบคุมฮาร์ดแวร์ (Integration & Control)

วัตถุประสงค์การออกแบบ:

เชื่อมต่อบระบบซอฟต์แวร์ (Object Detection + PID Control) เข้ากับหุ่นยนต์จริงผ่าน Serial Communication เพื่อสั่งการเคลื่อนที่ได้แบบเรียลไทม์

รายละเอียดการพัฒนา:

- สื่อสารระหว่าง Python กับ ESP32 ผ่าน PySerial
- ESP32 ควบคุมมอเตอร์ 4 ตัวด้วย driver ผ่านคำสั่งความเร็วแบบ PWM
- พัฒนาโค้ด serial_sender.py เพื่อส่งข้อมูลไปยัง ESP32
- ใช้ระบบ mode สำหรับสลับระหว่างไม่ขยับ คลิ๊กค้าง และคลิ๊กรว

ประเด็นทางวิศวกรรม:

- ระบบมี latency สะสม (I/O + มอเตอร์ response + inertia) ~48–50 ms → ตรวจวัดด้วย test_latency.py
- ค่าที่วัดได้สูงกว่า target (≤ 16 ms) → ระบบไม่สามารถตอบสนองต่อ fast flick หรือ fast strafe ได้ทัน
- ESP32 ไม่มี feedback → ไม่สามารถปรับความเร็วเชิง adaptive ได้
- การคลิกอัตโนมัติทำผ่าน detection threshold และ bounding box overlap

5.3 สรุปผลการทดลอง

5.3.1 ระบบตรวจจับเป้าหมาย (Object Detection with YOLOv5)

จากการทดลองในหัวข้อ 4.1.1 และ 4.3.1 พบว่า **ประสิทธิภาพของโมเดล YOLOv5 ขึ้นอยู่กับ 2 ปัจจัยหลัก** ได้แก่:

1. ขนาดของโมเดล (และจำนวนพารามิเตอร์ภายใน)
2. ลักษณะของ Scenario ที่ใช้ทดสอบ

ผลการทดลองสรุปได้ว่า:

- YOLOv5s_old และ YOLOv5s_new ตรวจจับได้ดีเฉพาะในฉากง่าย หรือเมื่อเป้าหมายอยู่ในระยะชัดเจน แต่ **ไม่สามารถตรวจจับได้ต่อเนื่อง**ในฉากที่มี dynamic สูง เช่น teleport หรือมุกล้อเอียง
- YOLOv5m ตรวจจับได้เสถียรที่สุดโดยไม่ส่งผลกระทบต่อ FPS หรือ latency อย่างมีนัยสำคัญ (~18.9 ms/frame) → ให้ input ต่อ PID อย่างสม่ำเสมอ
- YOLOv5x แม้จะมี accuracy สูงในการตรวจจับ (ตรวจเจอแทบทุก frame) แต่ latency เฉลี่ยเกิน 30 ms → ไม่เหมาะกับระบบที่ต้องตอบสนองภายใน 1 เฟรม

ข้อสังเกตเพิ่มเติม:

- โมเดลที่ใหญ่ขึ้นไม่ได้หมายถึง "เหมาะกับการใช้งานจริง" เสมอไป หากระบบ downstream (เช่น PID หรือ hardware) ไม่สามารถรับมือกับ latency ที่เพิ่มขึ้นได้
- dataset และการ augment ส่งผลชัดเจนกับความสามารถในการ generalize โดยเฉพาะในกรณีของ small_new vs small_old

5.3.2 ระบบควบคุม PID (PID Controller Performance)

การทดสอบ PID Controller ถูกแบ่งออกเป็น 2 ลักษณะของ Scenario:

กรณีที่ 1: เป้าเคลื่อนที่ต่อเนื่อง (เช่น Close Short Strafes)

- PID สามารถลดค่า dx, dy ได้อย่างรวดเร็วภายในไม่กี่เฟรมแรก

- มีการ overshoot บ้างในช่วงเริ่มต้น แต่ระบบเข้าสู่ steady-state ได้
- ความเบี่ยงเบนมาตรฐานยังค่อนข้างสูง ($dx_std \approx 310$ px) → บ่งชี้ว่าระบบยังมี jitter จากมอเตอร์และการสั่นของพื้น

กรณีที่ 2: เป้ามีการ teleport (Blink)

- เมื่อไม่มี input จากโมเดล → PID idle → ไม่เกิดการควบคุม
- เมื่อ input กลับมาอีกครั้ง → เกิด error spike → ระบบเร่งพรวด → overshoot รุนแรง
- เกิด oscillation หลังจากกลับมา track เป้าได้ → แสดงถึง lack of damping ในระบบควบคุม

สรุป:

- PID ที่จูนใน sandbox ทำงานได้ดีใน environment ที่ model ตรวจเจออย่างต่อเนื่อง
- แต่ไม่สามารถจัดการกับ input ที่ไม่ต่อเนื่องได้ → ต้องพิจารณาเสริม logic เช่น anti-windup, predictive control

5.3.3 ระบบรวม (Full-System Integration)

การทดสอบระบบรวมในหัวข้อ 4.3.1 (เล็งและคลิกในเกมจริงแบบ end-to-end) ให้ข้อมูลสำคัญว่าระบบมีข้อจำกัดในหลายมิติ

ปัญหาที่พบ:

- แม้จะตรวจจับได้ทุกเฟรม (เช่น YOLOv5x) → แต่ latency ในการประมวลผล + การแสดงผล (Aimbot View) ช้ากว่าระบบควบคุมทันได้ → ทำให้ไม่สามารถควบคุมได้ทันเวลา
- Accuracy ของระบบต่ำกว่า 40% ในทุกโมเดล ส่วนใหญ่เกิดจาก:
 - หน่วง overshoot เป้า → คลิปพลาด
 - PID รับ input ไม่ทัน → ตอบสนองช้าเกินเป้าเคลื่อนที่
- ไม่มีระบบ tracking 360° หากเป้าหลุดออกนอกเฟรม → ระบบหยุดนิ่ง → ต้องหมุนกล้องเอง

วิเคราะห์เชิงวิศวกรรม:

- Accuracy ต่ำไม่ได้เกิดจากโมเดลอย่างเดียว แต่เป็นผลรวมของ:
 - Latency จาก detection
 - ความไม่แม่นยำของมอเตอร์
 - พื้นลื่น (braking distance สูง)

- ไม่มี encoder → ไม่มี feedback correction
- ระบบยังไม่สามารถ "ปิด loop" ได้จริง หาก input หาย → output ไม่เกิด → ไม่มี control loop

สรุปภาพรวม

แม้ระบบจะสามารถทำงานแบบครบวงจร (จากการตรวจจับ → คำนวณ → ควบคุม → เคลื่อนไหว → คลิก) ได้จริง

แต่จากผลการทดสอบชัดเจนว่า:

- ความแม่นยำของระบบยังไม่เพียงพอสำหรับใช้งานจริง (Real Deployment)
- PID ที่ไม่มี feedback ไม่สามารถจัดการ dynamic scenario ได้
- การเลือกโมเดลและการจูนระบบต้องพิจารณาร่วมกันทั้งด้าน latency, detection continuity, และ physical capability

5.4 ปัญหาและข้อจำกัดของระบบ

ในกระบวนการพัฒนาและทดสอบระบบแบบครบวงจร พบว่ามีข้อจำกัดเชิงโครงสร้างหลายประการที่ส่งผลให้ระบบไม่สามารถทำงานได้อย่างมีประสิทธิภาพสูงสุด โดยสามารถแยกวิเคราะห์ตามหมวดได้ดังนี้:

หมวด	ปัญหา	ผลกระทบต่อระบบ	วิเคราะห์เชิงวิศวกรรม
1. โมเดลตรวจจับ (YOLOv5)	ตรวจจับไม่ต่อเนื่อง โดยเฉพาะใน YOLOv5s และ scenario ที่มี blink/teleport	PID ไม่ได้รับค่า dx/dy → ไม่มี output → ระบบหยุดนิ่ง (idle)	PID ทำงานในรูปแบบ feedforward หากไม่มี input จาก detection ระบบจะไม่สามารถคำนวณแรงควบคุมได้
2. ความเร็วของ Scenario	ความเร็วของเป้าหมายสูงเกินกว่าที่ระบบควบคุมทัน เช่น scenario ระดับฝึกหนักใน Kovaak	หุ่นยนต์เล็งไม่ทัน, overshoot เพิ่มขึ้น → ความแม่นยำโดยรวมต่ำ	PID ถูกจูนจาก sandbox ที่เป้าเคลื่อนช้า เมื่อใช้กับเป้าเร็ว → error derivative spike → ควบคุมผิดพลาด

หมวด	ปัญหา	ผลกระทบต่อระบบ	วิเคราะห์เชิงวิศวกรรม
3. ฮาร์ดแวร์ (มอเตอร์ + โครงสร้าง)	ใช้มอเตอร์ Yellow TT แรงบิดต่ำ ไม่มี encoder	ไม่สามารถหยุดได้ทันหรือควบคุมความเร็วอย่างแม่นยำ → overshoot, delay	ไม่มี feedback → PID ทำงานแบบ open-loop → ไม่สามารถ compensate แรงเฉื่อย หรือ nonlinear drag ได้
4. พื้นผิวของระบบเคลื่อนที่	พื้นเป็นวัสดุกระเบื้อง มีความลื่น	ล้อหยุดหมุนแล้ว แต่หุ่นยังลื่น → PID overshoot, jitter	ความลื่นทำให้สัญญาณ PWM หยุดแล้ว แต่แรงเฉื่อยยังพาเคลื่อนต่อ → response ช้ากว่าที่ PID คำนวณไว้
5. ความล่าช้า (Latency) และ FPS	โมเดลขนาดใหญ่ (YOLOv5x) แม้จะตรวจแม่นยำ แต่มี latency สูง (>30 ms/frame)	Aimbot View แสดงผลช้า → PID ควบคุมล่าช้า → คลิกเป้าไม่ทัน	เมื่อ input lag > control update interval → ค่าที่ PID ใช้ไม่ใช่ "เป้าหมายล่าสุด" → เกิด control delay
6. การติดตามเป้า (Target Tracking)	ไม่มีระบบหมุนกล้องหรือ logic ติดตาม 360°	หากเป้าหมายหลุดเฟรม → ระบบตรวจไม่เจอ → หยุดนิ่ง	ระบบไม่มี memory หรือ predictive model ในการประมาณตำแหน่งเป้าเมื่อหลุดเฟรม → ต้องใช้ manual assist

ข้อวิเคราะห์เพิ่มเติม

- ปัญหาที่พบ ไม่ได้จำกัดอยู่แค่ส่วนเดียวของระบบ แต่เกิดจากการทำงานร่วมกันของหลายปัจจัย (เช่น ตรวจจับ → ส่ง input → ควบคุม → เคลื่อนจริง)
- ทุกจุดอ่อนมี “จุดกระทบต่อ chain ทั้งระบบ” ทำให้ไม่สามารถแยกปัญหาใดปัญหาหนึ่งออกมาจัดการลำพังได้ ต้องออกแบบการแก้ไขแบบ system-level

5.5 แนวทางการพัฒนาในอนาคต

จากผลการทดสอบในบทที่ 4 และการวิเคราะห์ข้อจำกัดในหัวข้อ 5.4 พบว่าระบบยังมีปัญหาหลายด้านที่เกี่ยวข้องกันระหว่าง โมเดล, ควบคุม, และฮาร์ดแวร์ ดังนั้นแนวทางการพัฒนาในอนาคตควรคำนึงถึงการแก้ปัญหาในระดับ

ระบบ (System-Level Improvement) เพื่อให้เกิดการทำงานที่สอดคล้องและเสถียรมากขึ้น โดยสามารถจัดหมวด
แนวทางได้ดังนี้:

1. เพิ่มระบบ Feedback (Encoder Feedback Integration)

แนวคิด:

ติดตั้ง encoder บนล้อหรือมอเตอร์เพื่อตรวจจับความเร็วหรือระยะทางจริงของหุ่นยนต์ → ส่งค่า feedback กลับ
เข้าสู่ PID

เหตุผลทางวิศวกรรม:

- PID ปัจจุบันเป็น open-loop → ไม่รู้ว่าหุ่นเคลื่อนไปไกลแค่ไหนจริง (ใช้ vision ในการรับ Feedback)
- หากมี encoder → สามารถปรับ gain หรือหยุดการเคลื่อนที่ได้ตรงจุดจริง
- ลดปัญหา overshoot และ jitter โดยเฉพาะเมื่อพื้นผิวลื่น

ประโยชน์ที่จะเกิดขึ้น:

- เพิ่มความแม่นยำของการเคลื่อนที่
- ปรับ PID แบบ real-time ได้ตาม velocity feedback

2. เปลี่ยนชุดมอเตอร์ (Motor Upgrade)

แนวคิด:

เปลี่ยนจากมอเตอร์ Yellow TT (ราคาถูก, torque ต่ำ) เป็น มอเตอร์ที่ควบคุมได้แม่นยำกว่า เช่น:

- DC motor พร้อม encoder
- Stepper motor + driver ที่รองรับ microstepping

เหตุผลทางวิศวกรรม:

- มอเตอร์เดิมมีแรงเฉื่อยน้อย ควบคุมความเร็วต่ำไม่ได้
- Stepper หรือ brushed DC motor พร้อม feedback → ควบคุม acceleration/deceleration ได้ดีขึ้น

ประโยชน์ที่จะเกิดขึ้น:

- ลด overshoot ในจังหวะหยุด
- ตอบสนองแม่นยำในเฟรมแรกหลังการตรวจจับเป้า

3. พัฒนา logic สำหรับ tracking 360° (Full-Frame Target Reacquisition)

แนวคิด:

เพิ่มความสามารถในการหมุนกล้องหรือหมุนแพลตฟอร์มเพื่อ **ดึงเป้าหมายกลับเข้าสู่เฟรม** อัตโนมัติ

เหตุผลทางวิศวกรรม:

- ขณะนี้ระบบไม่มีความสามารถติดตามเมื่อเป้าหมายหายออกนอกเฟรม
- ต้องหมุนกล้องด้วยมือ → ไม่สอดคล้องกับระบบอัตโนมัติ
- สามารถใช้ logic เช่น:
 - ตรวจสอบ last known direction → ค่อย ๆ หมุนกลับ
 - ใช้ bounding box ขนาดใหญ่ (search mode) เมื่อไม่มี detection

ประโยชน์ที่จะเกิดขึ้น:

- ระบบสามารถทำงานต่อได้แม้เป้าหมายหลุดเฟรม
- ลดภาระผู้ใช้งาน และรองรับ scenario dynamic ได้ดีขึ้น

4. เสริมเทคนิคควบคุม (Advanced PID Strategies)

แนวคิด:

พัฒนา PID ให้สามารถรับมือกับ input ที่ไม่ต่อเนื่อง และแรงแปรผันสูง โดยเสริมฟังก์ชันเช่น:

- **Deadzone** → ลดการสั่นในช่วง error ใกล้ 0
- **Anti-windup** → ป้องกัน I term สะสมในช่วงไม่มี input
- **Fuzzy/Adaptive PID** → ปรับ gain อัตโนมัติตามพฤติกรรมเป้าหมาย

เหตุผลทางวิศวกรรม:

- ปัจจุบัน PID standard ไม่เหมาะกับ input ที่กระโดดหรือขาดหาย
- การใช้ anti-windup จะช่วยให้ระบบไม่ overreact เมื่อกลับมาตรวจเจอเป้า

ประโยชน์ที่จะเกิดขึ้น:

- เพิ่มเสถียรภาพของการควบคุมใน scenario ที่ model ตรวจเจอแบบกะพริบ
- ทำให้ motion control เข้ากับ dynamic ของเกมจริงมากขึ้น

5. ปรับปรุง Dataset และ Retrain โมเดล

แนวคิด:

เพิ่มข้อมูลภาพที่ครอบคลุมกรณีเป่าย่อ หันข้าง หรือ blink teleport → เพื่อให้ YOLO generalize ได้มากขึ้น

เหตุผลทางวิศวกรรม:

- YOLOv5s_new ที่ retrain แล้ว ยังตรวจจับเป้าที่อยู่ในมุมเฉียงหรือกำลังเคลื่อนย้ายตำแหน่งไม่ได้ดี
- การเพิ่ม data variation เช่น:
 - เป้าย่อและหายใต้ HUD
 - การหันข้างแบบเร็ว
 - การ overlap หลายตัว

ประโยชน์ที่จะเกิดขึ้น:

- โมเดลสามารถตรวจจับได้แม่นยำมากขึ้นแม้ในสถานการณ์ยาก
- ลด input missing ที่ส่งผลกระทบต่อ PID

6. ติดตั้งบนพื้นผิวที่มีแรงเสียดทานสูง

แนวคิด:

เปลี่ยนพื้นผิวการทดสอบจากกระเบื้องเรียบ → เป็นวัสดุที่เพิ่มแรงเสียดทาน เช่น ยาง หรือ EVA foam

เหตุผลทางวิศวกรรม:

- หุ่นยนต์บนพื้นลื่นแม่สั่งหยุดแล้ว → ลื่นต่อ → ทำให้ PID คำนวณผิด
- แรงเบรกทางกลไม่เพียงพอ → เกิด overshoot แม้ค่าควบคุมจะเหมาะสมแล้ว

ประโยชน์ที่จะเกิดขึ้น:

- หยุดแม่นยำขึ้น → ลด overshoot และ jitter
- ทำให้ระบบ PID มีเสถียรภาพขึ้นโดยไม่ต้องปรับ code

5.6 สรุปภาพรวมของโครงงาน

โครงงาน “Physical Aimbot for FPS Games using Computer Vision” เป็นการทดลองบูรณาการเทคโนโลยีหลายแขนง ได้แก่ Computer Vision, Control Theory และ Embedded Hardware เข้าด้วยกันภายใต้เป้าหมายร่วมคือ **สร้างระบบที่สามารถตรวจจับและเล็งเป้าหมายได้แบบอัตโนมัติในเกม FPS ที่มีความ dynamic สูง** แม้ผลลัพธ์สุดท้าย ยังไม่สามารถสร้างความแม่นยำที่น่าพอใจในเชิงปฏิบัติ (accuracy โดยรวมต่ำกว่า 40%, มี overshoot และ jitter บ่อยครั้ง) แต่โครงงานนี้ถือว่าบรรลุเป้าหมายหลักในเชิงวิศวกรรมหลายด้าน ได้แก่:

- การสร้างระบบทำงานจริงแบบครบวงจร (end-to-end integration) จากการตรวจจับ → ควบคุม → เคลื่อนที่ → คลิก
- การวิเคราะห์ปัญหาอย่างมีระบบ ตั้งแต่ระดับโมเดล ไปจนถึงฮาร์ดแวร์และพื้นผิวการเคลื่อนที่
- การพัฒนากระบวนการทดสอบและวิเคราะห์เชิง **Data-Driven PID Tuning** โดยใช้ Log และ Visualization เพื่ออธิบายพฤติกรรมของระบบได้อย่างมีเหตุผล

โครงการนี้ยังแสดงให้เห็นว่า แม้จะมีข้อจำกัดมากมาย เช่น ฮาร์ดแวร์ราคาประหยัด ไม่มี feedback, ระบบตรวจจับที่ไม่ต่อเนื่อง หรือการควบคุมที่ล่าช้า — แต่ด้วยแนวคิดทางวิศวกรรมที่ถูกต้อง เช่น:

- การแยกโมดูลเพื่อวิเคราะห์เป็นส่วน ๆ
- การวัดค่าเชิงตัวเลขทุกเฟรม (dx, dy)
- การใช้ Scenario ที่หลากหลายเพื่อทดสอบระบบ
- การวิเคราะห์ผลรวมระหว่างกราฟ, log และพฤติกรรมภาพรวม

ผู้วิจัยสามารถ “พิสูจน์ได้เชิงวิศวกรรม” ว่าทำไมระบบจึงไม่แม่นยำ

และหากมีการพัฒนาในจุดที่วิเคราะห์ไว้ (เช่น เพิ่ม encoder, ปรับโมเดล, เสริม logic tracking) ระบบนี้สามารถก้าวไปสู่เวอร์ชันที่มีประสิทธิภาพจริงได้ในอนาคต

“ความล้มเหลวในการเล็ง ไม่ใช่ความล้มเหลวของระบบ แต่คือข้อเท็จจริงที่เราวัดได้ และเข้าใจมันอย่างมีเหตุผล”
และนั่นคือจุดเริ่มต้นของการพัฒนาที่ยั่งยืน

เอกสารอ้างอิง

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.
- [2] Jocher, G., et al. (2020). *YOLOv5 by Ultralytics* [Online]. Available: <https://github.com/ultralytics/yolov5>
- [3] Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv:2004.10934.
- [4] Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767.
- [5] Spong, M. W., Hutchinson, S., & Vidyasagar, M. (2006). *Robot Modeling and Control*. John Wiley & Sons.
- [6] Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control*. Springer.
- [7] Kelly, R. (2006). *Modern Control Theory Applied to Inverse Kinematics for Robot Manipulators*. IEEE Transactions on Industrial Electronics, 53(2), 629–637.
- [8] Åström, K. J., & Murray, R. M. (2010). *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press.
- [9] Åström, K. J., & Hägglund, T. (2006). *Advanced PID Control*. ISA—The Instrumentation, Systems, and Automation Society.
- [10] Meyer, G. (2020). *Understanding and Using Mecanum Wheels* [White Paper]. AndyMark, Inc. Available: <https://www.andymark.com/>
- [11] Espressif Systems. (2020). *ESP32 Technical Reference Manual* [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [12] Arduino. (2023). *DRV8833 Dual Motor Driver Carrier* [Online]. Available: <https://www.pololu.com/product/2130>
- [13] Texas Instruments. (n.d.). *DRV8833 Dual H-Bridge Motor Driver Datasheet* [Online]. Available: <https://www.ti.com/lit/ds/symlink/drv8833.pdf>
- [14] Roboflow Inc. (2024). *Roboflow Dataset Management and Augmentation Platform* [Online]. Available: <https://roboflow.com/>
- [15] OpenCV Team. (2024). *OpenCV-Python Tutorials* [Online]. Available: <https://docs.opencv.org/>
- [16] Python Software Foundation. (n.d.). *Python 3.8 Documentation* [Online]. Available: <https://docs.python.org/3.8/>
- [17] PySerial Project. (2023). *pySerial Documentation* [Online]. Available: <https://pyserial.readthedocs.io/>
- [18] Pynput Developers. (2023). *pynput — Control and monitor input devices* [Online]. Available: <https://pynput.readthedocs.io/>
- [19] Logitech. (2021). *G Pro X Superlight Specifications* [Online]. Available: <https://www.logitechg.com/>

[20] NVIDIA Corporation. (2023). *NVIDIA Tesla V100 GPU Architecture Whitepaper* [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>

ภาคผนวก

ภาคผนวก A : Source Code

GitHub

สามารถเข้าถึง Source code ทั้งหมดของงานได้ที่

GitHub : https://github.com/mudmini009/FRA361_Open_Topic.git

extract_frame.py

```
import cv2
import os

# Define the path for the videos and the output directory
video_path = r"D:\UNIVERSITY\YR3\FRA361_Open_Topic\DATASET\VIDEO"
output_base_path = r"D:\UNIVERSITY\YR3\FRA361_Open_Topic\DATASET\PICTURE"

# List of video files in the folder
video_files = [
    "3D Switching Robots.mp4",
    "Buff Robots 360.mp4",
    "Close Fast Strafes Easy Invincible Robot 25% Slower.mp4",
    "CLS Click Robots Rounded.mp4",
```

```

"Correction Accuracy - Close I Strafe Robot.mp4"
]

# Iterate through each video file
for video_file in video_files:
    # Construct full path to the video
    video_full_path = os.path.join(video_path, video_file)

    # Open the video file
    cap = cv2.VideoCapture(video_full_path)

    # Check if the video was successfully opened
    if not cap.isOpened():
        print(f"Error: Could not open video file {video_file}")
        continue

    # Create a folder to store the frames from the video
    output_folder = os.path.join(output_base_path, video_file.split('.')[0])
    os.makedirs(output_folder, exist_ok=True)

    frame_count = 0
    fps = int(cap.get(cv2.CAP_PROP_FPS)) # Get the FPS of the video
    success, frame = cap.read()

    while success:
        # Save frame as image
        frame_filename = os.path.join(output_folder, f"frame_{frame_count:04d}.png")
        cv2.imwrite(frame_filename, frame)

        # Read the next frame
        success, frame = cap.read()
        frame_count += 1

    cap.release()

```



```
print(f"Frames extracted for {video_file} and saved to {output_folder}")
```

```
print("All videos processed.")
```

test.cpp

```
#include <Arduino.h>
```

```
// =====
```

```
// Pin configuration
```

```
// =====
```

```
// Front Left Motor
```

```
#define FL_IN1 25
```

```
#define FL_IN2 26
```

```
// Front Right Motor
```

```
#define FR_IN1 27
```

```
#define FR_IN2 14
```

```
// Back Left Motor
```

```
#define BL_IN1 12
```

```
#define BL_IN2 13
```

```
// Back Right Motor
```

```
#define BR_IN1 32
```

```
#define BR_IN2 33
```

```
void setMotor(int in1, int in2, bool forward) {
```

```
    digitalWrite(in1, forward ? HIGH : LOW);
```

```
    digitalWrite(in2, forward ? LOW : HIGH);
```

```
}
```

```
void stopAll() {
```

```
digitalWrite(FL_IN1, LOW); digitalWrite(FL_IN2, LOW);  
digitalWrite(FR_IN1, LOW); digitalWrite(FR_IN2, LOW);  
digitalWrite(BL_IN1, LOW); digitalWrite(BL_IN2, LOW);  
digitalWrite(BR_IN1, LOW); digitalWrite(BR_IN2, LOW);  
}
```

```
void setup() {  
  pinMode(FL_IN1, OUTPUT); pinMode(FL_IN2, OUTPUT);  
  pinMode(FR_IN1, OUTPUT); pinMode(FR_IN2, OUTPUT);  
  pinMode(BL_IN1, OUTPUT); pinMode(BL_IN2, OUTPUT);  
  pinMode(BR_IN1, OUTPUT); pinMode(BR_IN2, OUTPUT);  
}
```

```
void loop() {  
  // === เดินหน้า ===  
  setMotor(FL_IN1, FL_IN2, true);  
  setMotor(FR_IN1, FR_IN2, true);  
  setMotor(BL_IN1, BL_IN2, true);  
  setMotor(BR_IN1, BR_IN2, true);  
  delay(500); stopAll(); delay(500);
```

```
  // === ถอยหลัง ===  
  setMotor(FL_IN1, FL_IN2, false);  
  setMotor(FR_IN1, FR_IN2, false);  
  setMotor(BL_IN1, BL_IN2, false);  
  setMotor(BR_IN1, BR_IN2, false);  
  delay(500); stopAll(); delay(500);
```

```
  // === สไลด์ขวา ===  
  setMotor(FL_IN1, FL_IN2, true);  
  setMotor(FR_IN1, FR_IN2, false);  
  setMotor(BL_IN1, BL_IN2, false);  
  setMotor(BR_IN1, BR_IN2, true);  
  delay(500); stopAll(); delay(500);
```

```
// === สไตร์ฟซ้าย ===
```

```
setMotor(FL_IN1, FL_IN2, false);  
setMotor(FR_IN1, FR_IN2, true);  
setMotor(BL_IN1, BL_IN2, true);  
setMotor(BR_IN1, BR_IN2, false);  
delay(500); stopAll(); delay(500);
```

```
// === หมุนขวา ===
```

```
setMotor(FL_IN1, FL_IN2, true);  
setMotor(FR_IN1, FR_IN2, false);  
setMotor(BL_IN1, BL_IN2, true);  
setMotor(BR_IN1, BR_IN2, false);  
delay(500); stopAll(); delay(500);
```

```
// === หมุนซ้าย ===
```

```
setMotor(FL_IN1, FL_IN2, false);  
setMotor(FR_IN1, FR_IN2, true);  
setMotor(BL_IN1, BL_IN2, false);  
setMotor(BR_IN1, BR_IN2, true);  
delay(500); stopAll(); delay(500);
```

```
// === เฉียงหน้า-ขวา === (FL + BR forward)
```

```
setMotor(FL_IN1, FL_IN2, true);  
setMotor(BR_IN1, BR_IN2, true);  
delay(500); stopAll(); delay(500);
```

```
// === เฉียงหน้า-ซ้าย === (FR + BL forward)
```

```
setMotor(FR_IN1, FR_IN2, true);  
setMotor(BL_IN1, BL_IN2, true);  
delay(500); stopAll(); delay(500);
```

```
// === เฉียงหลัง-ขวา === (FR + BL backward)
```

```

setMotor(FR_IN1, FR_IN2, false);
setMotor(BL_IN1, BL_IN2, false);
delay(500); stopAll(); delay(500);

// === เหยียงหลัง-ซ้าย === (FL + BR backward)
setMotor(FL_IN1, FL_IN2, false);
setMotor(BR_IN1, BR_IN2, false);
delay(500); stopAll(); delay(500);

while (true) delay(1000); // หยุดดูหลังทดสอบครบ
}

```

minspeedtest.py

```
# minspeedtest.py
```

```
import threading, time, serial, keyboard
```

```
from config import SERIAL_PORT, SERIAL_BAUD
```

```
# ——— Setup Serial —————
```

```
ser = serial.Serial(SERIAL_PORT, SERIAL_BAUD, timeout=1)
```

```
time.sleep(2)
```

```
print(f"[+] Opened {SERIAL_PORT}@{SERIAL_BAUD}")
```

```
# ——— Shared state —————
```

```
speeds = [30, 30, 30, 30] # [FL, FR, BL, BR]
```

```
wheel_names = ['Front-Left', 'Front-Right', 'Back-Left', 'Back-Right']
```

```
selected = 0
```

```
stop_flag = False
```

```
print("Controls: Z=cycle wheel, C=+1 PWM, X=-1 PWM, ESC=quit")
```

```
print(f"Selected → {wheel_names[selected]} = {speeds[selected]}")
```

——— Sender thread ———

```
def sender_loop():
```

```
    """Continuously send the latest speeds at 20 Hz."""
```

```
    while not stop_flag:
```

```
        msg = "{};{};{};{}\n".format(*speeds)
```

```
        ser.write(msg.encode('utf-8'))
```

```
        print("→ sent:", msg.strip())
```

```
        time.sleep(0.05) # 20 Hz
```

```
thread = threading.Thread(target=sender_loop, daemon=True)
```

```
thread.start()
```

——— Key-handling loop ———

```
last = {'z': False, 'c': False, 'x': False}
```

```
try:
```

```
    while True:
```

```
        # Cycle selected wheel on Z press
```

```
        z = keyboard.is_pressed('z')
```

```
        if z and not last['z']:
```

```
            selected = (selected + 1) % 4
```

```
            print(f"Selected → {wheel_names[selected]} = {speeds[selected]}")
```

```
            last['z'] = z
```

```
        # Increase PWM on C
```

```
        c = keyboard.is_pressed('c')
```

```
        if c and not last['c']:
```

```
            speeds[selected] += 1
```

```
            print(f"{wheel_names[selected]} PWM → {speeds[selected]}")
```

```
            last['c'] = c
```

```
        # Decrease PWM on X
```

```
        x = keyboard.is_pressed('x')
```

```
        if x and not last['x']:
```

```
            speeds[selected] -= 1
```

```
    print(f"{wheel_names[selected]} PWM → {speeds[selected]}")
    last['x'] = x
```

```
# Quit on ESC
if keyboard.is_pressed('esc'):
    print("Exiting...")
    break
```

```
time.sleep(0.01) # small debounce
```

finally:

```
# signal sender to stop, wait a moment, then clean up
stop_flag = True
thread.join(0.1)
ser.close()
print("Serial closed.")
```

frame_count.py

```
import cv2
```

```
def main():
```

```
    # === EDIT THESE PATHS/PARAMETERS ===
    video_path = "RoboTS180.mp4" # <-- Replace with your video file path
    skip_frames = 1 # Change to N to sample every Nth frame (e.g., 2 to see every other frame)
    # =====
```

```
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print(f"Error: cannot open video {video_path}")
        return
```

```
    frame_idx = 0
    counts = []
```

```

while True:
    ret, frame = cap.read()
    if not ret:
        break
    if frame_idx % skip_frames != 0:
        frame_idx += 1
        continue

    cv2.imshow("Frame", frame)
    cv2.waitKey(1) # Necessary to render the window

    # Prompt user for manual count input
    while True:
        val = input(f'Frame {frame_idx}: Enter detected robot count (or 'q' to quit): ')
        if val.lower() == 'q':
            cap.release()
            cv2.destroyAllWindows()
            summary(counts)
            return
        try:
            count = int(val)
            break
        except ValueError:
            print("Invalid input. Please enter an integer.")

    counts.append(count)
    frame_idx += 1

cap.release()
cv2.destroyAllWindows()
summary(counts)

def summary(counts):

```

```

total_frames = len(counts)
total_detections = sum(counts)
false_negatives = sum(1 for c in counts if c == 0)

print("\n=== Summary ===")
print(f"Frames counted: {total_frames}")
print(f"Total detections: {total_detections}")
print(f"Average detections per frame: {total_detections/total_frames if total_frames else 0:.2f}")
print(f"False negatives (frames with zero detections): {false_negatives}")

if __name__ == "__main__":
    main()

```

modeltest.py

```

import os
import pathlib

# ← Patch PosixPath → WindowsPath so torch.load can reconstruct paths on Windows
pathlib.PosixPath = pathlib.WindowsPath

import yolov5
import cv2

# === EDIT THESE PATHS & PARAMETERS ===
VIDEO_DIR = r"D:\UNIVERSITY\YR3\FRA361_Open_Topic\Experience\modeltest"
VIDEO_FILES = [
    "[N] CLS Click Robots.mp4",
    "Cata IC Fast Strafes Robot.mp4",
    "Close Fast Colosseum Robots No Shooting.mp4",
    "Close Fast Strafes Invincible OW Robot.mp4",
    "RoboTS180.mp4"
]

```



```

MODEL_CONFIGS = [
    {"name": "small_old",
     "weights": r"model\model1(y5s)\weights\best.pt",
     "imgsz": 640},
    {"name": "small_new",
     "weights": r"model\model2(y5s)\yolov5s_best.pt",
     "imgsz": 640},
    {"name": "medium",
     "weights": r"model\model3(y5m)\yolov5m_best.pt",
     "imgsz": 640},
    {"name": "large",
     "weights": r"model\model4(y5l)\yolov5l_best.pt",
     "imgsz": 640},
    {"name": "x",
     "weights": r"model\model5(y5x)\yolov5x_best.pt",
     "imgsz": 640},
]

```

```

CONF_THRESHOLD = 0.25  # minimum confidence
SKIP_FRAMES    = 1     # process every Nth frame (1 = every frame)
# =====

```

```

def analyze_video(model, video_path, imgsz):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print(f" X could not open {os.path.basename(video_path)}")
        return None

    total_frames = detected_frames = no_detection_frames = total_detections = 0
    sum_confidences = 0.0
    frame_idx = 0

    while True:

```

```

ret, frame = cap.read()
if not ret:
    break

if frame_idx % SKIP_FRAMES != 0:
    frame_idx += 1
    continue

total_frames += 1
results = model(frame, size=imgsz)
preds = results.pred[0]

nbox = preds.shape[0]
if nbox > 0:
    detected_frames += 1
    total_detections += nbox
    sum_confidences += float(preds[:, 4].sum())
else:
    no_detection_frames += 1

frame_idx += 1

cap.release()
avg_conf = (sum_confidences / total_detections) if total_detections else 0.0
return {
    "total_frames": total_frames,
    "detected_frames": detected_frames,
    "no_detection_frames": no_detection_frames,
    "total_detections": total_detections,
    "avg_confidence": avg_conf,
}

def main():
    for cfg in MODEL_CONFIGS:

```

```

print(f"\n=== Model: {cfg['name']} ===")
model = yolov5.load(cfg["weights"])
model.conf = CONF_THRESHOLD

for vid in VIDEO_FILES:
    path = os.path.join(VIDEO_DIR, vid)
    stats = analyze_video(model, path, cfg["imgsz"])
    if stats is None:
        continue

    print(f"Video: {vid}")
    print(f" • Total frames      : {stats['total_frames']}")
    print(f" • Detected frames    : {stats['detected_frames']}")
    print(f" • No-detect frames   : {stats['no_detection_frames']}")
    print(f" • Total detections   : {stats['total_detections']}")
    print(f" • Avg. confidence    : {stats['avg_confidence']:.2f}")

if __name__ == "__main__":
    main()

modeltest_latency.py

import os
import pathlib
import time

# Monkey-patch to avoid PosixPath unpickle errors on Windows
import pathlib as _pl
_pl.PosixPath = _pl.WindowsPath

import yolov5
import cv2

# === EDIT THESE PATHS & PARAMETERS ===
VIDEO_DIR = r"D:\UNIVERSITY\YR3\FRA361_Open_Topic\Experience\modeltest"

```

```
VIDEO_FILES = [
```

```
    "[N] CLS Click Robots.mp4",
    "Cata IC Fast Strafes Robot.mp4",
    "Close Fast Colosseum Robots No Shooting.mp4",
    "Close Fast Strafes Invincible OW Robot.mp4",
    "RoboTS180.mp4"
```

```
]
```

```
MODEL_CONFIGS = [
```

```
    {"name": "small_old", "weights": r"model\model1(y5s)\weights\best.pt", "imgsz": 640},
    {"name": "small_new", "weights": r"model\model2(y5s)\yolov5s_best.pt", "imgsz": 640},
    {"name": "medium", "weights": r"model\model3(y5m)\yolov5m_best.pt", "imgsz": 640},
    {"name": "large", "weights": r"model\model4(y5l)\yolov5l_best.pt", "imgsz": 640},
    {"name": "x", "weights": r"model\model5(y5x)\yolov5x_best.pt", "imgsz": 640},
```

```
]
```

```
CONF_THRESHOLD = 0.25 # minimum confidence
```

```
SKIP_FRAMES = 1 # process every Nth frame (1 = every frame)
```

```
# =====
```

```
def analyze_video(model, video_path, imgsz):
```

```
    cap = cv2.VideoCapture(video_path)
```

```
    if not cap.isOpened():
```

```
        print(f" X could not open {os.path.basename(video_path)}")
```

```
        return None
```

```
    latencies = []
```

```
    frame_idx = 0
```

```
    # we only need timing, no need to count detections here
```

```
    while True:
```

```
        ret, frame = cap.read()
```

```
        if not ret:
```

```
            break
```

```

if frame_idx % SKIP_FRAMES != 0:
    frame_idx += 1
    continue

t0 = time.time()
_ = model(frame, size=imgsz) # inference
t1 = time.time()

latencies.append((t1 - t0) * 1000.0) # ms
frame_idx += 1

cap.release()
if not latencies:
    return None

return {
    "frames": len(latencies),
    "mean_ms": sum(latencies) / len(latencies),
    "min_ms": min(latencies),
    "max_ms": max(latencies),
}

def main():
    print(f"Running inference timing on {len(VIDEO_FILES)} videos x {len(MODEL_CONFIGS)} models...\n")
    for cfg in MODEL_CONFIGS:
        print(f"=== Model: {cfg['name']} ===")
        model = yolov5.load(cfg["weights"])
        model.conf = CONF_THRESHOLD

        for vid in VIDEO_FILES:
            path = os.path.join(VIDEO_DIR, vid)
            stats = analyze_video(model, path, cfg["imgsz"])
            if stats is None:

```

```

        continue

    print(f'{vid:40s} frames: {stats['frames']:3d} '
          f'mean: {stats['mean_ms']:.1f} ms '
          f'min: {stats['min_ms']:.1f} ms '
          f'max: {stats['max_ms']:.1f} ms")
    print()

if __name__ == "__main__":
    main()

```

modeltest_runwithgame.py

```

import pygetwindow as gw
import mss
import numpy as np
import cv2
import yolov5
import pathlib as _pl
_pl.PosixPath = _pl.WindowsPath

# === CONFIGURATION ===
MODEL_PATH = r'model\mmodel5(y5x)\yolov5x_best.pt' # path to your .pt
CONFIDENCE = 0.25 # min detection confidence
IMGSZ = 640 # inference size
TARGET_WINDOW = "KovaaK" # window title substring
WINDOW_NAME = "Aimbot View"
# =====

def find_game_window(target=TARGET_WINDOW):
    """Find the first visible window whose title contains target."""
    windows = [w for w in gw.getAllWindows()]

```

```

        if target.lower() in w.title.lower() and w.visible]
    if not windows:
        raise RuntimeError(f"No visible window containing '{target}' found!")
    return windows[0]

def get_game_capture(window):
    """Yields BGR frames captured from the given window."""
    with mss.mss() as sct:
        monitor = {
            "top": window.top,
            "left": window.left,
            "width": window.width,
            "height": window.height,
            "mon": 0
        }
        while True:
            img = np.array(sct.grab(monitor))
            yield cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)

def main():
    # load model
    model = yolov5.load(MODEL_PATH)
    model.conf = CONFIDENCE

    # find game window & setup display
    window = find_game_window()
    cv2.namedWindow(WINDOW_NAME, cv2.WINDOW_NORMAL)

    try:
        for frame in get_game_capture(window):
            # inference + render
            results = model(frame, size=IMGSZ)
            out = results.render()[0]
            # show

```

```

        cv2.imshow(WINDOW_NAME, out)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    finally:
        cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

test_latency.py

test_latency.py

```

import time
import pyautogui
from serial_sender import SerialSender

```

——— Configuration ———

```

TEST_COUNT = 10          # number of trials per direction
SPEED      = 70          # wheel % impulse (±)
THRESHOLD  = 5           # px cursor must move to register
PAUSE      = 1.0         # seconds between trials

```

——— Helper: wait for cursor to move THRESHOLD px from y0 ———

```

def wait_for_movement(y0, direction):

```

```

    """

```

```

    Block until cursor has moved by THRESHOLD px in 'up' or 'down' dir.

```

```

    """

```

```

    while True:

```

```

        _, y = pyautogui.position()

```

```

        dy = y - y0

```

```

        if direction == 'up' and dy <= -THRESHOLD:

```



```

        return

    if direction == 'down' and dy >= THRESHOLD:
        return

    time.sleep(0.001)

# ————— Run one batch of discrete-pulse tests —————
def run_tests(direction, sender):
    latencies = []
    for i in range(1, TEST_COUNT+1):
        # sample baseline cursor Y
        _, y0 = pyautogui.position()

        # 1) send single impulse
        pwm = SPEED if direction=='up' else -SPEED
        sender.send_wheels(pwm,pwm,pwm,pwm)

        # 2) record send timestamp
        t0 = time.time()

        # 3) immediately stop wheels
        sender.send_wheels(0,0,0,0)

        # 4) wait for cursor move
        wait_for_movement(y0, direction)
        latency = (time.time() - t0)*1000
        latencies.append(latency)
        print(f"[{direction.upper()}] #{i} {latency:.1f} ms")

        # rest before next trial
        time.sleep(PAUSE)

    return latencies

```

— Main

```
if __name__ == "__main__":
    print("Hardware→Mouse latency test (discrete pulses)")
    sender = SerialSender()
    if not sender.enabled:
        print("Serial port unavailable, aborting.")
        exit(1)

    print("Get your cursor on a blank area. Starting in 3s...")
    time.sleep(3)

    up_lat = run_tests('up', sender)
    down_lat = run_tests('down', sender)

    sender.close()

    print("\n=== LATENCY RESULTS ===")
    print("Up   ms:", [f"{l:.1f}" for l in up_lat])
    print("Down ms:", [f"{l:.1f}" for l in down_lat])
    print(f"Avg Up:  {sum(up_lat)/len(up_lat):.1f} ms")
    print(f"Avg Down: {sum(down_lat)/len(down_lat):.1f} ms")
```

main.cpp

```
#include <Arduino.h>

// =====
// Pin configuration
// =====
// unsigned long lastMicros = 0;
// Front Left Motor
#define FL_IN1 25
#define FL_IN2 26
```

```

// Front Right Motor
#define FR_IN1 27
#define FR_IN2 14

// Back Left Motor
#define BL_IN1 12
#define BL_IN2 13

// Back Right Motor
#define BR_IN1 32
#define BR_IN2 33

// =====
// PWM configuration
// =====
const int PWM_FREQ      = 5000; // 5 kHz PWM
const int PWM_RESOLUTION = 8;   // 8-bit (0-255)

// Channels for each IN1/IN2 pair
enum {
    CH_FL_F = 0, CH_FL_R,
    CH_FR_F, CH_FR_R,
    CH_BL_F, CH_BL_R,
    CH_BR_F, CH_BR_R
};

// Last commanded wheel PWMs (-255...+255)
int fl_pwm = 0, fr_pwm = 0, bl_pwm = 0, br_pwm = 0;

/**
 * Map percent input (-100...+100) to PWM:
 *
 * pct == 0   → 0
 * pct == +1  → +xxx
 * pct == +100 → +255

```

* pct == -1 → -xxx

* pct == -100 → -255

*/

```
int percentToPwm(int pct) {
```

```
    if (pct > 0) {
```

```
        // map [1..100] → [155..255]
```

```
        //return map(pct, 1, 100, 155, 255);
```

```
        return map(pct, 1, 100, 140, 255);
```

```
    } else if (pct < 0) {
```

```
        // map [-1..-100] → [-145..-255]
```

```
        // map() needs increasing ranges, so flip:
```

```
        int absPct = -pct;
```

```
        //int pwm    = map(absPct, 1, 100, 155, 255);
```

```
        int pwm    = map(absPct, 1, 100, 140, 255);
```

```
        return -pwm;
```

```
    }
```

```
    // pct == 0
```

```
    return 0;
```

```
}
```

```
// Drive a single motor: chF = forward channel, chR = reverse channel
```

```
void driveMotor(int chF, int chR, int pwm) {
```

```
    if (pwm >= 0) {
```

```
        ledcWrite(chF, pwm);
```

```
        ledcWrite(chR, 0);
```

```
    } else {
```

```
        ledcWrite(chF, 0);
```

```
        ledcWrite(chR, -pwm);
```

```
    }
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    while (!Serial) { delay(10); }
```

```
Serial.println("ESP32 Wheel Sync with Deadzone Mapping Ready");
```

```
// Configure pins as outputs
```

```
pinMode(FL_IN1, OUTPUT); pinMode(FL_IN2, OUTPUT);
```

```
pinMode(FR_IN1, OUTPUT); pinMode(FR_IN2, OUTPUT);
```

```
pinMode(BL_IN1, OUTPUT); pinMode(BL_IN2, OUTPUT);
```

```
pinMode(BR_IN1, OUTPUT); pinMode(BR_IN2, OUTPUT);
```

```
// Attach each pin to a PWM channel
```

```
ledcSetup(CH_FL_F, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(FL_IN1, CH_FL_F);
```

```
ledcSetup(CH_FL_R, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(FL_IN2, CH_FL_R);
```

```
ledcSetup(CH_FR_F, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(FR_IN1, CH_FR_F);
```

```
ledcSetup(CH_FR_R, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(FR_IN2, CH_FR_R);
```

```
ledcSetup(CH_BL_F, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(BL_IN1, CH_BL_F);
```

```
ledcSetup(CH_BL_R, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(BL_IN2, CH_BL_R);
```

```
ledcSetup(CH_BR_F, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(BR_IN1, CH_BR_F);
```

```
ledcSetup(CH_BR_R, PWM_FREQ, PWM_RESOLUTION); ledcAttachPin(BR_IN2, CH_BR_R);
```

```
}
```

```
void loop() {
```

```
    // Read a line if available
```

```
    if (Serial.available()) {
```

```
        String line = Serial.readStringUntil('\n');
```

```
        line.trim();
```

```
        if (line.length() > 0) {
```

```
            int p0, p1, p2, p3;
```

```
            int n = sscanf(line.c_str(), "%d,%d,%d,%d", &p0, &p1, &p2, &p3);
```

```
            if (n == 4) {
```

```
                fl_pwm = percentToPwm(p0);
```

```
                fr_pwm = percentToPwm(p1);
```

```
                bl_pwm = percentToPwm(p2);
```

```

    br_pwm = percentToPwm(p3);
    Serial.printf(
        "Rx %%%: [%d, %d, %d, %d] → PWM: [FL=%d, FR=%d, BL=%d, BR=%d]\n",
        p0, p1, p2, p3, fl_pwm, fr_pwm, bl_pwm, br_pwm
    );
} else {
    Serial.println("! parse error");
}
}
}

```

```

// Re-apply last PWM values at 100 Hz
driveMotor(CH_FL_F, CH_FL_R, fl_pwm);
driveMotor(CH_FR_F, CH_FR_R, fr_pwm);
driveMotor(CH_BL_F, CH_BL_R, bl_pwm);
driveMotor(CH_BR_F, CH_BR_R, br_pwm);

```

```

// // record and echo timestamp
// unsigned long now_us = micros();
// Serial.printf("ACK %lu\n", now_us);
delay(10);
}

```

annotator.py

```

#annotate_and_collect
# annotator.py
import cv2
import math
from distance import get_center
from config import CENTER_BOX_SIZE, CHEST_Y_FACTOR

def annotate_and_collect(frame, boxes, mode, logger):
    """

```

Draw aim-box + marker/line, compute dx, dy, on_target,
log them (dx,dy,mode), and return (dx,dy,on_target), annotated frame.

"""

```
frame = frame.copy()
```

```
h, w = frame.shape[:2]
```

```
cx, cy = get_center(w, h)
```

```
half = CENTER_BOX_SIZE // 2
```

```
left, top = cx - half, cy - half
```

```
right, bottom = cx + half, cy + half
```

```
# draw central "aim" box
```

```
cv2.rectangle(frame, (left, top), (right, bottom), (0,255,0), 2)
```

```
dx, dy, on_target = 0.0, 0.0, 0
```

```
if boxes.size:
```

```
    # pick closest box
```

```
    best = min(
```

```
        ((math.hypot((b[0]+b[2])/2 - cx, (b[1]+b[3])/2 - cy), b)
```

```
        for b in boxes),
```

```
        key=lambda x: x[0]
```

```
    )[1]
```

```
    x1, y1, x2, y2 = best[:4]
```

```
# chest point
```

```
tx = (x1 + x2) / 2
```

```
ty = y1 + (y2 - y1) * CHEST_Y_FACTOR
```

```
# pixel error: +right, +up
```

```
dx = tx - cx
```

```
dy = cy - ty
```

```
# on_target if overlapping
```

```
if not (x2 < left or x1 > right or y2 < top or y1 > bottom):
```

```
    on_target = 1
```

```

# draw marker + line
cv2.circle(frame, (int(tx), int(ty)), 5, (0,0,255), -1)
cv2.line(frame, (cx, cy), (int(tx), int(ty)), (255,0,0), 2)

# log dx, dy, mode
logger.log(round(dx,1), round(dy,1), mode)

# overlay
cv2.putText(frame, f"Mode: {mode}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,0), 2)
cv2.putText(frame, f"dx: {dx:.1f}px", (10, 70),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)
cv2.putText(frame, f"dy: {dy:.1f}px", (10,110),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)
cv2.putText(frame, f"OnT: {on_target}", (10,150),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,255), 2)

return (dx, dy, on_target), frame

```

capture.py

```

#to pull the screen out
# capture.py
import pygetwindow as gw
import mss
import numpy as np
import cv2
import difflib

def select_game_window():
    """List visible windows and prompt user to type part of the title."""
    windows = [w for w in gw.getAllWindows() if w.visible]

```



```

if not windows:
    raise RuntimeError("No visible windows found!")

titles = [w.title for w in windows]
print("Available windows:")
for t in titles:
    print(f" - {t}")

target = input("\nType part of the window title to select: ")
matches = difflib.get_close_matches(target, titles, n=1, cutoff=0)
if not matches:
    raise RuntimeError(f"No window matches '{target}' found.")
selected = matches[0]
print(f"\nSelected window: '{selected}'")

return next(w for w in windows if w.title == selected)

```

```

def get_game_capture(window):
    """Generator yielding BGR frames of that window."""
    with mss.mss() as sct:
        mon = {
            "top": window.top,
            "left": window.left,
            "width": window.width,
            "height": window.height,
            "mon": 0
        }
        while True:
            img = np.array(sct.grab(mon))
            yield cv2.cvtColor(img, cv2.COLOR_BGRA2BGR)

```

clicker.py

#handle mouse click

clicker.py

```

from pynput.mouse import Button, Controller
import time

mouse = Controller()
click_held = False
last_click_time = 0
click_toggle = False

def handle_mouse_click(mode, on_target):
    """
    mode 1: hold mouse while on_target
    mode 2: pulse-click at ~70ms on, ~72ms off while on_target
    """
    global click_held, last_click_time, click_toggle

    # if neither clicky mode → reset any held click
    if mode not in (1, 2):
        if click_held:
            mouse.release(Button.left)
            click_held = False
            click_toggle = False
            return

    if mode == 1:
        if on_target and not click_held:
            mouse.press(Button.left); click_held = True
        elif not on_target and click_held:
            mouse.release(Button.left); click_held = False

    elif mode == 2:
        now = time.time()
        if on_target:
            # ensure a full cycle (0.07 + 0.072s) before next pulse
            if not click_toggle or (now - last_click_time > 0.142):

```

```

        mouse.press(Button.left)
        time.sleep(0.07)
        mouse.release(Button.left)
        last_click_time = time.time()
        click_toggle = True
    else:
        click_toggle = False

```

config.py

```

# config.py
# MODEL_PATH    = r"model\model1(y5s)\weights\best.pt"
# MODEL_PATH    = r"model\model2(y5s)\yolov5s_best.pt"
MODEL_PATH     = r"model\model3(y5m)\yolov5m_best.pt"
# MODEL_PATH    = r"model\model4(y5l)\yolov5l_best.pt"
# MODEL_PATH    = r"model\model5(y5x)\yolov5x_best.pt"
CONFIDENCE     = 0.23
IMGSZ          = 640
CENTER_BOX_SIZE = 15 #30
CHEST_Y_FACTOR = 0.25 # aim 30% down from top of bbox

# ----- Serial settings -----
# set SERIAL_PORT to whatever your ESP32 enumerates as:
SERIAL_PORT    = 'COM5'
SERIAL_BAUD    = 115200

# -----
# Flip these to match your wiring / wheel orientation:
FL_DIR = +1
FR_DIR = -1
BL_DIR = -1
BR_DIR = +1

```

```
# Your PID gains...
```

```
# X-axis (ยังดีอยู่)
```

```
KP_X = 0.18
```

```
KI_X = 0.5
```

```
KD_X = 15.0
```

```
KP_Y = 0.13
```

```
KI_Y = 0.3
```

```
KD_Y = 7.0
```

```
# KP_X = 0.15; KI_X = 0.002; KD_X = 60 #deadzone at 140
```

```
# KP_Y = 0.05; KI_Y = 0.03; KD_Y = 22
```

```
# KP_X = 0.2; KI_X = 6; KD_X = 2.1 #deadzone at 135 65cm/360
```

```
# KP_Y = 0.15; KI_Y = 6; KD_Y = 2.5
```

```
MAX_WHEEL_PERCENT = 100
```

```
control.py
```

```
#keyboard-based mode/quit logic
```

```
# control.py
```

```
import keyboard
```

```
QUIT_KEY = 'z'
```

```
MODE_KEYS = {'x': 0, 'c': 1, 'v': 2}
```

```
def should_quit():
```

```
    return keyboard.is_pressed(QUIT_KEY)
```

```
def get_mode(current_mode):
```

```
    for key, m in MODE_KEYS.items():
```

```
        if keyboard.is_pressed(key):
```

```
        return m
    return current_mode
```

detect.py

```
#YOLO load/detect code
```

```
# detect.py
```

```
import yolov5
```

```
def load_model(model_path: str, confidence: float):
```

```
    """Load YOLOv5 model once and set its confidence threshold."""
```

```
    model = yolov5.load(model_path)
```

```
    model.conf = confidence
```

```
    return model
```

```
def detect_objects(model, frame, imgsz: int):
```

```
    """
```

```
    Run inference on a single frame.
```

```
    Returns:
```

```
    - raw bounding-boxes array (xyxy, conf, cls)
```

```
    - annotated BGR image
```

```
    """
```

```
    results = model(frame, size=imgsz)
```

```
    boxes_np = results.xyxy[0].cpu().numpy()
```

```
    annotated = results.render()[0]
```

```
    return boxes_np, annotated
```

distance.py

```
#calculate distance
```

```
#distance.py
```

```
import math
```

```

def get_center(w: int, h: int):
    """Return the pixel-center (cx,cy) of a frame."""
    return w // 2, h // 2

def calculate_distance_and_angle(box, frame_w: int, frame_h: int):
    """
    box = (x1,y1,x2,y2)
    returns (distance_px, angle_deg, (obj_cx,obj_cy))
    0°=up (12), 90°=right, 180°=down, 270°=left
    """
    x1,y1,x2,y2 = box
    ox = (x1+x2)/2
    oy = (y1+y2)/2

    cx, cy = get_center(frame_w, frame_h)
    dx = ox - cx
    dy = cy - oy # invert y so positive=up

    dist = math.hypot(dx, dy)
    ang = math.degrees(math.atan2(dy, dx))
    if ang < 0:
        ang += 360
    return dist, ang, (ox, oy)

```

kinematics.py

```

# mecanum inverse-kinematics
# kinematics.py
def mecanum_inverse(vx, vy, omega=0.0):
    """
    Normalised inverse kinematics (no wheel-radius or geometry scaling).
    vx, vy: translational speeds (forward+, right+)
    omega : rotation (counter-clockwise +)

```

Returns tuple (FL, FR, BL, BR) in arbitrary units.

```
"""
```

```
fl = vy + vx + omega
```

```
fr = vy - vx - omega
```

```
bl = vy - vx + omega
```

```
br = vy + vx - omega
```

```
return fl, fr, bl, br
```

```
def normalise_to_percent(*speeds, limit=100.0):
```

```
    mx = max(abs(s) for s in speeds) or 1.0
```

```
    scale = min(1.0, limit / mx)
```

```
    return [int(s * scale) for s in speeds]
```

logger.py

```
#data logging
```

```
# logger.py
```

```
import csv
```

```
class DataLogger:
```

```
    def __init__(self):
```

```
        self.entries = [] # list of (dx, dy, mode)
```

```
    def log(self, dx, dy, mode):
```

```
        """Append one sample."""
```

```
        self.entries.append((dx, dy, mode))
```

```
    def save_csv(self, filename="aimbot_log.csv"):
```

```
        """Save all entries to CSV."""
```

```
        with open(filename, "w", newline="") as f:
```

```
            w = csv.writer(f)
```

```
            w.writerow(["dx", "dy", "mode"])
```

```
            w.writerows(self.entries)
```

```
print(f"✅ Saved log to {filename}")
```

pid.py

```
# simple P-ID controller
```

```
class PID:
```

```
    def __init__(self, kp, ki=0.0, kd=0.0):
```

```
        self.kp, self.ki, self.kd = kp, ki, kd
```

```
        self.integral = 0.0
```

```
        self.prev_err = 0.0
```

```
    def update(self, error, dt):
```

```
        self.integral += error * dt
```

```
        deriv = (error - self.prev_err) / dt if dt > 0 else 0.0
```

```
        self.prev_err = error
```

```
        return (
```

```
            self.kp * error +
```

```
            self.ki * self.integral +
```

```
            self.kd * deriv
```

```
        )
```

plotter.py

```
# plotter.py
```

```
import matplotlib.pyplot as plt
```

```
def plot_log(entries):
```

```
    """
```

```
    entries: list of (dx, dy, mode)
```

```
    Produces a 3-row plot: mode, dx, dy vs frame index.
```

```
    """
```

```
    xs = list(range(len(entries)))
```

```
    dxs = [e[0] for e in entries]
```



```

dys = [e[1] for e in entries]
modes= [e[2] for e in entries]

fig, axs = plt.subplots(3, 1, sharex=True, figsize=(8, 6))

axs[0].plot(xs, modes, marker="o", linestyle="-")
axs[0].set_ylabel("mode")
axs[0].set_title("Aimbot Data Log")

axs[1].plot(xs, dxs, marker=".", linestyle="-")
axs[1].set_ylabel("dx (px)")

axs[2].plot(xs, dys, marker=".", linestyle="-")
axs[2].set_ylabel("dy (px)")
axs[2].set_xlabel("frame #")

plt.tight_layout()
plt.show()

```

serial_sender.py

```

#send serial to esp32
# serial_sender.py
import time
import serial
from serial.tools import list_ports
from config import SERIAL_PORT, SERIAL_BAUD

class SerialSender:
    def __init__(self):
        ports = [p.device for p in list_ports.comports()]
        print("Available serial ports:", ports)
        try:
            self.ser = serial.Serial(SERIAL_PORT, SERIAL_BAUD, timeout=1)

```

```

        time.sleep(2)
        print(f"Opened serial port {SERIAL_PORT}@{SERIAL_BAUD}")
        self.enabled = True
    except Exception as e:
        print(f"⚠️ Could not open port {SERIAL_PORT}: {e}")
        print("→ Continuing without serial output.")
        self.enabled = False

# def send(self, data):
#     if not self.enabled:
#         return
#     msg = "{:.1f},{:.1f},{:d},{:d}\n".format(*data)
#     self.ser.write(msg.encode('utf-8'))
#     # ← New line: echo so you know it went out
#     print("→ sent:", msg.strip())

def send_wheels(self, fl, fr, bl, br):
    if not self.enabled: return
    self.ser.write(f"{fl},{fr},{bl},{br}\n".encode())

def close(self):
    if self.enabled:
        self.ser.close()

```

wheel_control.py

```

# converts pixel error → [FL,FR,BL,BR] %
# wheel_control.py
import time
from pid import PID
from kinematics import mecanum_inverse, normalise_to_percent
from config import (KP_X, KI_X, KD_X, KP_Y, KI_Y, KD_Y,
                    MAX_WHEEL_PERCENT,
                    FL_DIR, FR_DIR, BL_DIR, BR_DIR)

```

```

pid_x = PID(KP_X, KI_X, KD_X)
pid_y = PID(KP_Y, KI_Y, KD_Y)
_last = time.time()

def pixel_error_to_wheels(dx_px, dy_px, mode):
    if mode == 0:
        return [0, 0, 0, 0]

    now = time.time()
    dt = now - _last if (_last := now) else 1/60

    # --- Apply pixel deadzone ---
    if abs(dx_px) < 10:
        dx_px = 0
    if abs(dy_px) < 10:
        dy_px = 0

    # compute desired forward (vx) and strafe (vy)
    vx = pid_y.update(dy_px, dt) # forward / backward
    vy = pid_x.update(dx_px, dt) # strafe right+
    # vx = pid_y.update(-dy_px, dt) # invert forward/back
    # vy = pid_x.update(-dx_px, dt) # invert strafe

    # raw IK outputs
    fl, fr, bl, br = mecanum_inverse(vx, vy, 0.0)

    # normalize into -MAX...+MAX percent
    perc = normalise_to_percent(fl, fr, bl, br, limit=MAX_WHEEL_PERCENT)
    # apply wiring direction flips
    flp = FL_DIR * perc[0]
    frp = FR_DIR * perc[1]
    blp = BL_DIR * perc[2]
    brp = BR_DIR * perc[3]

```

```
return [flp, frp, blp, brp]
```

main.py

```
#yeah this is the main code
```

```
# main.py
```

```
import pathlib as _pl
```

```
_pl.PosixPath = _pl.WindowsPath
```

```
import cv2
```

```
import time
```

```
import os
```

```
print("cwd:", os.getcwd())
```

```
from capture    import select_game_window, get_game_capture
```

```
from detect     import load_model, detect_objects
```

```
from config     import MODEL_PATH, CONFIDENCE, IMG_SZ
```

```
from annotator  import annotate_and_collect
```

```
from wheel_control import pixel_error_to_wheels
```

```
from serial_sender import SerialSender
```

```
from clicker    import handle_mouse_click
```

```
from control    import get_mode, should_quit
```

```
from logger     import DataLogger
```

```
from plotter    import plot_log
```

```
def main():
```

```
    window = select_game_window()
```

```
    cap = get_game_capture(window)
```

```
    model = load_model(MODEL_PATH, CONFIDENCE)
```

```
    sender = SerialSender()
```

```
    logger = DataLogger()
```

```
    mode = 0
```

```
cv2.namedWindow("Aimbot", cv2.WINDOW_NORMAL)
```

```
print("Press your quit key to exit and show plots.")
```

```
try:
```

```
    while True:
```

```
        frame = next(cap)
```

```
        boxes, ann = detect_objects(model, frame, IMG_SZ)
```

```
        # annotate + log
```

```
        (dx, dy, on_target), out = annotate_and_collect(ann, boxes, mode, logger)
```

```
        cv2.imshow("Aimbot", out)
```

```
        # wheel drive & clicks (unchanged)
```

```
        fl, fr, bl, br = pixel_error_to_wheels(dx, dy, mode)
```

```
        sender.send_wheels(fl, fr, bl, br)
```

```
        handle_mouse_click(mode, on_target)
```

```
        cv2.waitKey(1)
```

```
        if should_quit():
```

```
            break
```

```
        mode = get_mode(mode)
```

```
finally:
```

```
    sender.close()
```

```
    cv2.destroyAllWindows()
```

```
    # at exit: plot & save
```

```
    plot_log(logger.entries)
```

```
    logger.save_csv()
```

```
    print("Done.")
```

```
if __name__ == "__main__":
```

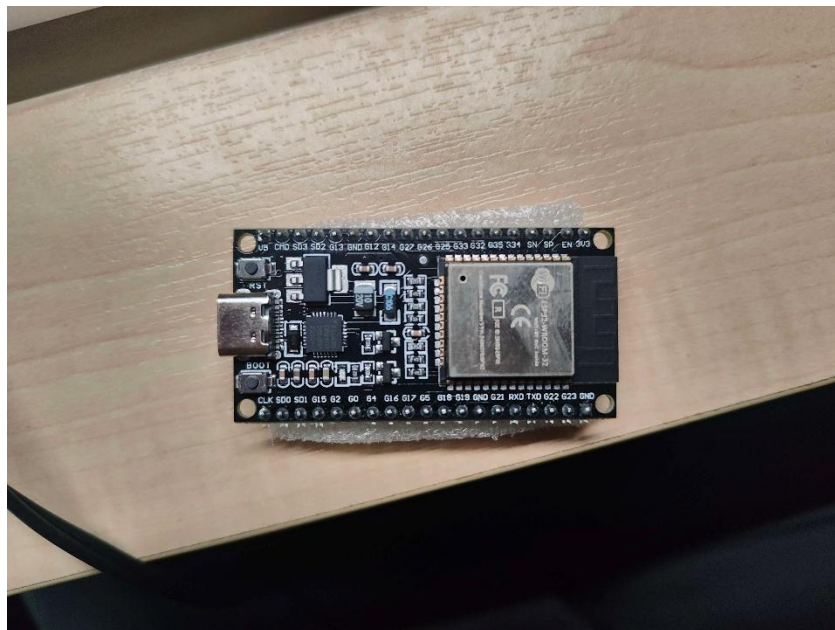
```
    main()
```


ภาคผนวก B : รูปภาพ

รูป รายการของที่สั่งและรอเบิกเงิน

รายการ	ชื่อสินค้า	નોட்டที่ใช่	ราคาต่อชิ้น	จำนวน	ราคารวม	ค่าจัดส่ง	รวมค่าส่ง	ลิงก์สินค้า	ขอใบกำกับภาษีได้	เบิกเงินแล้ว	สั่งแล้ว	ไปเเลของแล้ว	หมายเหตุ
1	Wenton USB A - C 3m		48	1	48	0	48	https://th.shp.ee/FMJBaQ8	☐	☐	☒	☐	Flash sale
2	PETG Pink		399	1	399	0	399	https://th.shp.ee/HTZzsgn	☐	☐	☒	☐	Flash sale
3	DRV8833 Driver		24	3	72		72	https://th.shp.ee/RQFnmf	☐	☐	☒	☐	Flash sale
4	ถ่างปลาตัวผู้		5	2	10		10	https://th.shp.ee/MKHm3K	☐	☐	☒	☐	Flash sale
5	รางสาย 18650 2 แถว		10	1	10		10	https://th.shp.ee/3De5c4a	☐	☐	☒	☐	Flash sale
6	ESP32 38 pin Type C		149	1	149		149	https://th.shp.ee/AvxGxgB	☐	☐	☒	☐	Flash sale
7	สาย 20 cm หูหัด หู-เมีย-เมีย		24	3	72		72	https://th.shp.ee/N3h4Mzb	☐	☐	☒	☐	Flash sale
8	ถ่าน 18650 1ๆ		20	4	80		80	https://th.shp.ee/C3Y6MnY	☐	☐	☒	☐	Flash sale
9	เกียร์ motor 1:48 (6-8 voltage)		18	4	72		72	https://th.shp.ee/fFrTuac	☐	☐	☒	☐	Flash sale
10	XL4016E1 Stepdown		75	1	75		75	https://th.shp.ee/P6KnB88	☐	☐	☒	☐	Flash sale
11	Mecanum 48mm 4 ล้อ		252	1	252		252	https://th.shp.ee/BsWaJgv	☐	☐	☒	☐	Flash sale

รูป ESP32 ที่ได้รับ



รูป ขั้นตอนการ Train และผลลัพธ์จากการ Train ครั้งแรก

```

      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.904      0.78      0.87      0.436

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
93/99    4.29G    0.01393  0.003581  0          9          640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.902      0.81      0.885      0.437

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
94/99    4.29G    0.0138  0.00361  0          20         640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.895      0.803      0.878      0.421

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
95/99    4.29G    0.01379  0.003574  0          15          640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.899      0.798      0.881      0.427

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
96/99    4.29G    0.01324  0.003549  0          13          640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.911      0.803      0.888      0.431

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
97/99    4.29G    0.01326  0.003557  0          15          640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.917      0.829      0.901      0.435

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
98/99    4.29G    0.01308  0.003506  0          13          640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.92      0.838      0.903      0.437

Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
99/99    4.29G    0.01305  0.003425  0          18          640: 100%| 391/3
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.917      0.841      0.903      0.432

100 epochs completed in 2.127 hours.
Optimizer stripped from runs/train/exp3/weights/last.pt, 14.4MB
Optimizer stripped from runs/train/exp3/weights/best.pt, 14.4MB

Validating runs/train/exp3/weights/best.pt...
Fusing layers...
YOLOv5s summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.932      0.804      0.892      0.504

Results saved to runs/train/exp3

```

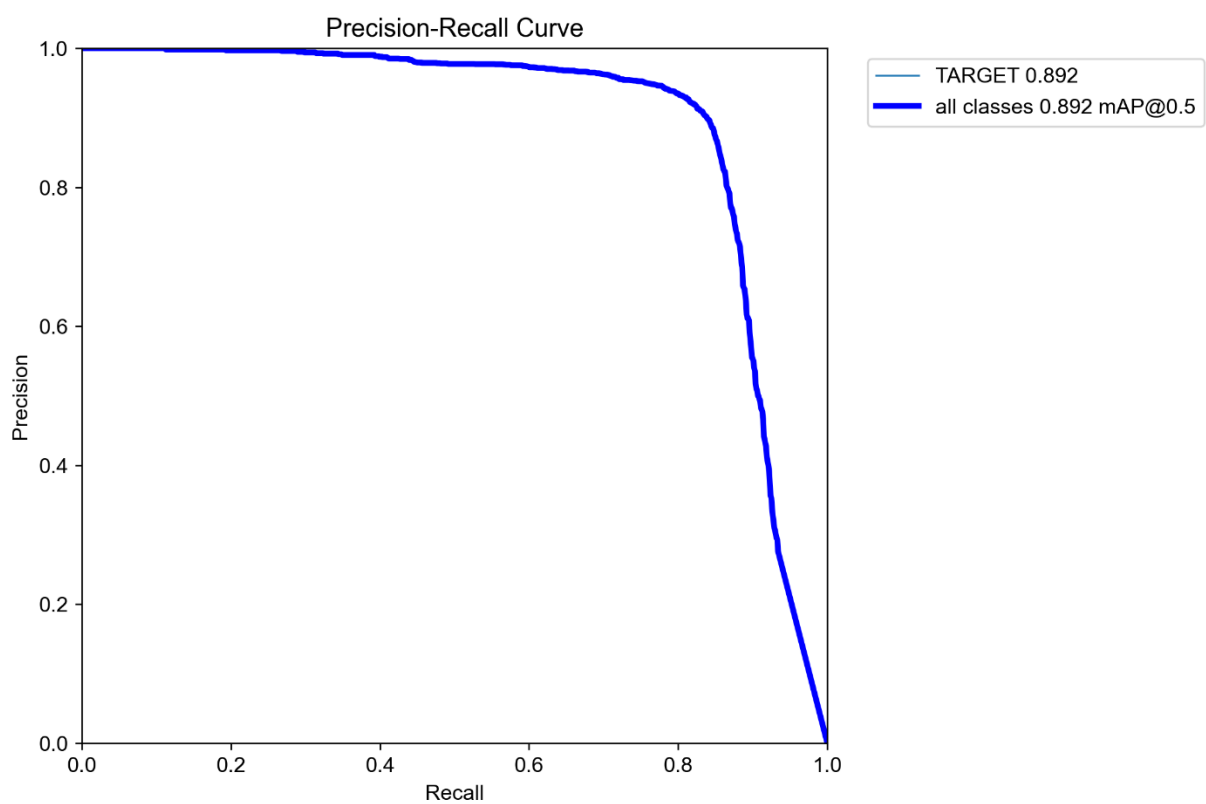
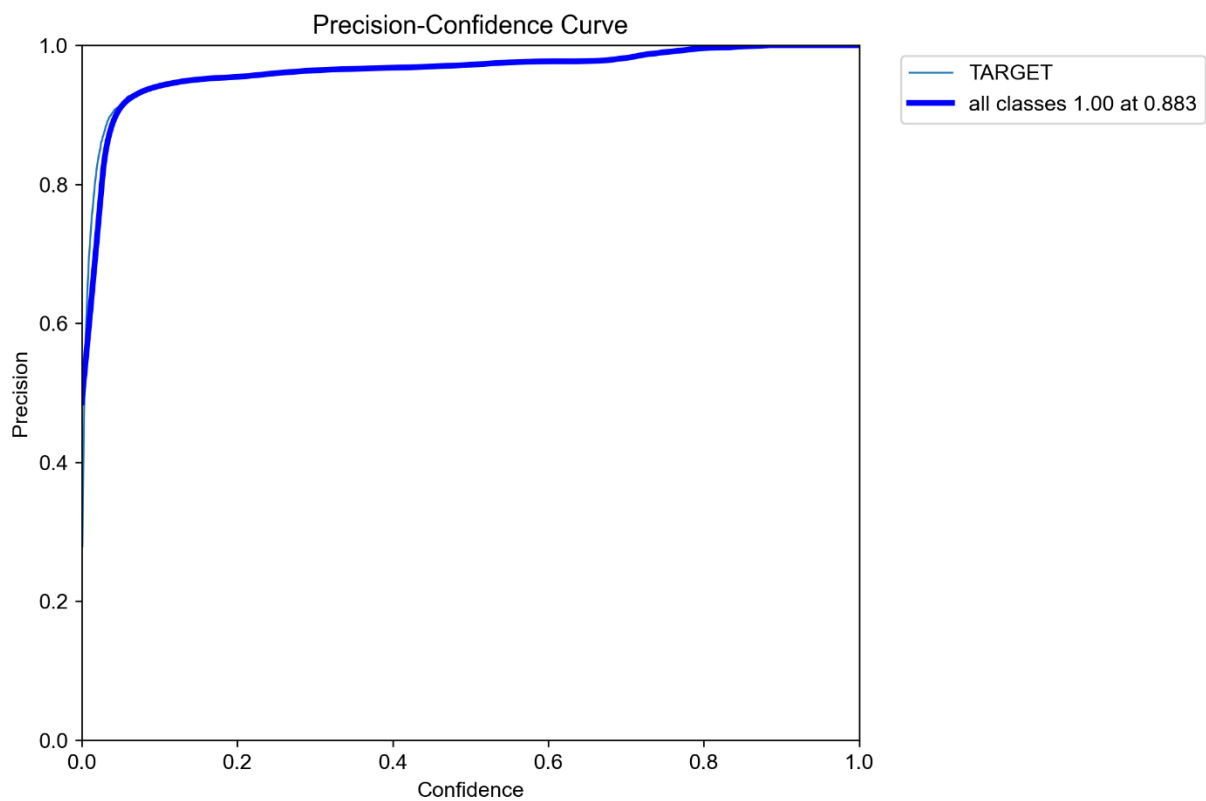
```

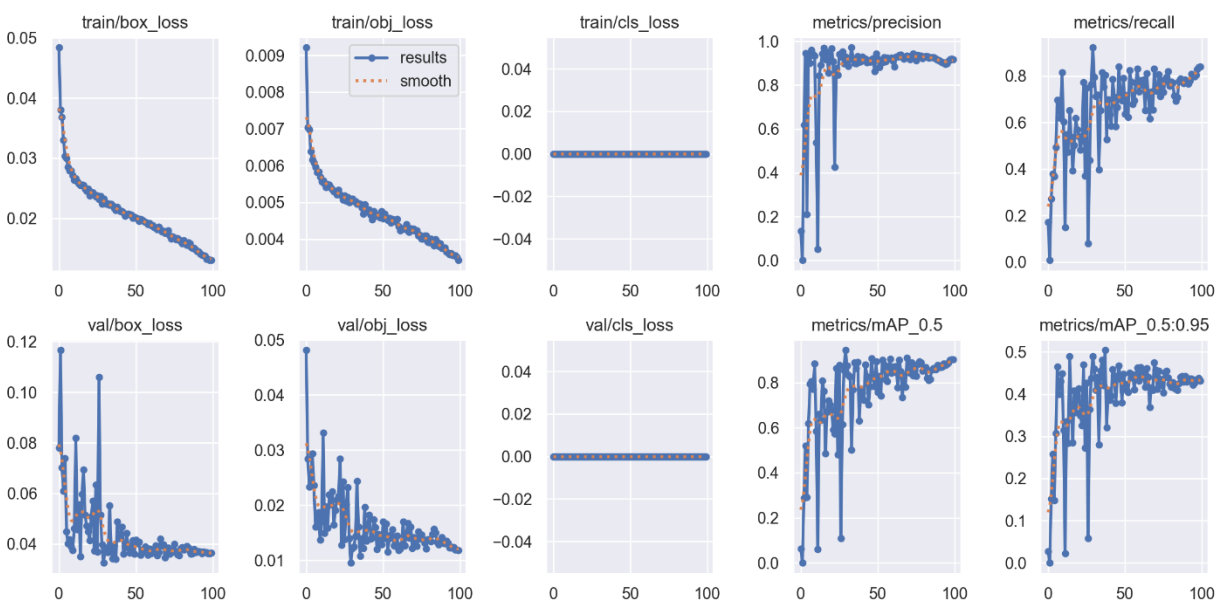
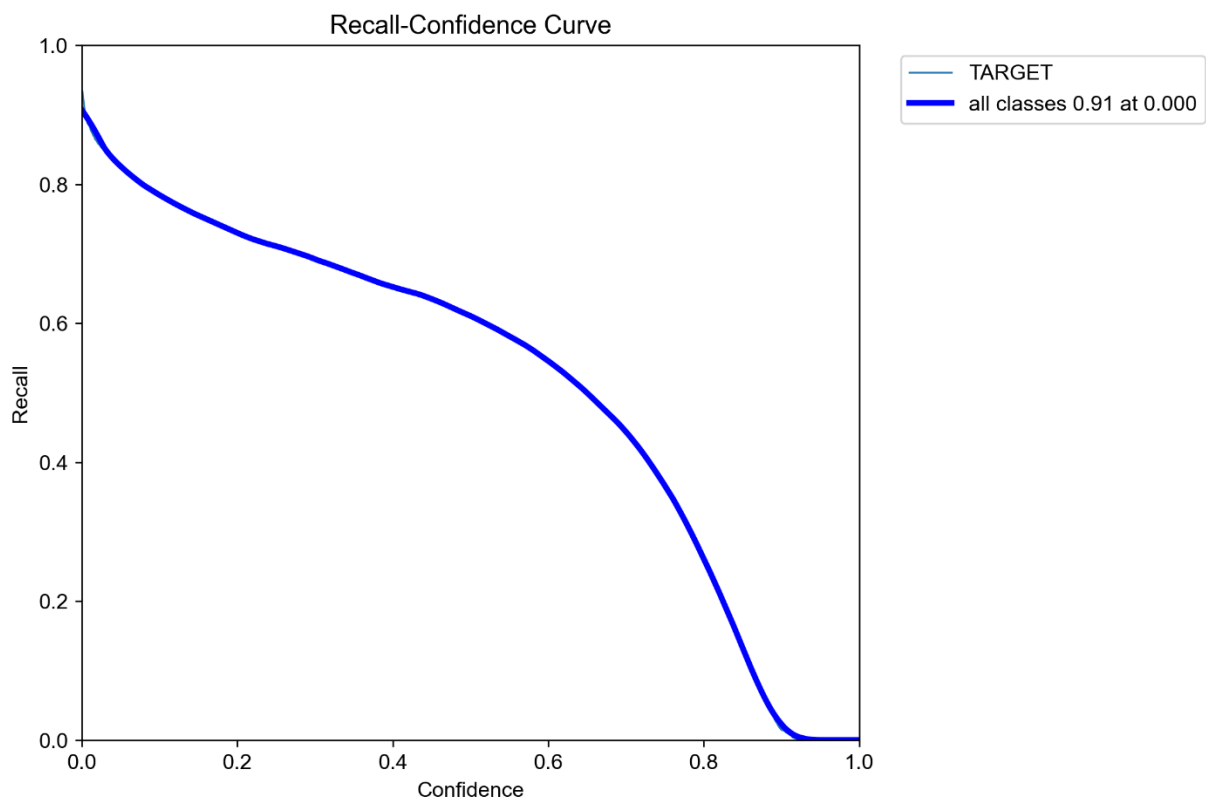
(yolov5) C:\Users\mudno\yolov5>python val.py --weights runs/train/exp3/weights/best.pt --data "D:/UNIVERSITY/YR3/FRA361_Open_Topic/DATASET/For Train/FRA361_OpenTopic_Aimbot.vli.yolov5pytorch/data.yaml" --img 640
val: data=D:/UNIVERSITY/YR3/FRA361_Open_Topic/DATASET/For Train/FRA361_OpenTopic_Aimbot.vli.yolov5pytorch/data.yaml, weights=['runs/train/exp3/weights/best.pt'], batch_size=32, imgsiz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs\val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 v7.0-399-g8cc44963 Python-3.8.20 torch-2.1.1+cu121 CUDA:0 (NVIDIA GeForce RTX 3060 Laptop GPU, 6144MiB)

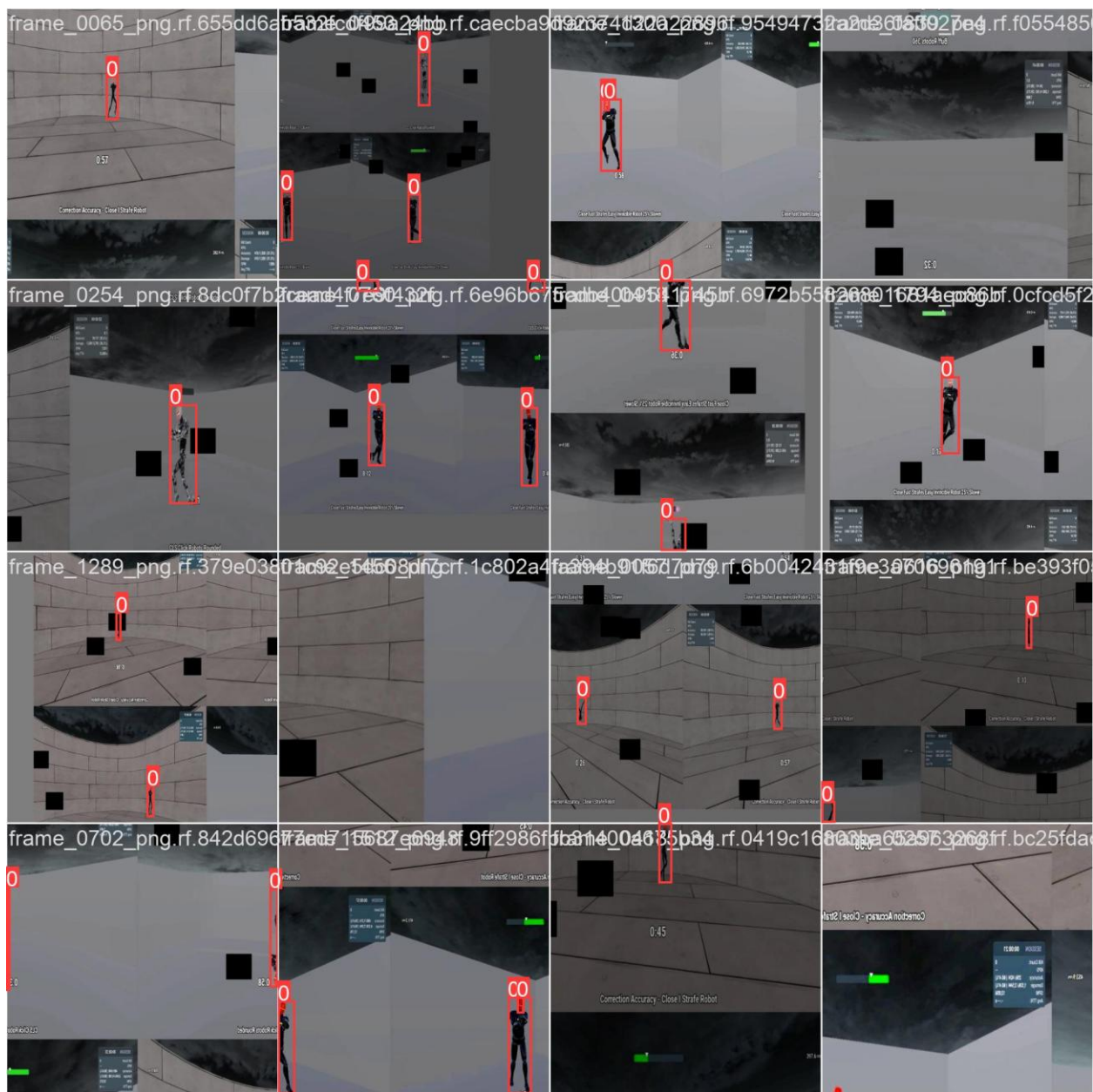
Fusing layers...
YOLOv5s summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs
val: Scanning D:/UNIVERSITY/YR3/FRA361_Open_Topic/DATASET/For Train/FRA361_OpenTopic_Aimbot.vli.yolov5pytorch/data.yaml
      Class      Images  Instances      P      R      mAP50  mAP50-95: 100%|
      all        1042      2640      0.933      0.802      0.891      0.504

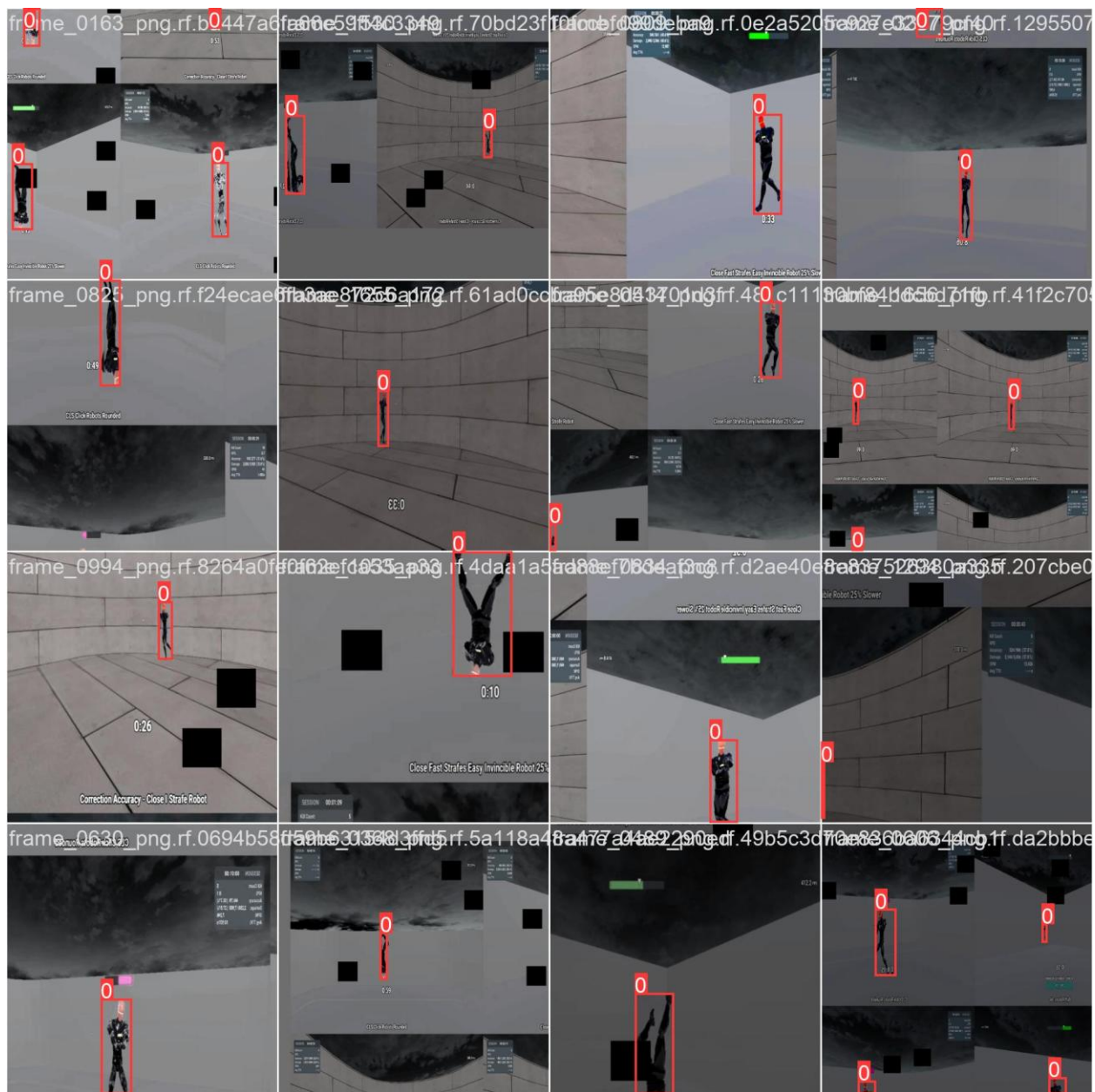
Speed: 0.3ms pre-process, 4.8ms inference, 1.7ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs\val\exp

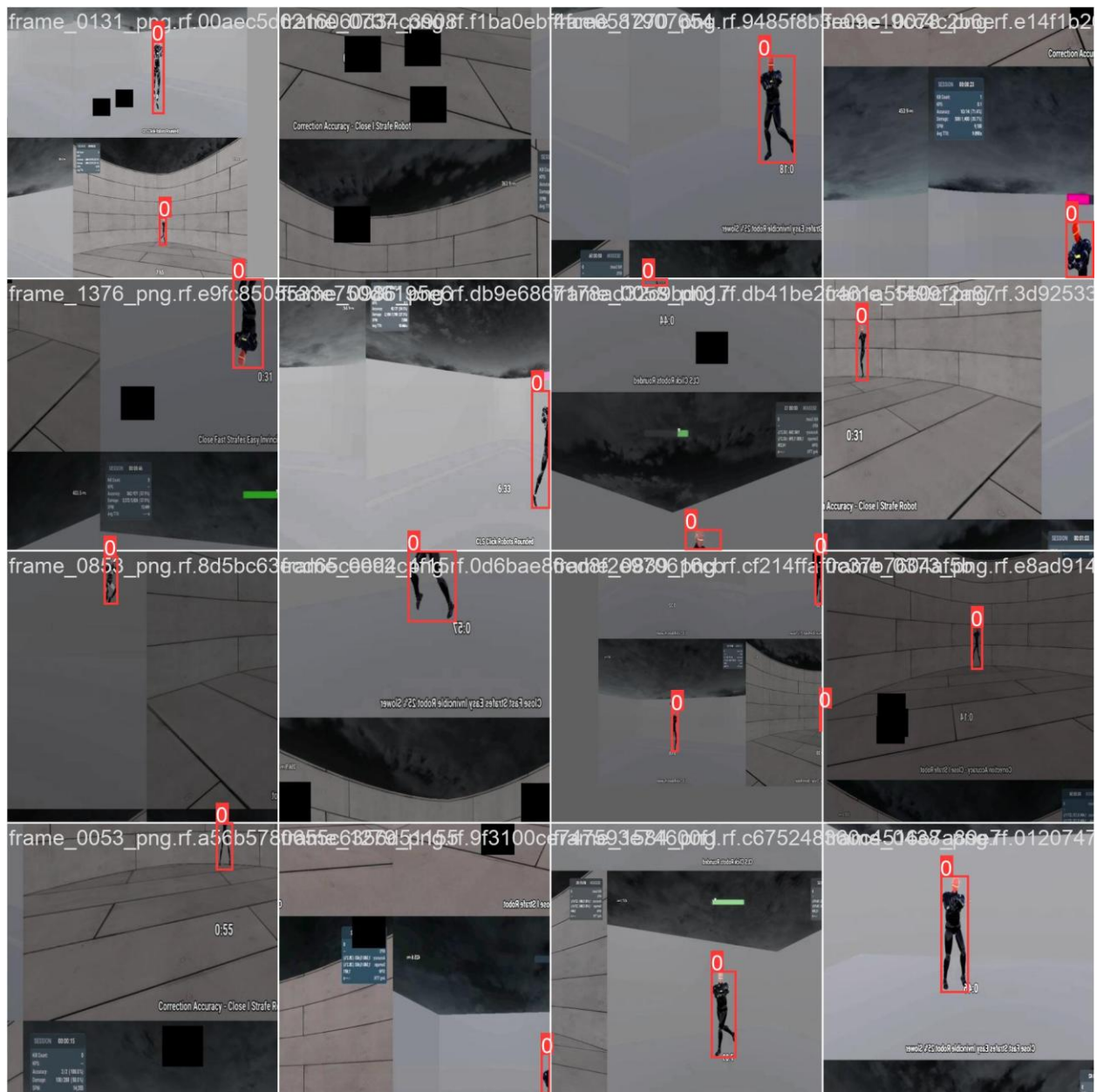
```







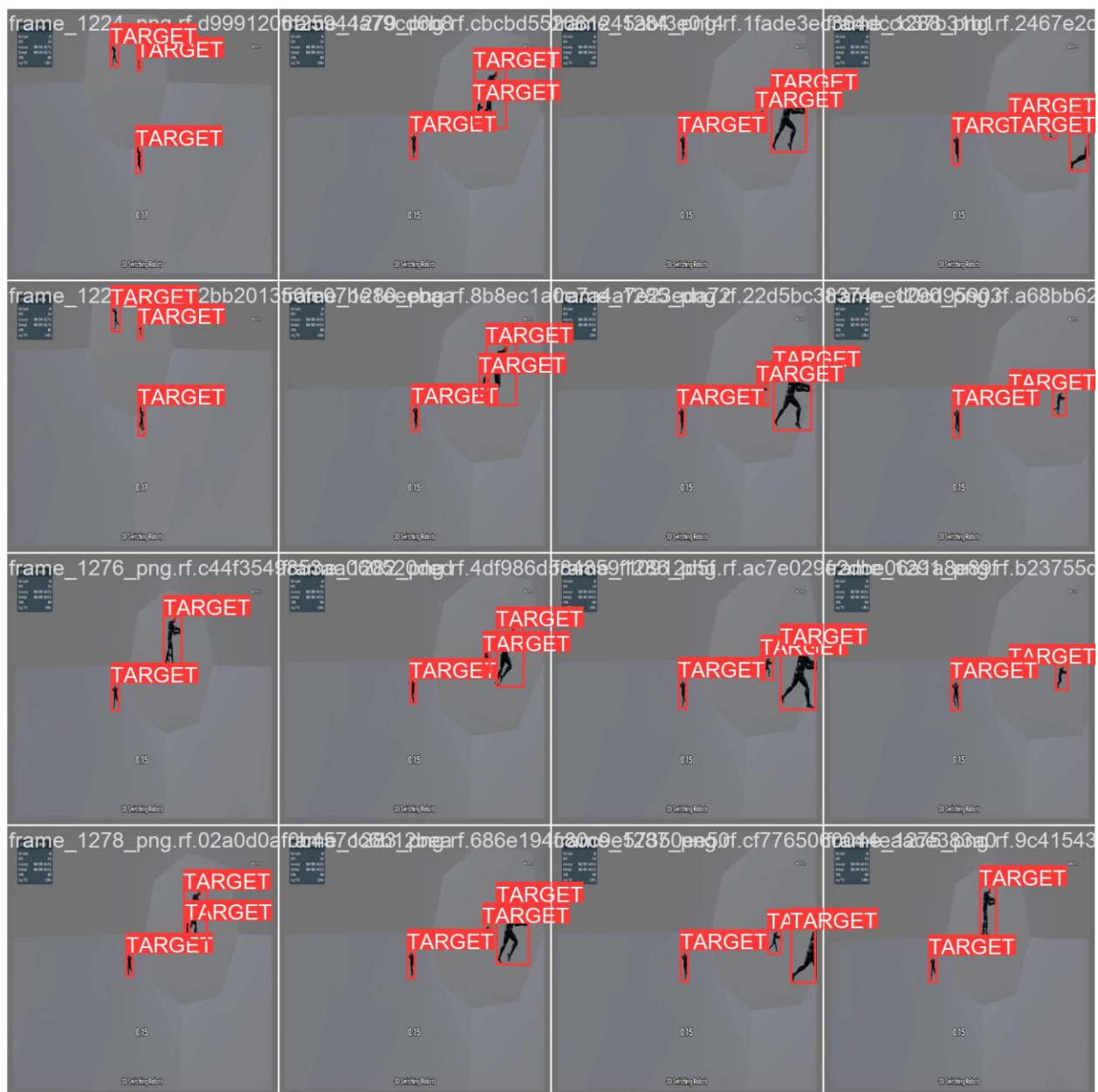




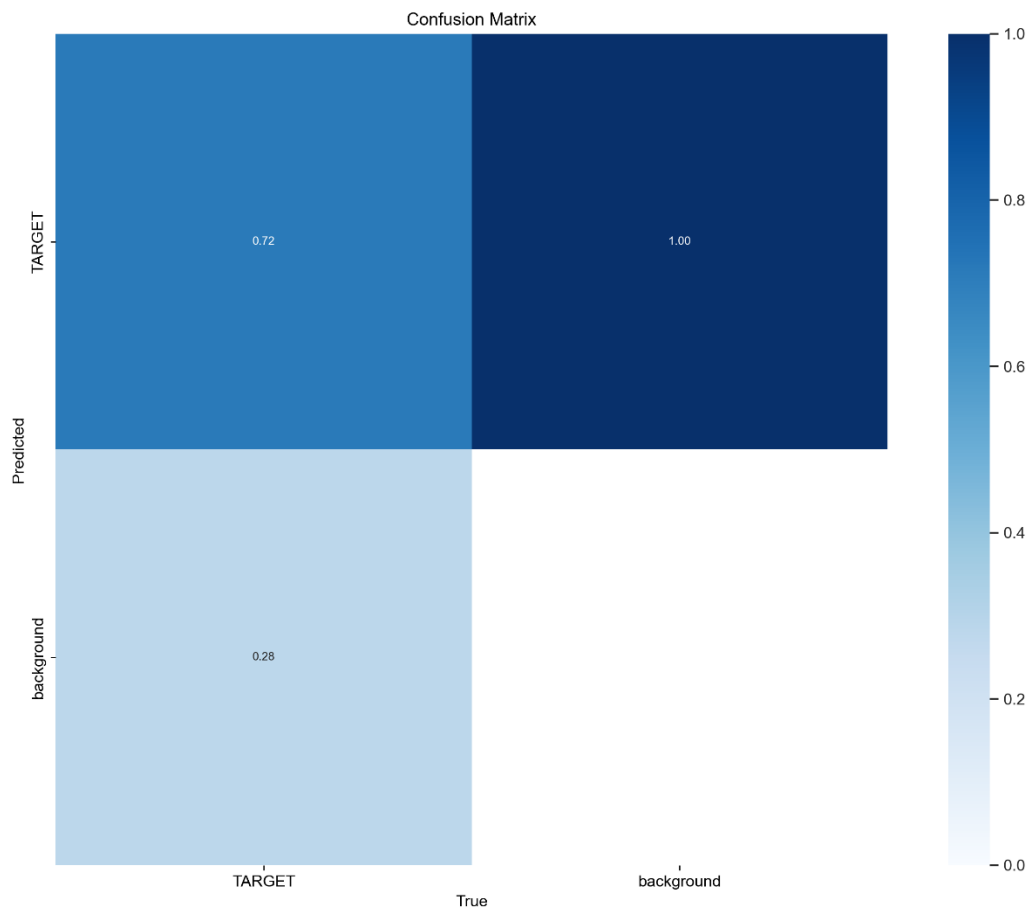


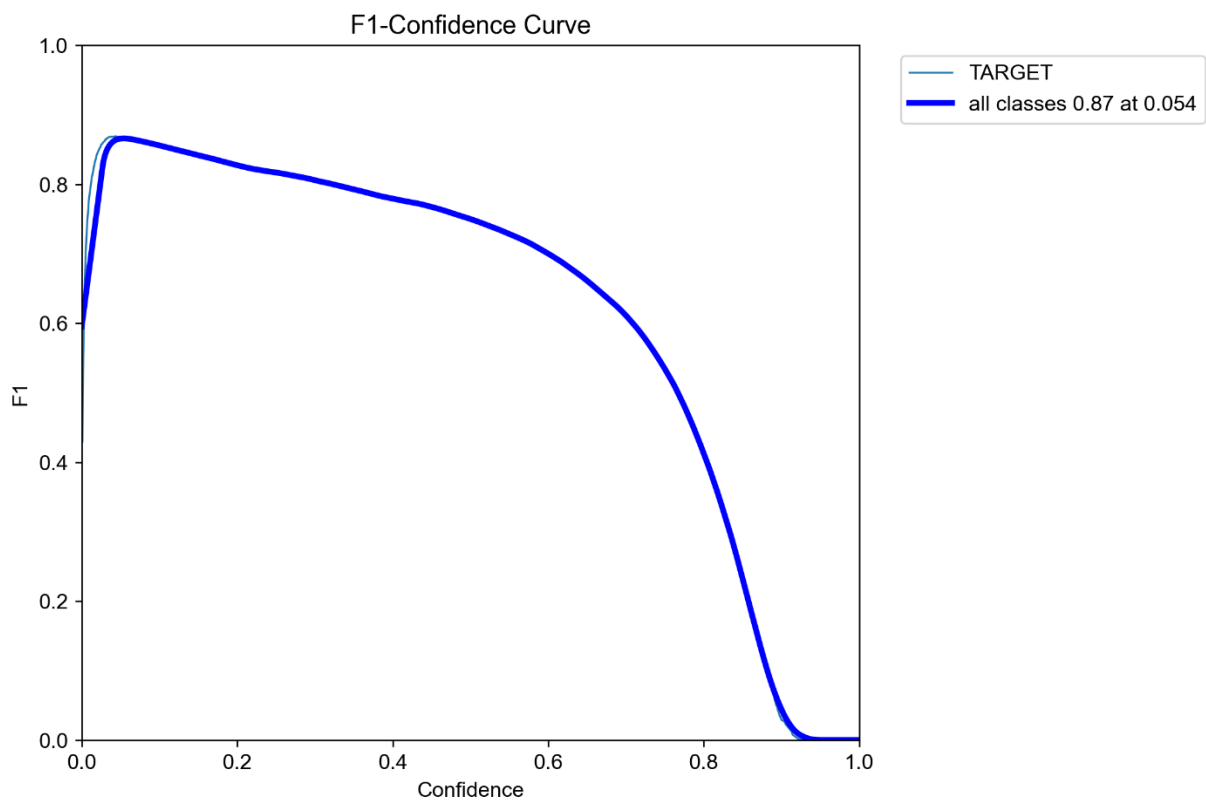


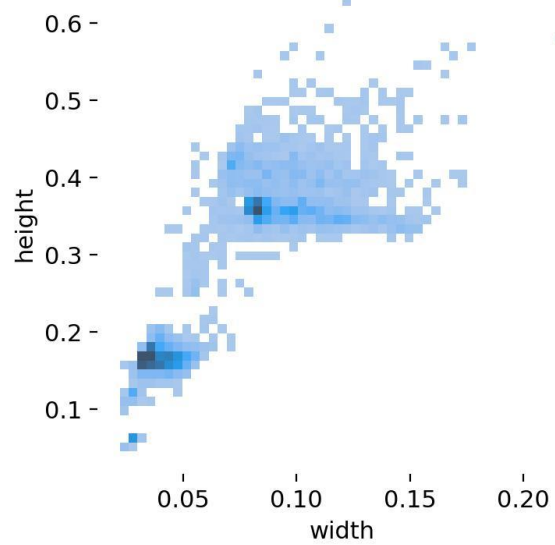
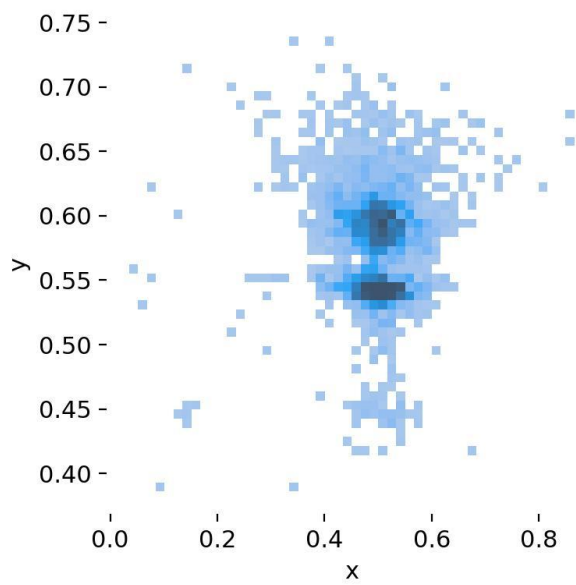
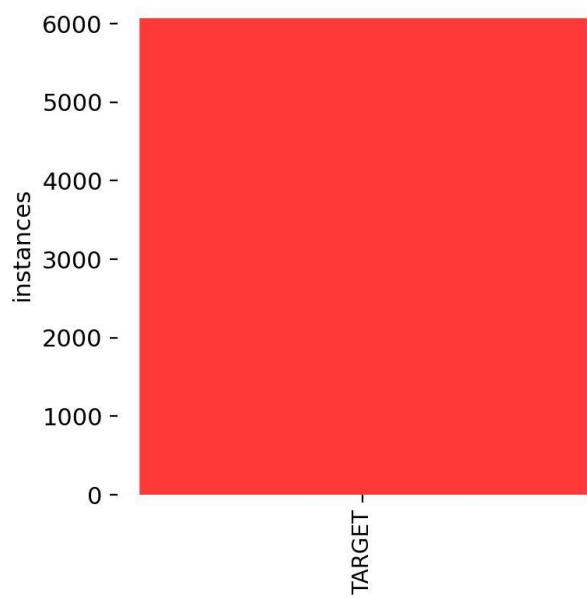


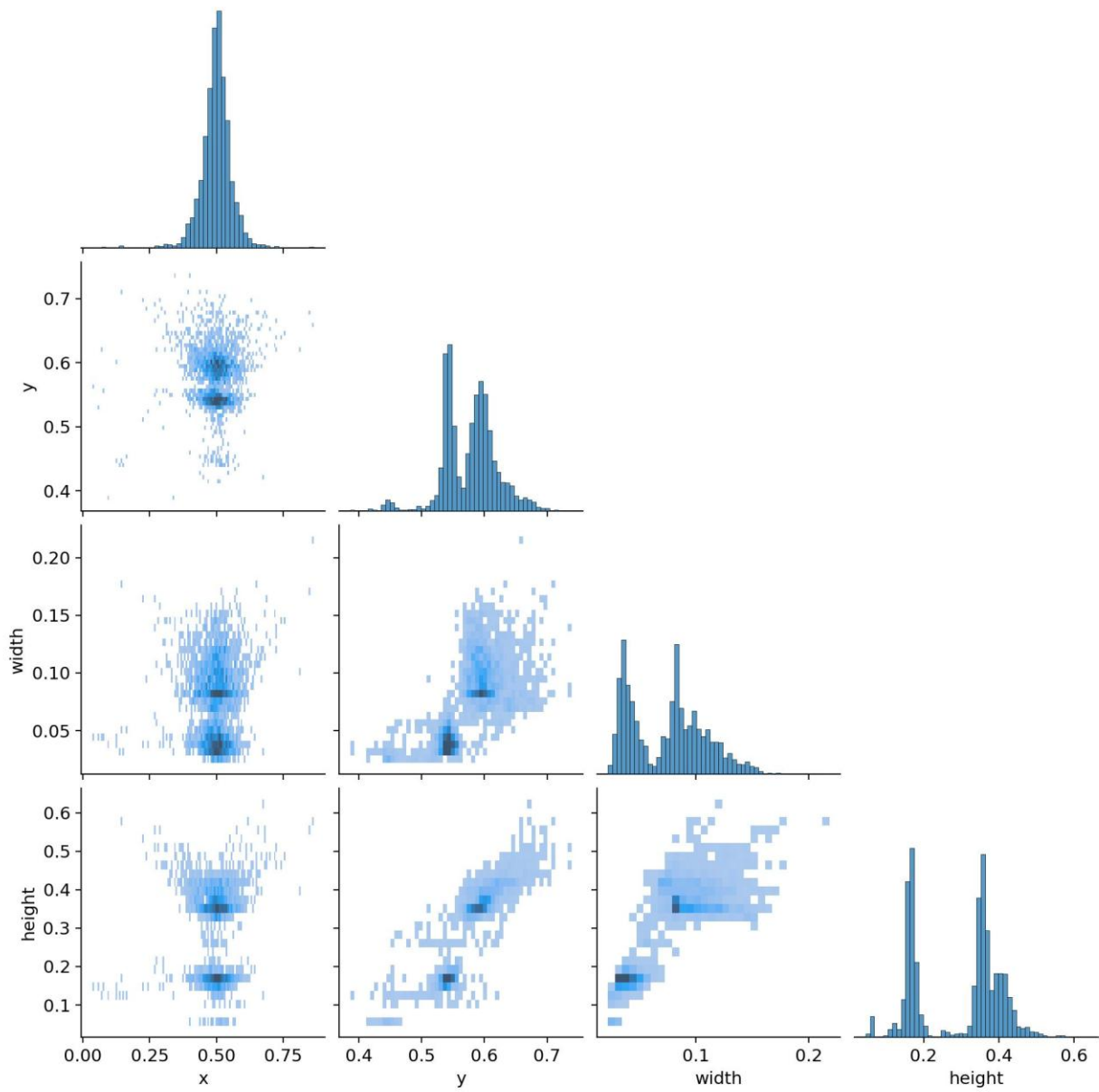












รูป ขั้นตอนการ Train และผลลัพธ์จากการ Train โมเดล Small V2 และ Medium


```

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
112/199   11G      0.02485  0.004743  0          24          640: 100% ██████████ 1514/1514 [05:13<00:00, 4.82it/s]
Class    Images  Instances  P          R          mAP50  mAP50-95: 100% ██████████ 22/22 [00:06<00:00, 3.29it/s]
all      1042    1961      0.842      0.519      0.68    0.187
Stopping training early as no improvement observed in last 100 epochs. Best results observed at epoch 12, best model saved as best.pt.
To update EarlyStopping(patience=100) pass a new patience value, i.e. 'python train.py --patience 300' or use '--patience 0' to disable EarlyStopping.

113 epochs completed in 10.043 hours.
Optimizer stripped from runs/train/yolov5m6/weights/last.pt, 42.2MB
Optimizer stripped from runs/train/yolov5m6/weights/best.pt, 42.2MB

Validating runs/train/yolov5m6/weights/best.pt...
Fusing layers...
YOLOv5m summary: 212 layers, 20852934 parameters, 0 gradients, 47.9 GFLOPs
Class    Images  Instances  P          R          mAP50  mAP50-95: 100% ██████████ 22/22 [00:09<00:00, 2.33it/s]
all      1042    1961      0.843      0.602      0.747    0.313
Results saved to runs/train/yolov5m6
(mud_yolov5) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$

Epoch   GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
109/149   8.16G    0.02343  0.004562  0          15          640: 100% ██████████ 1136/1136 [02:55<00:00, 6.48it/s]
Class    Images  Instances  P          R          mAP50  mAP50-95: 100% ██████████ 17/17 [00:05<00:00, 3.33it/s]
all      1042    1961      0.914      0.448      0.664    0.257
Stopping training early as no improvement observed in last 100 epochs. Best results observed at epoch 9, best model saved as best.pt.
To update EarlyStopping(patience=100) pass a new patience value, i.e. 'python train.py --patience 300' or use '--patience 0' to disable EarlyStopping.

110 epochs completed in 5.617 hours.
Optimizer stripped from runs/train/yolov5s_v22/weights/last.pt, 14.4MB
Optimizer stripped from runs/train/yolov5s_v22/weights/best.pt, 14.4MB

Validating runs/train/yolov5s_v22/weights/best.pt...
Fusing layers...
YOLOv5s summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs
Class    Images  Instances  P          R          mAP50  mAP50-95: 100% ██████████ 17/17 [00:07<00:00, 2.36it/s]
all      1042    1961      0.89       0.653      0.789    0.381
Results saved to runs/train/yolov5s_v22
(yolov5_py310) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ 0;10;1c0;10;1cclient_loop: send disconnect: Connection reset
PS C:\Users\mudno>

```

```

) (1.17.0)
Downloading pandas-1.5.3-cp310-cp310-manylinux_2_17_x86_64_manylinux2014_x86_64.whl (12.1 MB)
----- 12.1/12.1 MB 66.1 MB/s eta 0:00:00

Installing collected packages: pandas
Successfully installed pandas-1.5.3
(mud_yolov5) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ python val.py --weights runs/train/yolov5m6/weights/best.pt --data
~/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot_v2i.yolov5pytorch/data.yaml --task val
val: data=/home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot_v2i.yolov5pytorch/data.yaml, weights=['runs/train/yolov5m6/weights/best.pt'], batch_size=32,
imgsz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False,
save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 v7.0-418-ga493afe1 Python-3.10.16 torch-2.0.1+cu118 CUDA:0 (Tesla V100-SXM2-32GB, 32501MiB)

Fusing layers...
YOLOv5m summary: 212 layers, 20852934 parameters, 0 gradients, 47.9 GFLOPs
val: Scanning /home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot_v2i.yolov5pytorch/valid/labels.cache... 1042 images, 26 backgrounds, 0 corrupt
Class    Images  Instances  P          R          mAP50  mAP50-95: 100% ██████████ 33/33 [00:11<00:00, 2.83it/s]
all      1042    1961      0.843      0.602      0.748    0.312
Speed: 0.2ms pre-process, 4.7ms inference, 1.0ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs/val/exp
(mud_yolov5) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$

Found existing installation: pandas 2.2.3
Uninstalling pandas-2.2.3:
  Successfully uninstalled pandas-2.2.3
Successfully installed pandas-1.5.3
(yolov5_py310) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ python val.py --weights runs/train/yolov5s_v22/weights/best.pt
--data ~/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot_v2i.yolov5pytorch/data.yaml --task val
val: data=/home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot_v2i.yolov5pytorch/data.yaml, weights=['runs/train/yolov5s_v22/weights/best.pt'], batch_size=32,
imgsz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False,
save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 v7.0-418-ga493afe1 Python-3.10.16 torch-2.0.1+cu118 CUDA:0 (Tesla V100-SXM2-32GB, 32501MiB)

Fusing layers...
YOLOv5s summary: 157 layers, 7012822 parameters, 0 gradients, 15.8 GFLOPs
val: Scanning /home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot_v2i.yolov5pytorch/valid/labels.cache... 1042 images, 26 backgrounds, 0 corrupt
Class    Images  Instances  P          R          mAP50  mAP50-95: 100% ██████████ 33/33 [00:08<00:00, 3.82it/s]
all      1042    1961      0.885      0.652      0.789    0.38
Speed: 0.2ms pre-process, 2.0ms inference, 1.0ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs/val/exp2
(yolov5_py310) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$

```

รูป ขั้นตอนการ Train และผลลัพธ์จากการ Train โมเดล Large และ Extra


```

all 1042 1961 0.123 0.263 0.148 0.0326

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
57/199 11G 0.02454 0.004922 0 16 640: 100%| 2271/2271 [07:48<00:00,
Class Images Instances P R mAP50 mAP50-95: 100%| 33/33 [00:06<
all 1042 1961 0.124 0.277 0.152 0.0335

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
58/199 11G 0.02452 0.004895 0 16 640: 100%| 2271/2271 [07:48<00:00,
Class Images Instances P R mAP50 mAP50-95: 100%| 33/33 [00:07<
all 1042 1961 0.124 0.287 0.16 0.0347

Stopping training early as no improvement observed in last 50 epochs. Best results observed at epoch 8, best model saved as best.pt.
To update EarlyStopping(patience=50) pass a new patience value, i.e. 'python train.py --patience 300' or use '--patience 0' to disable EarlyStopping.

59 epochs completed in 7.832 hours.
Optimizer stripped from runs/train/yolov5l_v2/weights/last.pt, 92.9MB
Optimizer stripped from runs/train/yolov5l_v2/weights/best.pt, 92.9MB

Validating runs/train/yolov5l_v2/weights/best.pt...
Fusing layers...
YOLOv5l summary: 267 layers, 46188278 parameters, 0 gradients, 107.6 GFLOPs
Class Images Instances P R mAP50 mAP50-95: 100%| 33/33 [00:10<
all 1042 1961 0.823 0.682 0.761 0.255

Results saved to runs/train/yolov5l_v2
(mud_yolov5) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ |

```

```

(mud_yolov5) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ python val.py --weights runs/train/yolov5l_v2/weights/best.pt --data ~/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot.v21.yolov5pytorch/data.yaml --task val
val: data=/home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot.v21.yolov5pytorch/data.yaml, weights=['runs/train/yolov5l_v2/weights/best.pt'], batch_size=32, imgsz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, h
alf=False, dnn=False
YOLOv5 v7.0-418-ga093afe1 Python-3.10.16 torch-2.0.1+cu118 CUDA:0 (Tesla V100-SXM2-32GB, 32501MiB)

Fusing layers...
YOLOv5l summary: 267 layers, 46188278 parameters, 0 gradients, 107.6 GFLOPs
val: Scanning /home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot.v21.yolov5pytorch/valid/labels.cache... 1042 images, 26 backgrounds, 0 corrupt: 100%| 1042/1042 [00:00<, 71it/s]
Class Images Instances P R mAP50 mAP50-95: 100%| 33/33 [00:15<00:00, 2.16it/s]
all 1042 1961 0.821 0.681 0.761 0.256
Speed: 0.2ms pre-process, 0.2ms inference, 1.2ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs/val/exp3
(mud_yolov5) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ |

```

```

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
53/199 12.4G 0.02574 0.005266 0 12 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.0149 0.00765 0.00819 0.00272

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
54/199 12.4G 0.02576 0.005272 0 12 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.0107 0.00714 0.00595 0.00198

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
55/199 12.4G 0.02562 0.005282 0 11 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.00951 0.00765 0.00524 0.0017

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
56/199 12.4G 0.02576 0.00526 0 12 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.00678 0.00714 0.0038 0.00124

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
57/199 12.4G 0.02566 0.005252 0 11 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.00649 0.00765 0.00362 0.00116

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
58/199 12.4G 0.02574 0.005271 0 12 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.00762 0.00816 0.00428 0.00132

Epoch GPU_mem box_loss obj_loss cls_loss Instances Size
59/199 12.4G 0.02566 0.00524 0 12 640: 100%| 3028/3
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.00881 0.00918 0.00494 0.00149

Stopping training early as no improvement observed in last 50 epochs. Best results observed at epoch 9, best model saved as best.pt.
To update EarlyStopping(patience=50) pass a new patience value, i.e. 'python train.py --patience 300' or use '--patience 0' to disable EarlyStopping.

60 epochs completed in 14.226 hours.
Optimizer stripped from runs/train/yolov5x_v2/weights/last.pt, 173.1MB
Optimizer stripped from runs/train/yolov5x_v2/weights/best.pt, 173.1MB

Validating runs/train/yolov5x_v2/weights/best.pt...
Fusing layers...
YOLOv5x summary: 322 layers, 86173414 parameters, 0 gradients, 203.8 GFLOPs
Class Images Instances P R mAP50 mAP50-95: 100%|
all 1042 1961 0.843 0.681 0.771 0.291

Results saved to runs/train/yolov5x_v2
(yolov5.py310) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ |

```

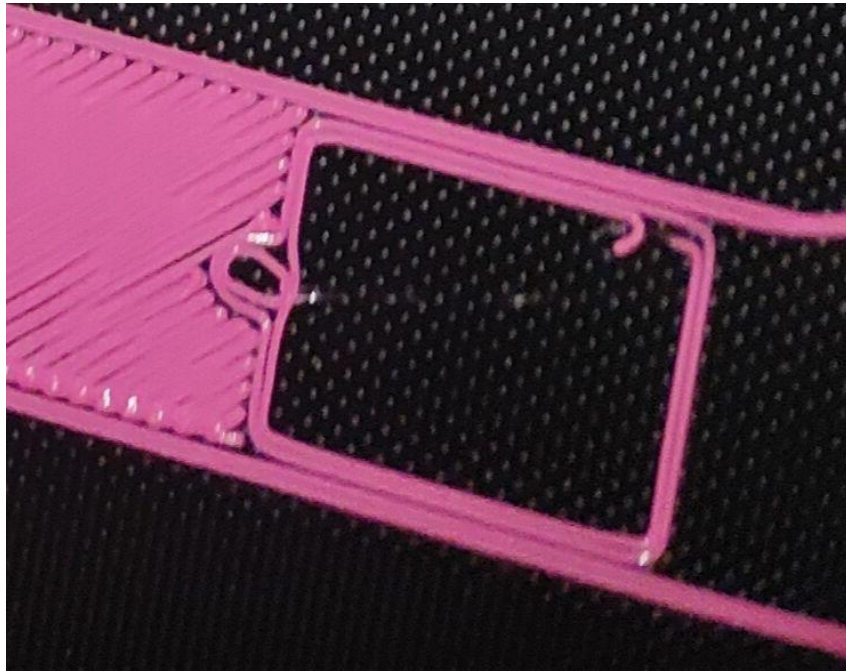
```

val: data=/home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot.v21.yolov5pytorch/data.yaml, weights=['runs/train/yolov5x_v2/weights/best.pt'], batch_size=32, imgsz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, h
alf=False, dnn=False
YOLOv5 v7.0-418-ga093afe1 Python-3.10.16 torch-2.0.1+cu118 CUDA:0 (Tesla V100-SXM2-32GB, 32501MiB)

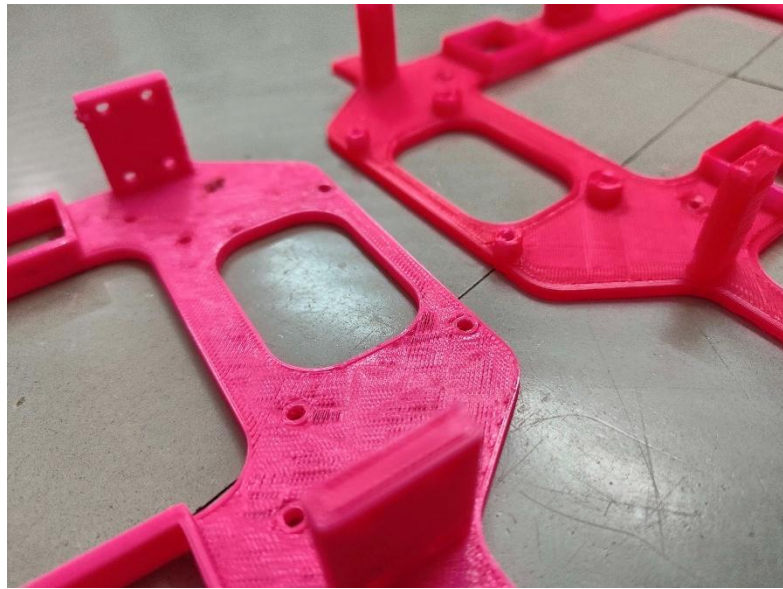
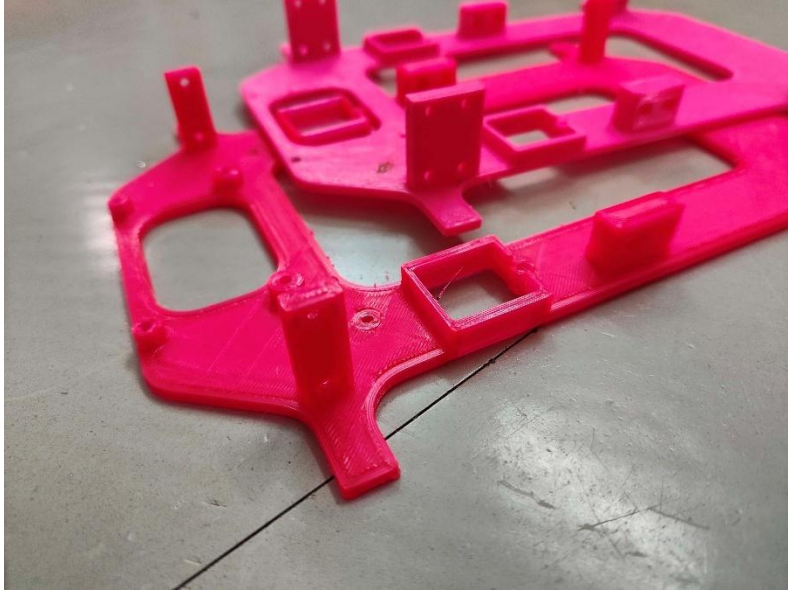
Fusing layers...
YOLOv5x summary: 322 layers, 86173414 parameters, 0 gradients, 203.8 GFLOPs
val: Scanning /home/wasupol/Pollapaat6546_WS/YoloOpen/FRA361_OpenTopic_Aimbot.v21.yolov5pytorch/valid/labels.cache... 1042 images, 26 backgrounds, 0 corrupt: 100%| 1042/1042 [00:00<, 71it/s]
Class Images Instances P R mAP50 mAP50-95: 100%| 33/33 [00:22<00:00, 1.49it/s]
all 1042 1961 0.844 0.681 0.77 0.29
Speed: 0.2ms pre-process, 14.6ms inference, 1.3ms NMS per image at shape (32, 3, 640, 640)
Results saved to runs/val/exp5
(yolov5.py310) wasupol@EDU3-Standard-PC-i440FX-PIIX-1996:~/Pollapaat6546_WS/YoloOpen/yolov5$ |

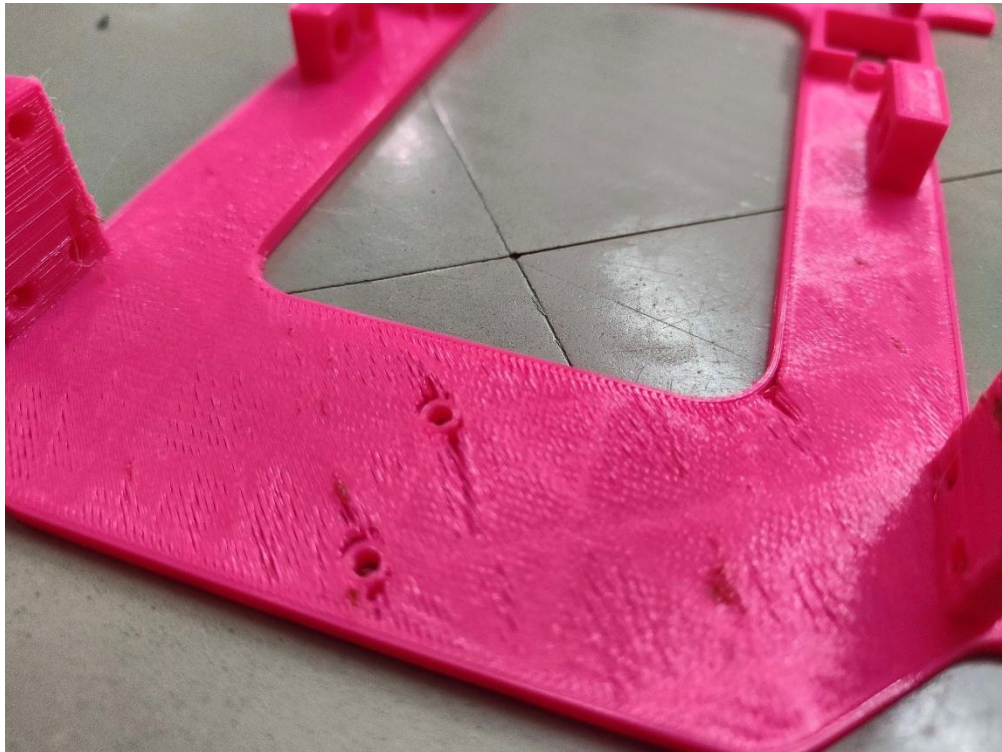
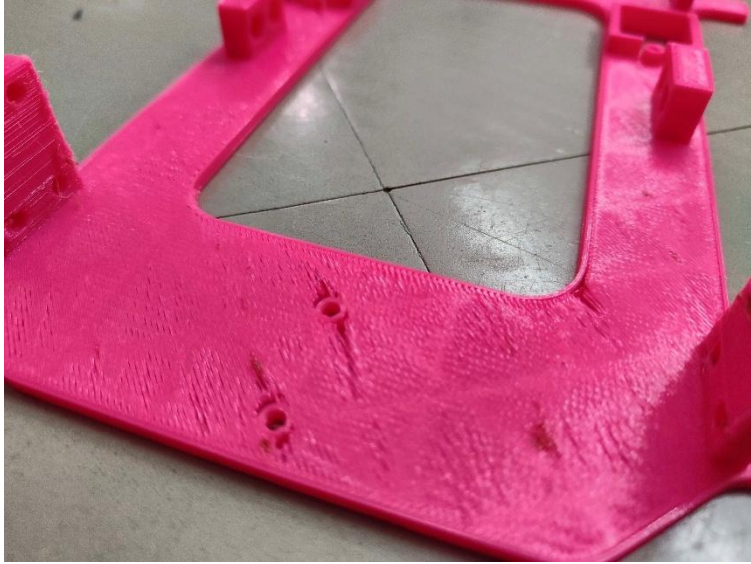
```

รูป เปรียบเทียบชิ้นส่วน 3D Print เก่าที่มีปัญหาและใหม่



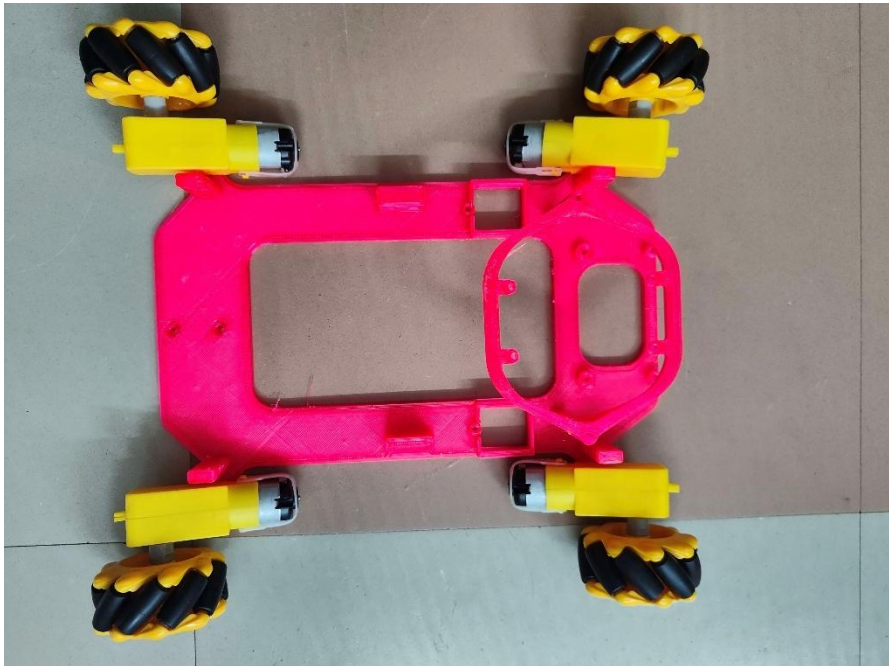


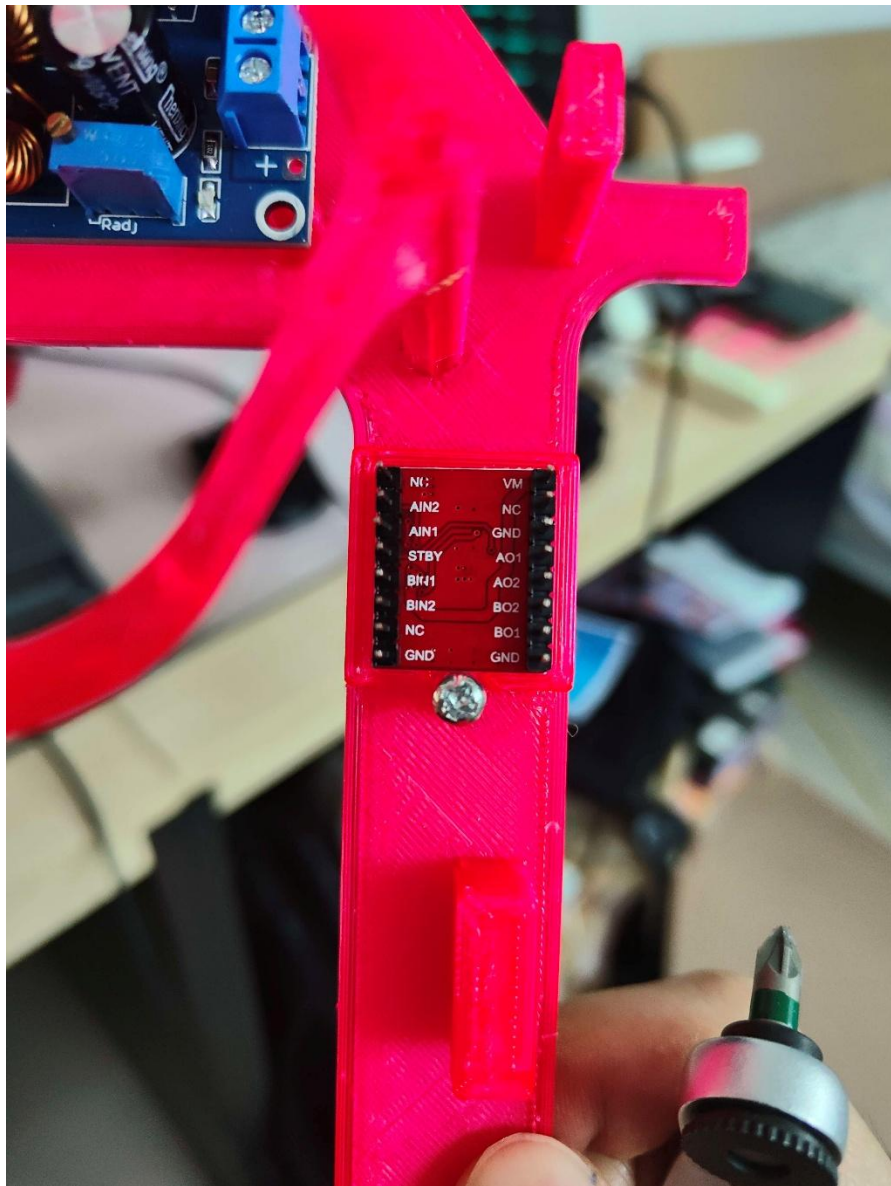




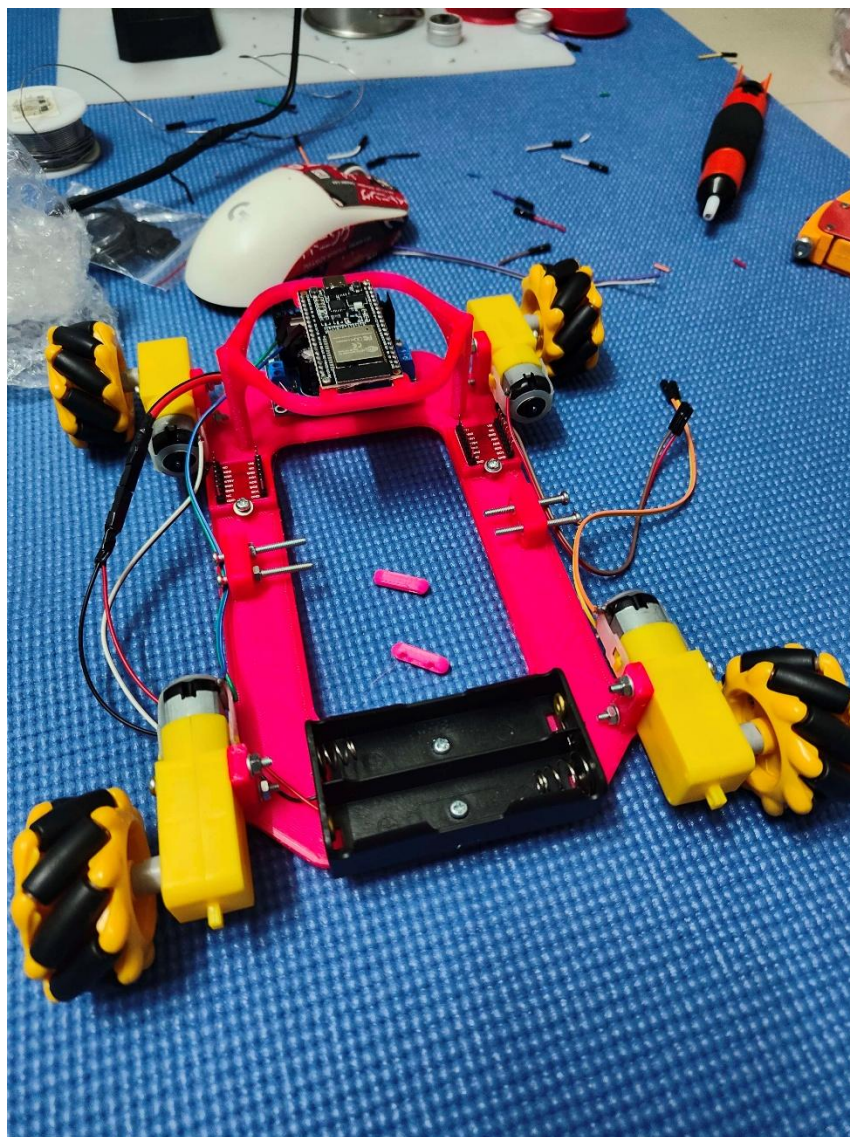


รูป ตัวหุ่นระหว่างทำการประกอบ

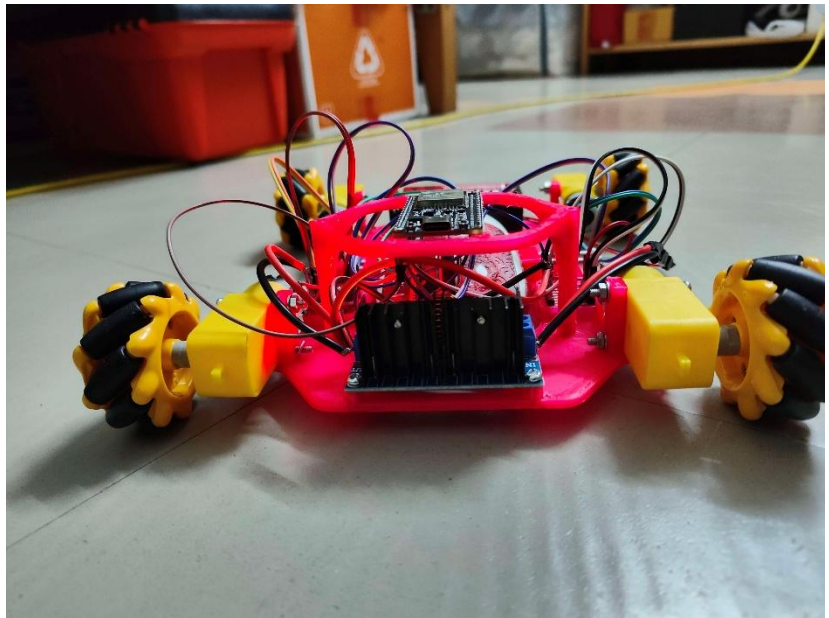
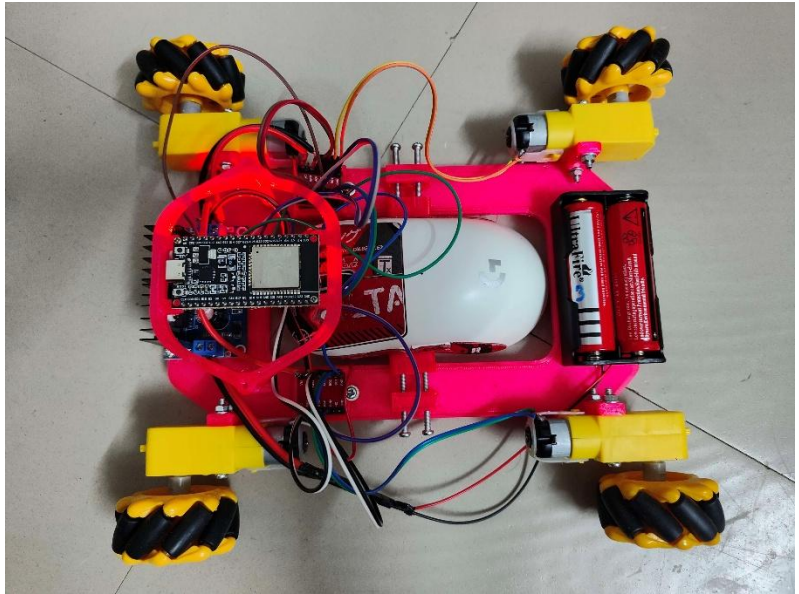


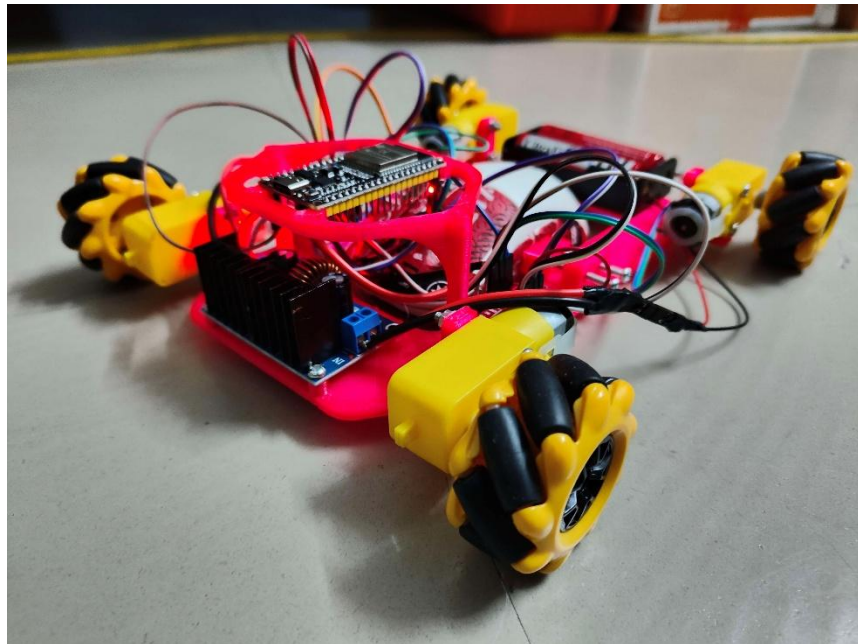
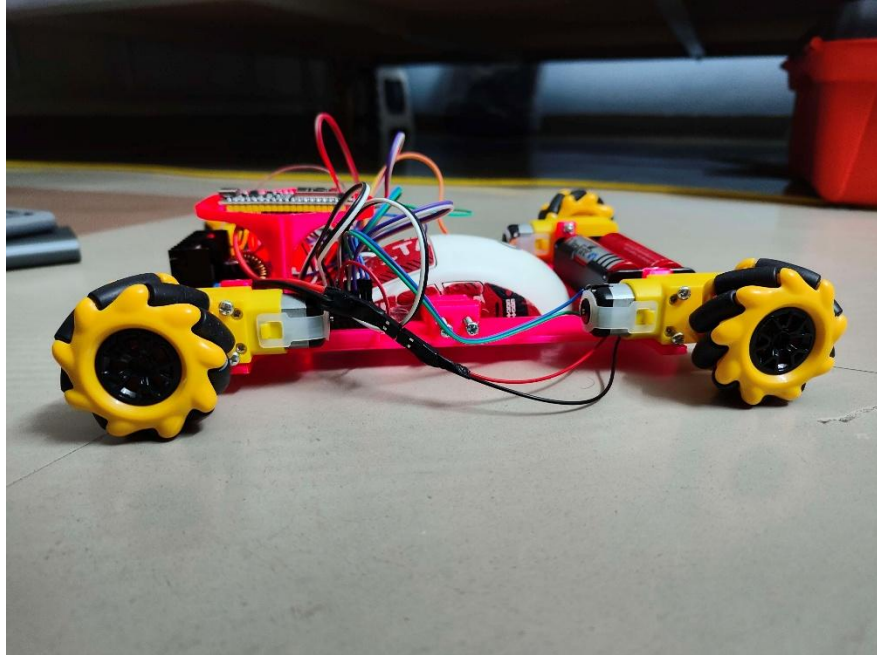


NI:	VM
AIN2	NC
AIN1	GND
STBY	AO1
BIK1	AO2
BIN2	BO2
NC	BO1
GND	GND



รูป ตัวหุ่นเมื่อประกอบเสร็จ





รูป กราฟ log ค่าระหว่างทดสอบการ Tune PID

